MycorrhizaFinder

MycorrhizaFinder
Full User Documentation

Tool version: V5.0.0

Royal Botanic Gardens Kew | NCEA Natural Capital and Ecosystem Assessment

SUMMARY

This document is intended as an exhaustive overview of the functionality of MycorrhizaFinder. If you are new to the tool, we recommend starting with the Quick-Start User Guide.

MycorrhizaFinder allows for high-throughput computer vision-based identification and quantification of Arbuscular (AM) and Ericoid (ErM) Mycorrhizal fungal colonisation using convolutional neural networks.

If you use MycorrhizaFinder, or a derivative of MycorrhizaFinder, in your research, please cite:

> Kowal, J., Upham, R., Kiani, A., Rickards, M., Serpell, E., Bidartondo, M.I., Evangelisti, E., Schornack, S., Sibbit, J., Treder, K.P., Weidinger, S. and Suz, L.M., 2026, *in prep*.

This version of the MycorrhizaFinder tool has been developed by the Royal Botanical Gardens Kew through the Natural Capital and Ecosystem Assessment (NCEA) programme. The NCEA is Defra's largest research and development programme, it is generating a robust evidence base of the location, extent and condition of our natural capital and ecosystems, and how the state of nature is changing over time. The original tool was developed by the Sainsbury Laboratory, University of Cambridge (see Evangelisti et al. 2021, https://doi.org/10.1111/nph.17697).

CONTENTS

01. TOOL INSTALLATION

In order to install the tool for use, there are only two prerequisites – you must have the correct executable downloaded for your Operating System (OS), and you must also have Postgres installed, which is the database the tool uses to store data. The current version of the tool has executables for Windows, MacOS and Linux operating systems. The Postgres database is the local solution for storing annotations and predictions. Instructions on how to install Postgres and run the executable are included below.

01.1 INSTALLING POSTGRES

You can install Postgres via the official website here. Select the relevant OS and follow the download instructions.

One thing to note here is that MycorrhizaFinder defaults to the postgres user having the password postgres. If you choose a different password when installing the program (which you will be prompted to do when downloading Postgres), then the best way to override this default is by editing a file in the _internal folder called “database.ini” (this may show up as just “database” if file extensions are hidden). Change the “password” field to the password chosen in the Postgres setup and save the file, making sure that you overwrite the old “database.ini” file and don’t accidentally append a “.txt” file extension. For example, in Windows Notepad you should choose the file type “All files” and filename “database.ini”. An alternative approach to specify the password is to do it by passing a flag --db-password to the executable when you run it, but you will have to do this every single time you spin up the program. The relevant command is below – you will need to run this via cmd prompt/terminal, depending on your OS.

```
./amf.exe --db-password example-password
```

01.2 RUNNING THE EXECUTABLE

Once you have downloaded Postgres and started it on your machine (this should be automatic), if you have set the postgres user password to postgres as outlined above you can run the tool by simply double clicking the executable for your OS. This will automatically open a command line interface that is running the tool, and will also open a web browser window at the address http://127.0.0.1:8001 - you are now ready to use the tool and all its functionalities!

To stop the tool, just close the command line interface. The interface in the web browser will then stop working, so you can also close this.

Note: *the executable comes with a folder called _internal. This* ***must*** *stay in the same folder as the executable, i.e. the exe and the _internal folder must always stay together.*

01.3 INSTALLATION NOTES

A simple way to access the executable for daily use is to create a desktop shortcut. In order to do this in Windows, right click on the executable and select “Send to”. Then select “Desktop (create shortcut)”. You will now see a shortcut on your desktop. You can also pin the executable to your taskbar.

Another thing to note is that if you experience the application freezing or getting stuck loading on Windows, this is likely an issue with the cmd prompt in Windows and not the application! Cmd prompt in windows has an option called "mark mode" that intentionally freezes the buffer to allow you to copy text, and if you click into the window that the exe opens it might trigger this. In order to disable this, you can turn off Quick Edit mode in the console settings - this post will show you how to do that. You can also always unfreeze the exe by clicking the cmd prompt and hitting enter. Also note that if you click into the exe window or highlight anything while the tool is running, the same thing will happen – once again, the fix is to hit enter.

Very occasionally, if you do not explicitly trust the application when first opening it, then Windows Defender could block the running of the application. To circumnavigate this, you can add the executable as an exception in Windows Defender. If you are having issues with the zip file, then right-click on the zip and click “Scan with Microsoft Defender…” (similarly you can do the same for the exe). You can then click on “Allowed threats” to allow the executable to be run. This is a rare case, and it is unlikely you will need to do this.

01.4 USING YOUR GPU

The tool can function using your CPU, but if your computer has a GPU then using this can result in much quicker performance for certain aspects of the tool, e.g. calculating predictions. The tool will default to using your GPU, if possible, otherwise it will use the CPU – you can change this default by altering the settings outlined here.

The minimum supported CUDA version for GPUs is 12.1. To check whether your GPU matches this requirement, you can open a cmd prompt and type `nvidia-smi` – you should see an output like the below.

```
PS C:\Users\d80105> nvidia-smi
Wed Mar 26 10:46:03 2025
+---------------------------------------------------------------------------------------+
| NVIDIA-SMI 538.78                 Driver Version: 538.78       CUDA Version: 12.2     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                     TCC/WDDM  | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  NVIDIA RTX A500 Laptop GPU   WDDM  | 00000000:03:00.0 Off |                  N/A |
| N/A   69C    P8               5W /  30W |    632MiB /  4096MiB |      0%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+

+---------------------------------------------------------------------------------------+
| Processes:                                                                            |
|  GPU   GI   CI        PID   Type   Process name                            GPU Memory |
|        ID   ID                                                             Usage      |
|=======================================================================================|
|    0   N/A  N/A     13324      C   ...0_x64__qbz5n2kfra8p0\python3.11.exe    N/A      |
+---------------------------------------------------------------------------------------+
```

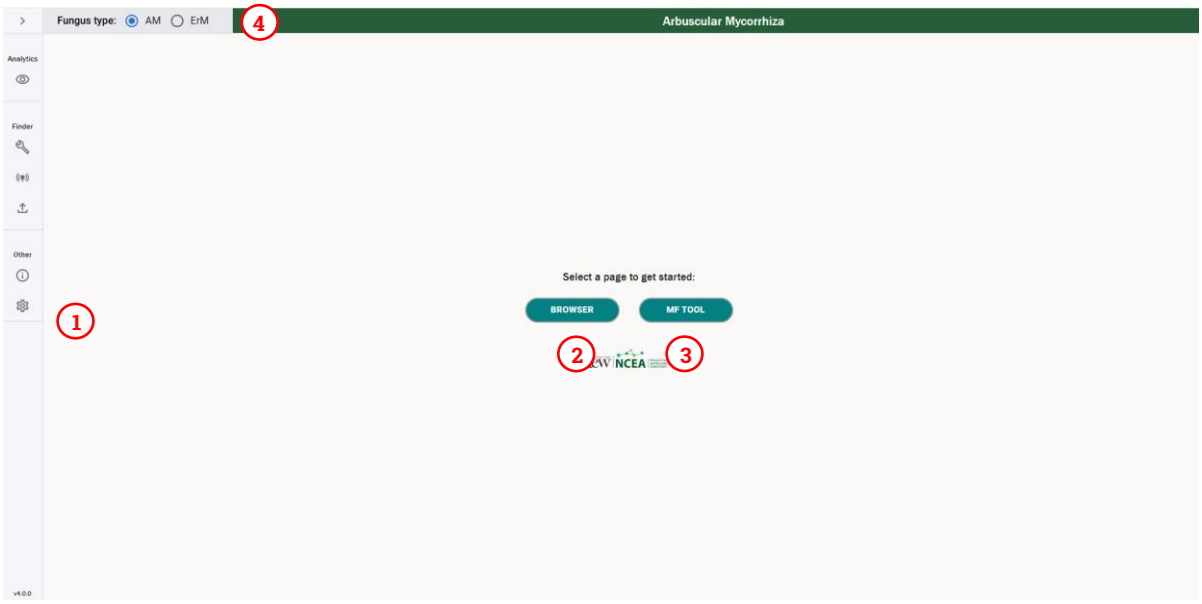
Your CUDA version can be seen in the top right. If this version is < 12.1, you will need to upgrade – the CUDA toolkit download can be found here.

02. MYCORRHIZA FINDER TOOL OVERVIEW

Mycorrhiza Finder is split into two main sections – Browser and Finder. These two functionalities handle the modelling aspects of the tool and the interaction with tool outputs respectively.

02.1 HOMEPAGE

When starting the tool, a user is greeted with the following screen, which is hosted at http:///127.0.0.1:8001.



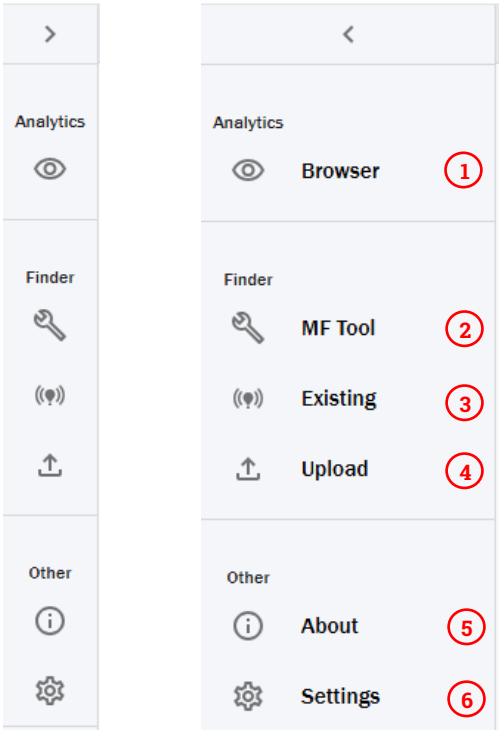
1. This is the navigation sidebar. This allows you to navigate between sections of the tool using the icons and can be opened/closed using the arrow at the top. This is present in all pages of the tool and is detailed further here.
2. This is the Browser section of the tool, which you can use to browse predictions and annotations for a particular image. Further details can be found here.
3. This is the MF Tool section, which is used to interact with the modelling side of the tool. Further details can be found here – on a high level, this section of the tool encompasses:
    a. Calculating predictions and converting these to annotations,
    b. Training and testing new versions of the tool,
    c. Converting MRXS files to JPGs,
    d. Calculating summary metrics on the colonisation of a root,
    e. Viewing and downloading existing annotations and predictions,
    f. Uploading annotations and predictions to be viewed in the tool.
4. This is the colonisation type toggle, where a user can switch between Arbuscular (AM) and Ericoid (ErM) Mycorrhiza. This changes the functionality of various aspects of the tool, including changing the Browser to only view AM or ErM roots,

and also switching the functionality of the tool to calculate predictions for AM or ErM. Further details on this can be found here.

Note that throughout the use of the tool, the main way the application links annotations and predictions to an image is via the **image name.** Therefore, if you calculate predictions or annotations for a particular image, and later down the line you change the images name, when you load the image back into the tool it will not be able to find the previous annotations or predictions. Please be very conscious of this when using the tool!

02.2 NAVIGATING THE TOOL

You can use the left sidebar to navigate between different functionalities of the tool. Below we can see both the closed and opened versions of the sidebar.



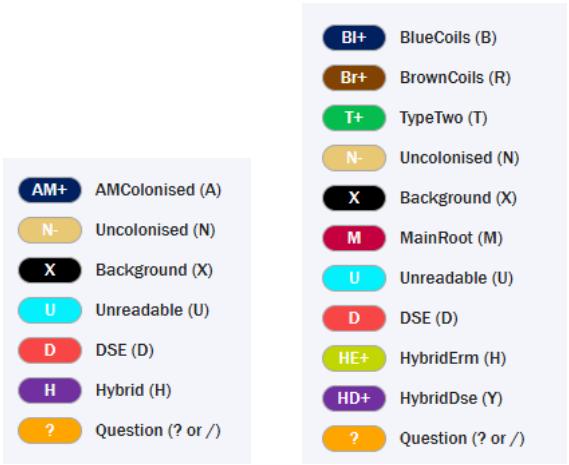
1. **Browser** – where a user can view annotations and predictions.
2. **MF Tool** – where a user can interact with the model.
3. **Existing** – where a user can view existing saved annotations and predictions.
4. **Upload** – where a user can upload CSVs of annotations and predictions.
5. **About** – where a user can view further information about the tool.
6. **Settings** – where a user can edit the global settings for the tool.

02.3 SWITCHING BETWEEN ARBUSCULAR AND ERICOID

As previously mentioned, the tool has two modes – AM (Arbuscular Mycorrhiza) and ErM (Ericoid Mycorrhiza). Naturally, the two different modes of the tool are for assessing the colonisation of two different Mycorrhizal types in different roots. The assumption is that any root you pass through the tool will only be colonised by Arbuscular **or** Ericoid Mycorrhiza and not both simultaneously – thus the two functionalities are treated as distinct.

In order to switch between the two different tool modes, we have a toggle at the top of the page that looks like the below.

We see in the above that we can toggle between AM and ErM to switch the type of fungus we are assessing. Note – this drastically changes the functionality of the tool, as the different fungus types have different colonisation features, and as a result need different models to assess the colonisation. This means the labels you will see in annotations and predictions are different – we can see this below in the different legends shown in the annotations view.



The left-hand side are the labels for AM roots, and the right ErM roots – the definitions of these labels are detailed in [4] and [6].

Switching between the two modes affects many of the functionalities of the tool, including both the Browser and Finder functionalities. The differences are highlighted in the respective sections throughout the documentation – it is worth being aware then when you change the fungus type, the tool will change to only work with the type of fungus you have selected, and you should always double check you have the correct type selected to avoid unexpected results.

02.4 SUMMARY OF KEY TERMS

Throughout the document, several pieces of terminology are used that are specific to the context of the tool. Some of the main terms are summarised below.

- Predictions – in the context of the tool, predictions are probabilities assigned to the tiles of a root image that estimate what image features are contained in the tile. For each tile, every class the tile could belong to will be assigned a value, and the tile with the highest value is what the model believes the tile is most likely to be.
- Annotations – annotations are a way of denoting the contents of a tile with a specific label. A tile can only have one annotation in the CNN1 case – for CNN2, it can have multiple.
- Postgres – Postgres is an open-source database that is used to store annotations, predictions, and their associated metadata. The official Postgres website can be found here.
- Performance metrics – these are metrics that describe the overall performance of a model. They are output by the test functionality and should be used to gauge the efficacy of a model at predicting different classes. The main metrics we focus on are F1-score, Recall, Precision and Accuracy – these are detailed here.
- Colonisation metrics – colonisation metrics are a collection of summary statistics to assess the amount of colonisation within a given root. More specifically, the metrics give the percentage colonisation for different types of fungus, depending on whether you are viewing an AM or ErM root. This is detailed further here.
- CNN1 and CNN2 – there are two main modes of prediction for the tool, which are denoted by CNN1 and CNN2. CNN1 handles the prediction of colonisation for a root and is the **only supported model in this version of the tool**. CNN2 handles classifying colonisation structures in colonised tiles – predictions with this network are not supported in this version of the tool, however one can label AM data with CNN2 annotations. This is outlined further here.

03. TYPICAL WORKFLOW

Below, we outline what we would consider to be a typical workflow in the tool. For each step, we link to the relevant part of the documentation that will give further details on how to conduct that part of the workflow. The below workflow is applicable for both AM and ErM roots.

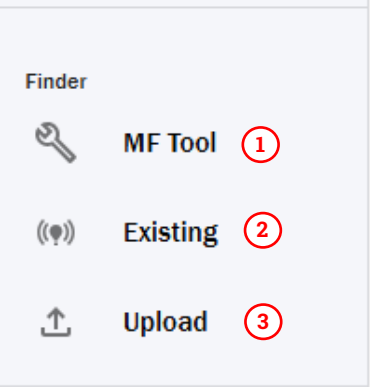
1. **Slide Conversion** – our workflow begins with an MRXS file (**note -** if you already have JPG images of roots you want to analyse, you can skip to step 2). We assume that the user has an extended focus slide that they are viewing in SlideViewer and can export this to a TIF with SlideMaster. The user would create annotations in SlideViewer around the parts of the slide they want to use as images in the tool, and then convert these to JPGs using the conversion functionality of the tool. Details on how to do this can be found here. Note that one can also use PNGs as input files, but this comes with certain caveats – further details can be found here.
2. **Calculating predictions** – after the above step is completed, you should have several JPG or PNG images of roots that would like to assess. To calculate predictions on these roots, you would pass the parent folder of these roots into the predict functionality of the AMF Tool. By passing in the root folder, you can calculate a batch of predictions for multiple root images at once. Once this has completed, you will have an output CSV that gives you colonisation metrics, along with an uncertainty measure for each set of predictions, and you are also ready to view the predictions in the Browser.
3. **Viewing predictions** – you can now view the predictions via the Browser functionality. This allows you to assess individual predictions, and manually override low confidence predictions to ensure a high degree of accuracy. Once you are happy with the set of predictions, you can convert them to annotations. You can also carry out automatic conversion via the AMF Tool functionality. Moreover, you can also view colonisation percentages associated with a particular prediction via CSV outputs, which also includes the confidence associated with each particular colonisation metric.
4. **Viewing annotations** – once you have converted predictions to annotations, you can view these annotations via the Browser. In this view, you can add questions to annotations to discuss with other researchers, browse and edit annotations as you see fit and save the changes to the database. You can also create new sets of annotations for labelling purposes.
5. **Outputting colonisation percentage** – the final step of the main workflow is to output the final colonisation percentages based on your final set of annotations. You can achieve this via first enabling the relevant set of annotations and then using the test functionality to output the colonisation percentage for said set of annotations.

04. FINDER

The Finder is the section of the tool that handles all modelling aspects and would be envisioned as the start of a typical workflow. The functionalities of the Finder section of the tool are laid out in detail below.

04.1 FINDER BREAKDOWN

Via the sidebar, a user has three different functionalities they can navigate to under the Finder functionalities.



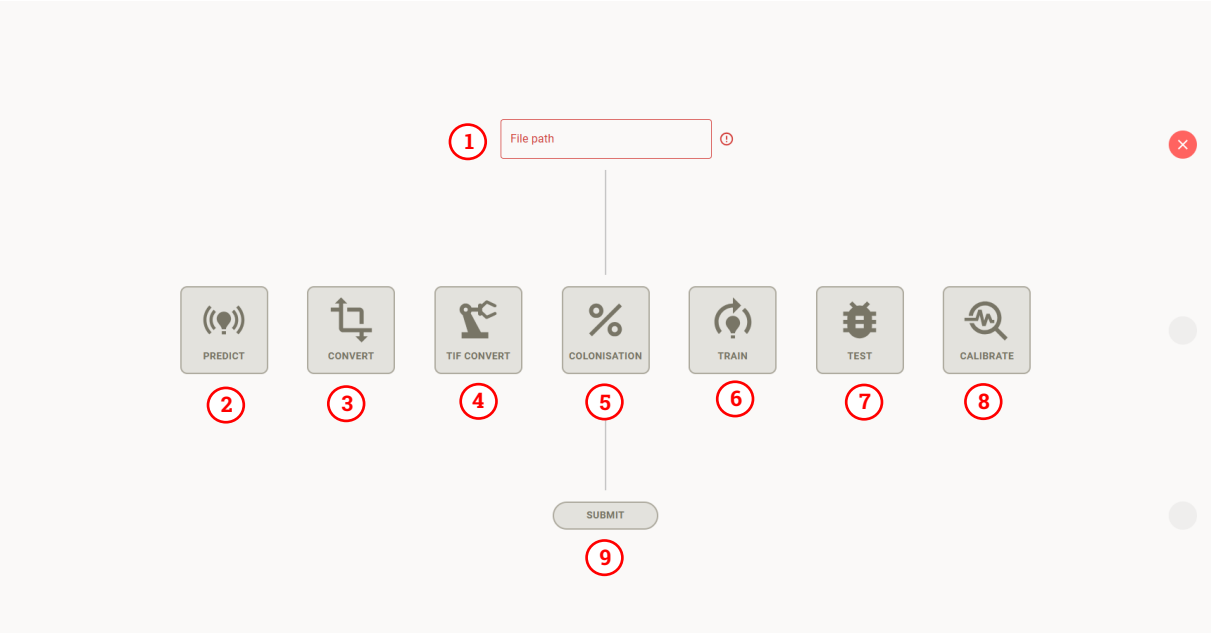
1. **MF Tool** - functionalities inside this section are detailed here – on a high-level, these include:
    a. **Predict** – carry out a prediction on an image,
    b. **Convert** – convert predictions to annotations (including some more complex functionalities which are detailed further here),
    c. **TIF Convert** – convert TIFs into JPGs, as detailed here,
    d. **Colonisation** – calculate the percentage colonisation metrics for a set of annotations,
    e. **Train** – train a new neural network,
    f. **Test** – test a trained neural network,
    g. **Calibrate** – calibrate a trained network to allow uncertainty measures to be calculated on predictions.
2. **Existing Predictions & Annotations** – under this section, you can view all existing predictions and annotations for any image that are stored in the local database for the tool. You can also download these predictions and annotations to a CSV file, which can be transferred between machines and used for further analysis.
3. **Upload Predictions & Annotations** – this functionality can be used to upload both current and legacy annotation files in the form of CSVs. You can use this to transfer data between machines, and even upload older formats of data to the tool.

04.2 MF TOOL

The MF tool can be used to trigger the backend functionalities of the tool. Below we discuss each of these functionalities in detail.

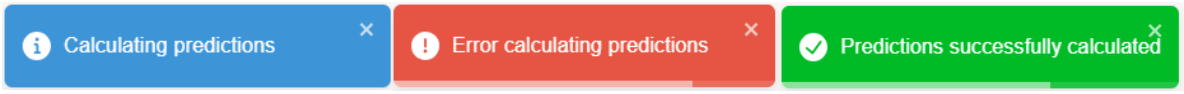
04.2.01 OVERVIEW OF FUNCTIONALITY

When you click on MF Tool in the picture above (denoted by option 2), you will be greeted with the following page.



1. **File Path Input** – this is an input for the directory path for the input to the command – this is usually a path to the folder where you want to run a command on a set of images (although it could also be a parent folder depending on the particular use case). This path **must** be an absolute file path to the relevant directory in your system – the format for this varies by OS. If your file path is invalid, then the tiles will remain disabled, the outline of the text input will be red, and you will have a red cross on the right-hand side of the screen. Once you input a valid file path, the cross will change to a tick and the tiles will become enabled – you will now be able to trigger the various functionalities. **The file path is a mandatory field for all functionalities of the tool.** Examples of valid file paths for each supported OS are below.
    a. Windows – C:\Valid\File\Path
    b. Linux & MacOS - /valid/file/path
2. **Predict** – this functionality carries out inference using a specified model on one or more JPG or PNG images. The output is a list of probabilities that a certain tile belongs to a certain class, along with a contextual label provided via max voting if enabled. More details on viewing these predictions can be found here.

3. **Convert** – convert can be used to automatically convert predictions to annotations using a given threshold. It can also be used to upscale 126x126 annotations to 252x252 annotations.
4. **TIF Convert** – this functionality is used to convert TIFF images to JPGs or PNGs.
5. **Colonisation** – this functionality is used to calculate the colonisation metrics for a particular set of annotations, which are then output as a CSV.
6. **Train** – train is used to train new versions of the model. It uses a specific split of data to create new versions of the model, using various hyperparameters that can be specified in the settings.
7. **Test** – this can be used to run a test on an existing model to get summary metrics of the performance of the model on a test dataset, and to output misclassified tiles which can be used to understand where the model is underperforming.
8. **Calibrate** – this is used to calibrate newly trained networks, to determine the temperature factor used for scaling probabilities.
9. **Submit** – once you have selected a tile, you need to click Submit to trigger the functionality in the backend. This will disable all the tiles, the icon to the right will become a spinner and a notification will appear in the bottom right of the tool to let the user know the process is ongoing. The notification will then turn green if the process was successful, or red if it was not – this looks like the below.



Also note that several of the above functionalities have different settings that can be changed that will alter the functionality of the tool. These can be accessed via the "Settings" section of the sidebar and are explained in detail in the following sections.

04.2.02 SWITCHING BETWEEN AM AND ERM

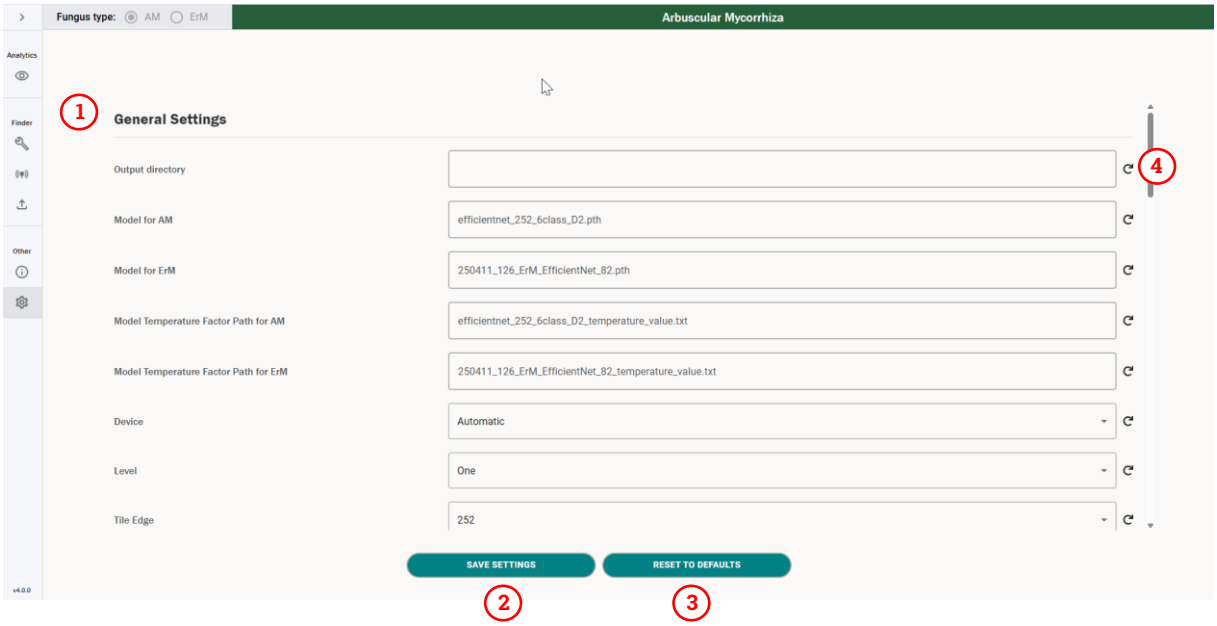
As outlined earlier, one can switch between AM and ErM fungus types. This greatly affects the functionality of the MF Tool section – a summary of the functionalities it affects are below, and these are expanded on in the respective sections.

1. **Predict** – when running predict, if you have the AM toggle selected, it will use the AM model (as specified in the general settings) and will calculate a prediction with the AM labels. If you have ErM selected, it will calculate a prediction using the ErM model, which will have the ErM labels. Naturally if you try to calculate a prediction using the ErM model on an image of an AM colonised root, you will get erroneous predictions.
2. **Convert** – this will convert a set of predictions to annotations – as such, the predictions you specify must have the corresponding classes to the colonisation type you have selected.
3. **TIF Convert** – this is not affected by the colonisation type.

4. **Colonisation** – depending on the colonisation type you have selected, this will be expecting annotations that match that colonisation type. The output will also differ, as the definitions of colonisation percentage vary by colonisation type – you can find these definitions here.
5. **Train** – this will train a model that matches the colonisation type, and thus has a corresponding number of classes to match the labels. The training data you specify must correspond to the colonisation type you have selected.
6. **Test** – this expects test data to match the model specified in the general settings for AM and ErM. If it does not match, you will get an error. This will also change the model that is being tested depending on which colonisation type you choose – it will either use "Model for AM" or "Model for ErM" that you have specified in the settings for AM and ErM colonisation type respectively.
7. **Calibrate** – this expects a calibration dataset with labels that match the selected colonisation type. Similarly to test, this will calibrate the model from "Model for AM" or "Model for ErM" that you have specified in the settings for AM and ErM colonisation type respectively.

04.2.03 GLOBAL SETTINGS

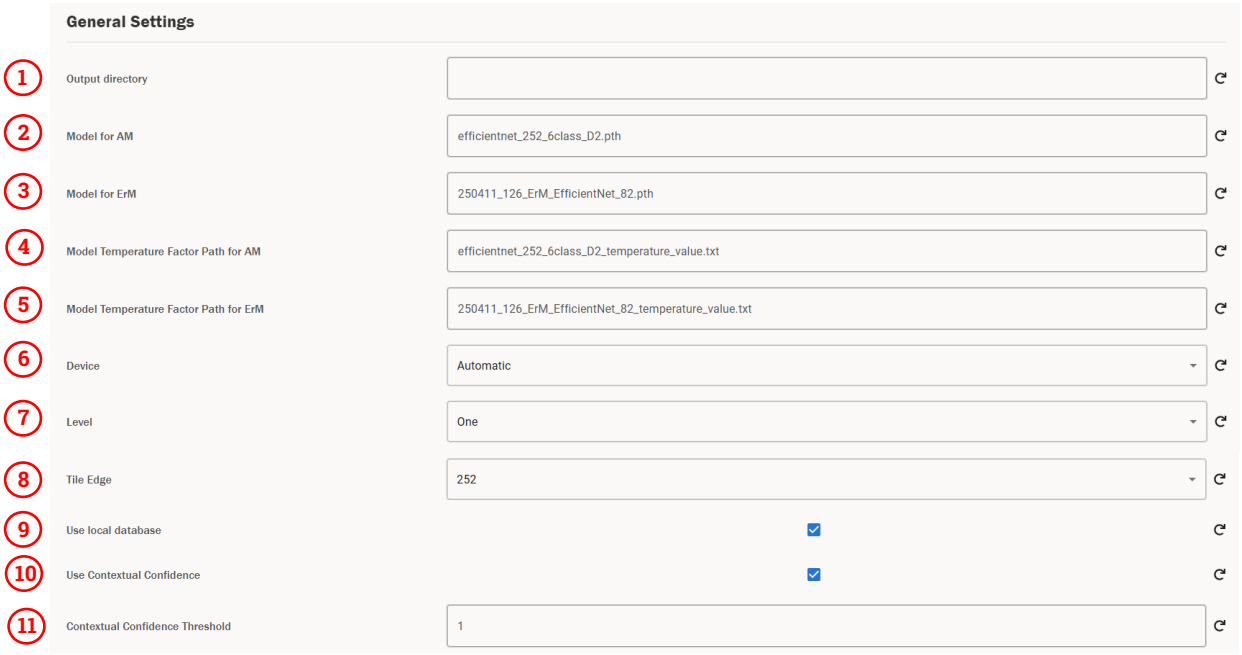
One can edit the settings that are passed to the backend of the tool, which changes how the tool runs its various processes. These settings can be opened by navigating to the settings page using the sidebar. When you navigate here, you will be greeted with the following screen.



1. This is the settings container, that contains all the names and values for the settings that are currently stored in the database.

2. This is the save settings button – when you change the settings, you need to click this to save the new values down to the database. Note that you do not need to click this for the settings to be used throughout the tool – as soon as you have changed a setting, it will be used by the functionality of the tool.
3. By clicking this button, you will reset all values to the defaults that are stored in the database. This also changes the value in the database, i.e. you do not need to click save settings after this.
4. Each value has a redo icon next to it – if you click this, it will reset that individual setting to its default value. This also changes the value in the database, i.e. you do not need to click save settings after this.

At the top of the settings section there is a **General Settings** subsection – these are settings that apply to multiple functionalities of the AMF Tool. The other settings subsections will be detailed in the corresponding sections below.



A description of each of the general settings is as follows.

1. **Output directory** – several of the functionalities of the tool involve outputting various pieces of data which are saved onto your local file system. The output directory field specifies where to save these outputs. By default, it is blank, and each functionality will save to different areas in the file system – these are addressed on a case-by-case basis in the following sections.
2. **Model for AM** – this specifies the model to be used when calculating predictions, retraining a model, calibrating a model, or calculating test metrics on a model for AM roots – this model should be trained specifically for AM, and is used when a user has AM selected in the Fungus Type toggle.

3. **Model for ErM –** this specifies the model to be used when calculating predictions, retraining a model, calibrating a model, or calculating test metrics on a model for ErM roots – this model should be specifically trained for ErM, and is used when a user has ErM selected in the Fungus Type toggle. For both this and the previous field, if you specify just the name of a model, the tool will look in a trained_networks folder for pre-saved models which is built into the tool. The contents of this folder can be found here. If you specify an absolute file path (e.g. C:\Path\To\Model.pth), the tool will look for the model at the specified path and output an error if it cannot find this model. Note that you can only use pth model files for the current implementation of the tool.
4. **Model Temperature Factor Path for AM** – this is the path to the temperature factor for the AM model, which is calculated using calibrate and is necessary for scaling the probabilities to calculate uncertainty metrics. This path is used when a user has AM selected in the Fungus Type toggle.
5. **Model Temperature Factor Path for ErM** – this is the path to the temperature factor for the ErM model, which is calculated using calibrate and is necessary for scaling the probabilities to calculate uncertainty metrics. This path is used when a user has ErM selected in the Fungus Type toggle.
6. **Device** – this specifies which device to run the various AMF Tool functionalities on, which can either be your CPU, or if applicable, your GPU. The various options are explained below**.**
    a. Automatic – this is the default setting and will automatically select the best device based on your system's capabilities. If you have a GPU and its CUDA version is greater than or equal to 12.1**,** it will select GPU. Otherwise, it will select CPU. This is the recommended setting – more details are outlined here.
    b. CPU – this will run the functionality using your CPU.
    c. Cuda (GPU) – this will run the functionality using your GPU.
7. **Level** – this specifies whether to use a network for assessing colonisation or for assessing intraradical structures. **In the current version of the tool, only Level One is supported, and trying to use Level Two will likely result in an error.** This is included regardless to aid future development.
8. **Tile Edge** – this can be used to switch between a tile size of 252 (default for AM) and 126 (default for ErM) for all functionalities.
9. **Use local database** – this checkbox determines whether to use the database for storing predictions and annotations, along with metadata about an image, or whether to save these down to CSVs. By default, it is true, as using a database is the intended usage of the tool, however if you need to conduct training runs on detached infrastructure, you can change this option to false to output e.g. annotations to CSV files.

10. **Use Contextual Confidence –** this toggles whether to generate contextual labels in predict, test, and convert, which are labels that consider the surrounding context of a tile and use a max voting system to suggest a contextual label for the tile. This is outlined in more detail here.
11. **Contextual Confidence Threshold –** if you have specified to use context as above, this sets the threshold for which tiles we should calculate contextual labels for – for any tile with confidence below this specified threshold, we will create contextual labels.

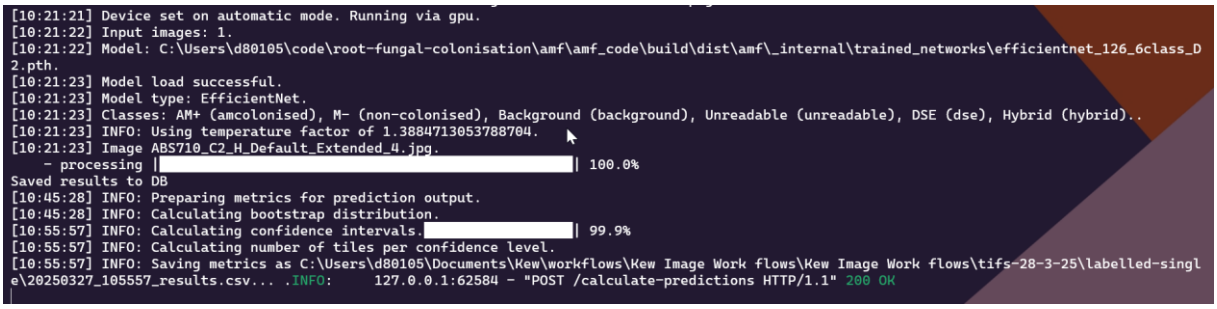
04.2.04 PREDICT

In order to calculate a prediction for a root, you need a JPG image of that root. More information on how to obtain a JPG image from an MRXS file can be found here. You can also use PNGs, but this comes with certain risks – these are outlined here. The general process for how the tool calculates a prediction is as follows.

1. You pass a folder path to a directory that contains one or more JPGs or PNGs into the input specified above. Note that this can be a parent folder – the tool will recursively search all folders below the folder provided to find all JPGs or PNGs to calculate predictions for. Note that only images with the extension .jpg, jpeg or .png will be considered for predictions.
2. The tool then tiles each of these images into 252x252px tiles for AM and 126x126px tiles for ErM. It then calculates predictions for each tile using the model you have specified in the "Model for AM" or "Model for ErM" setting for AM and ErM fungal types respectively. This allocates probabilities that the tile belongs to one of the following six categories for AM, and ten categories for ErM. Further details on the definitions of these classes can be found in the documents [4] and [6].
3. These predictions are then stored in the local Postgres Database. You can also choose to save these predictions down as CSV files instead of storing them in the Database – we provide more information on this functionality here.

Note: *the day-to-day use of the tool is intended to use the Postgres database and not CSVs. Be aware that if you choose to use CSVs, you will have to manage this data yourself. Also, when using the Browser, you will have to import these CSVs into the database to be able to view them. You can see more details on this here.*

When you have specified a file path and clicked the "Predict" button, you will get a notification that your predictions are in progress. In the exe cmd window, you will be able to see a progress bar for the progress of the predictions – this looks like the below.



Note – the above is a prediction calculated without context. See here for the prediction output when calculated with context.

In the above output we can see that we have successfully calculated predictions for an image and have calculated confidence intervals for that image. You can use the above console to debug if there are any issues with the prediction – more on this here.

Once your predictions are successfully calculated, you will get another notification informing you they were successful. You can then view the predictions in the Browser – more details on this can be found here.

Note: to calculate the uncertainty metrics for your predictions correctly, your model needs to be calibrated and have a corresponding temperature factor. You need to be pointing to the correct model and temperature factor in general settings. All the models in trained_networks have a corresponding temperature factor – if for example the model is named model.pth, the temperature factor file will be called model_temperature_value.txt. As such, if you are changing models to calculate predictions, **you also need to change the temperature factor path to the corresponding temperature factor file.** If you retrain the model, you also need to calibrate the model and change the temperature factor path. There are two options for the input here.

1. If you just specify the name of the file, then the tool will assume it is stored under the trained_networks folder and will only look here for the file. More information of on this can be found here.
2. If you put an absolute path to the file, e.g. C:\Example\Path\To\File.txt, then the tool will use the file located at the absolute path.

If you do not specify a path to the file, then the temperature factor will default to 1.

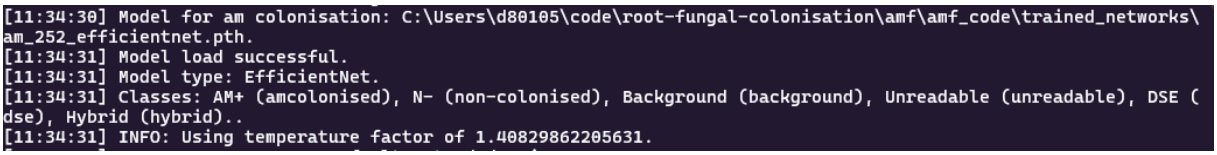
04.2.04.01 AM VS ERM PREDICTIONS

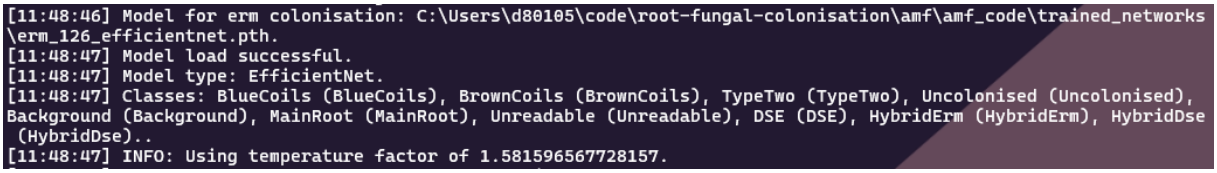
If you have AM selected, you will calculate predictions using the model specified in the "Model for AM" general setting, along with the corresponding temperature factor path specified in "Model Temperature Factor Path for AM". Similarly, if you have ErM selected, the model used will be the "Model for ErM" general setting, along with the

corresponding temperature factor path specified in “Model Temperature Factor Path for ErM”.

If you select AM, each prediction will consist of predictions for the six AM classes. Similarly, if you choose ErM, the output will be predictions for the ten ErM classes. You need to be very careful to select the correct model for the image you are wanting to calculate predictions for, as the model will give you predictions regardless of the image you feed in, but naturally if you feed a root with ErM features into an AM model, the model will give you nonsense predictions back out.

We can see in the console output the difference between AM and ErM – the model selected is different, as are the classes and temperature factor.





One major thing to note for AM vs ErM models is that AM is trained using 252x252px tiles and ErM is trained with 126x126 tiles, and as such the tile size you get when calculating predictions is different. This is due to differences in the biological makeup of AM and ErM roots. The tile size that is used globally throughout the tool automatically switches between 252 and 126 when you switch the fungal type, but it is worth remembering this when calculating predictions and working with the output.

04.2.04.02 ADDING CONTEXT TO PREDICTIONS

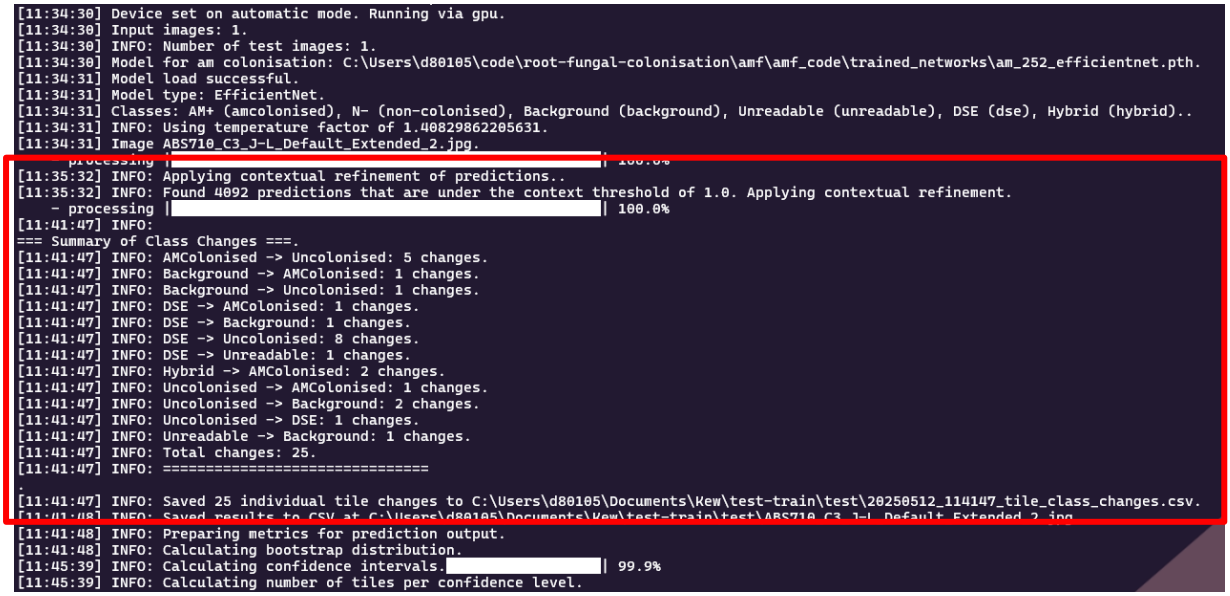
To improve the contextual reasoning of the model, for a particular tile we can consider predictions the model makes on surrounding tiles to inform us on what the prediction for said tile should be. More details on the exact process are included here.

To turn on the contextual predictions, you need to tick the box in general settings labelled “Use Contextual Confidence” – this is true by default. Then, when you calculate a prediction, for every tile that has a maximum confidence below the threshold specified under the “Contextual Confidence Threshold” (which can also be found in general settings and defaults to 1), the tool will go back through and calculate four more predictions for the tiles surrounding the central tile. For each set of tiles, it will see which class occurs the most frequently (based on the maximum confidence class of

each tile) and then select the most frequent tile as the contextual label using a max vote system.

When viewing these predictions in the Browser, you can then see the calculated contextual label, and decide whether or not to use these contextual labels when converting the tiles to annotations. You can also decide to use contextual labels when using the convert tool.

The output in the command line for calculating a prediction with context looks like the below.



We can see the difference highlighted in the red box – the tool tells us how many tiles it is calculating contextual labels for based on the contextual threshold in the settings, and then also gives us a summary of the class changes, detailing how many tiles have changed for each class. Note that we also get a CSV file for the predictions which tells us exactly which tiles have had a contextual label added – this is output as {timestamp}_tile_class_changes.csv.

Note – we only save the contextual label when it differs from the maximum confidence class, which is the usual label we would use for automatically converting predictions to annotations. As such, even though in the above example we calculated contextual labels for every tile, we can see we only made 25 changes, and as such there would only be 25 contextual labels saved that could then be used for conversion.

04.2.04.03 COLONISATION METRICS FOR A PREDICTION

On top of the predictions being saved to the database, there will also be a CSV output to the same folder as the image (or the directory you have specified in “Output directory” in the settings) which will contain the total count of predicted tiles for each class, along

with the colonisation percentages for the prediction – these metrics are based on the maximum probability associated with the tile. The naming convention will be *{date}_{time}_results.csv*.

For each metric reported, there will also be three further metrics, which are based on a sampling procedure rather than just taking the maximum probability of the models output as the predicted class. One of the additional metrics is the mean of the sampling procedure, and the other two are the upper and lower bound for a confidence interval on the metric. The confidence interval can be viewed as a starting point, as the usual workflow will involve a researcher manually labelling the uncertain tiles from a prediction. Naturally, the manual labelling will decrease the uncertainty of the overall prediction values, and hence the confidence interval would shrink assuming the same formulas are used. An in-depth explanation of the sampling approach and the resulting predictions is given here.

Note – the above metrics will naturally differ for AM and ErM, as the classes differ.

Additionally, there will also be an overview of the number of tiles per confidence threshold for a prediction. More specifically, for each tile we look at the maximum probability that is output by the model. Then, given this max probability, this tile is counted towards one of the confidence thresholds buckets, where the borders range from 0.1 up to 1.0 in steps of 0.1. The buckets are cumulative, meaning that all the tiles that are in the bucket with threshold 0.2 are also in the bucket with threshold 0.3 and above. As an example, we consider a tile, where the model predicts a class with max probability of 0.42. This probability is lower or equal to 0.5. Hence, the tile counts towards the buckets “Tiles with confidence <= 0.5”, “Tiles with confidence <= 0.6”, ..., “Tiles with confidence <= 1.0”. See an example output below.

AO	AP	AQ	AR	AS	AT	AU	AV
Tiles with confidence <= 0.3	Tiles with confidence <= 0.4	Tiles with confidence <= 0.5	Tiles with confidence <= 0.6	Tiles with confidence <= 0.7	Tiles with confidence <= 0.8	Tiles with confidence <= 0.9	Tiles with confidence <= 1.0
12	67	255	675	1044	1543	2217	6893

04.2.05 CONVERT

Convert can be used to automatically convert predictions to annotations and also to aggregate tiles to upscale the tile size by 2 (which is only enabled for AM roots). The various functionalities can be triggered by changing the settings. Navigate to the global settings and move down to the **Convert** subsection – you should see a screen like the below.

1. **Threshold** – this setting can be used to change the threshold for automatically converting predictions to annotations. You can change this to a float between 0 to 1 inclusive. Changing this setting will also change the conversion threshold in the predictions screen in the Browser.
2. **Aggregate tiles** – this setting can be used to aggregate 126x126 tile labels into 252x252 tile labels according to pre-defined rules for combining four tiles into one. When ticked, convert will look for images with 126x126 labels and aggregate them into 252x252 labels.

04.2.05.01 CONVERTING PREDICTIONS TO ANNOTATIONS

Note: you can also carry out this conversion in the Browser – this is explained here. Carrying out the conversion in the Browser would be the recommended method, as it gives a greater degree of control for labelling uncertain tiles.

To convert a prediction into annotations, you need to specify the file path to an image (or images) that have predictions attached to them. When you click convert, this will then convert the predictions to annotations by taking the class with the highest confidence and assigning the tile this category as an annotation, if the probability is above the threshold described above (unless you are converting with context). If there are no classes with a probability above a certain threshold, then no annotation is assigned to that tile. This threshold has a default value of 0.5 – this can be changed in the settings as explained here.

When predictions have been successfully converted to annotations, they will be linked to the same database object as the predictions and will be saved under the same timestamp. You can view/edit the annotations and predictions together in the Browser – this is outlined here.

Note: this will carry out conversion for the enabled set of annotations/predictions.

04.2.05.02 CONVERTING WITH CONTEXT

If we have calculated a prediction with context, then we can use these contextual labels to convert our predictions to annotations. If the "Use Contextual Confidence" setting is ticked in the general settings, then if a prediction has a contextual label attached to it, it will use this label for the conversion, so long as the maximum

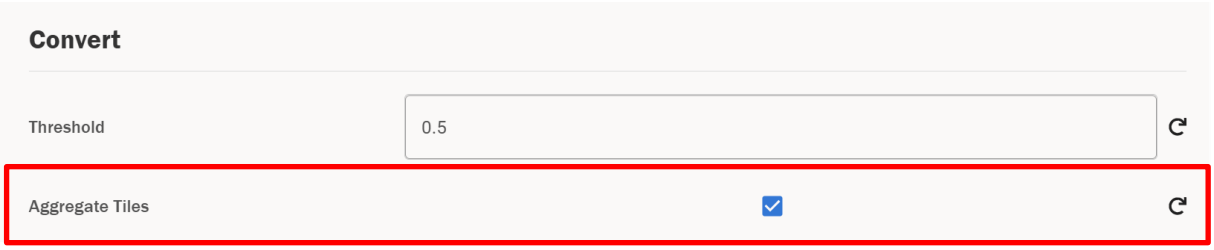
confidence for that class is above the conversion threshold. If the maximum confidence is below the conversion threshold, it will not convert this to an annotation at all. If it does not have a contextual label, then it will use the maximum confidence class as normal.

As mentioned above, the maximum confidence of the tile still needs to be above the conversion threshold for us to assign an annotation, regardless of whether we are using the maximum class or the contextual label. So, if a tile has a contextual label but the maximum class has a confidence lower than the threshold, then this won't be assigned a label, regardless of whether you are converting with confidence or not.

04.2.05.03 UPSCALING ANNOTATIONS

There is also a functionality in the tool to upscale from 126 to 252 tiles (or in fact, to double the tile size of any tile set, although 126 and 252 are the only tile sizes supported in the rest of the tool). This happens by combining 4 adjacent tiles into one tile, which can then be mapped onto a tiling of the image where the tile edge is double the size (and thus the tile is four times the size).

This is likely not a functionality that will be used in the day-to-day functionality of the tool, however it can be useful for training models with different context windows. To upscale the annotations, you need to tick the following box in the settings page.



This will enable tile aggregation. Then, go to the MF Tool page and click "Convert". You need to pass the image path to the images you want to upscale the annotations for, and naturally these images need to have valid annotations attached to them. Once this succeeds, you will see that the tile edge associated to the annotations has doubled. Note that this will **overwrite the existing set of annotations** you have selected for this image, so make sure to first save a copy if you still want access to the original annotations.

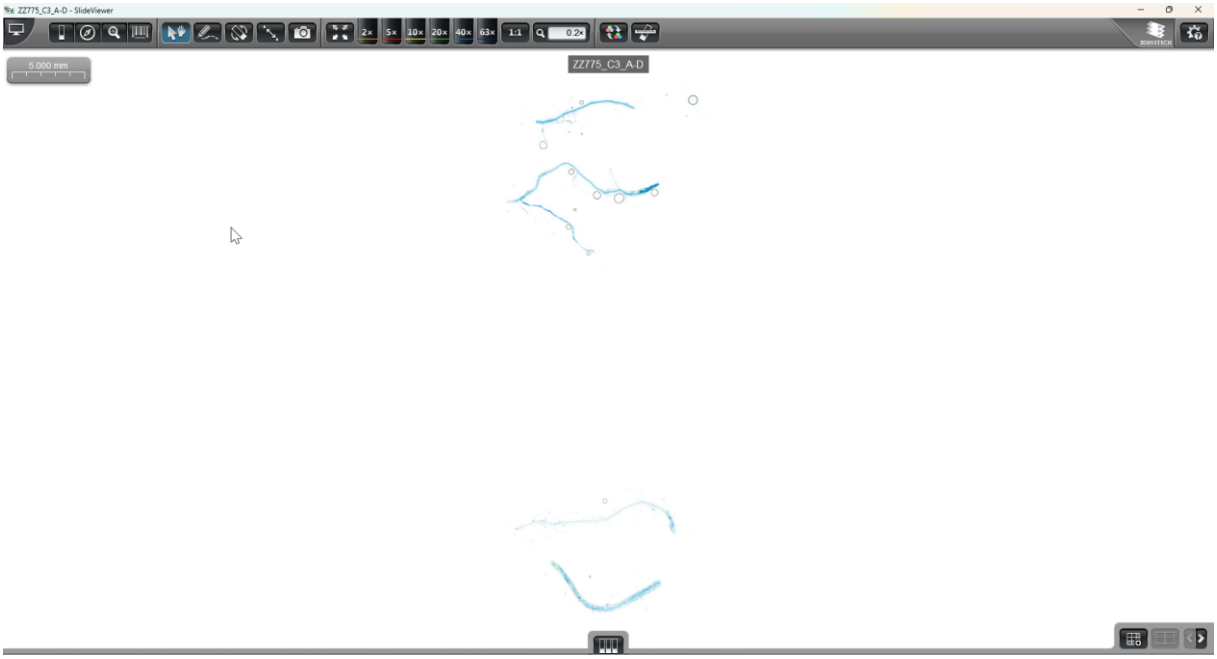
Note that this will aggregate the **enabled** set of annotations for a particular image.

Warning – this functionality is only supported for AM annotations and not ErM annotations. You will receive an error if you try to aggregate tiles for ErM annotations – we can see the console output for this error below.

```
[17:20:20] ERROR: Only support AM Colonisation for converting tile size, not erm. Cancel operation.
```

04.2.06 TIF CONVERSION

The workflow for converting an MRXS file to JPGs begins with an extended focus image in SlideViewer. We expect a user to have a slide scanned in MRXS format, and for this to be an extended focus image. An example of a typical slide in SlideViewer could look like the below.



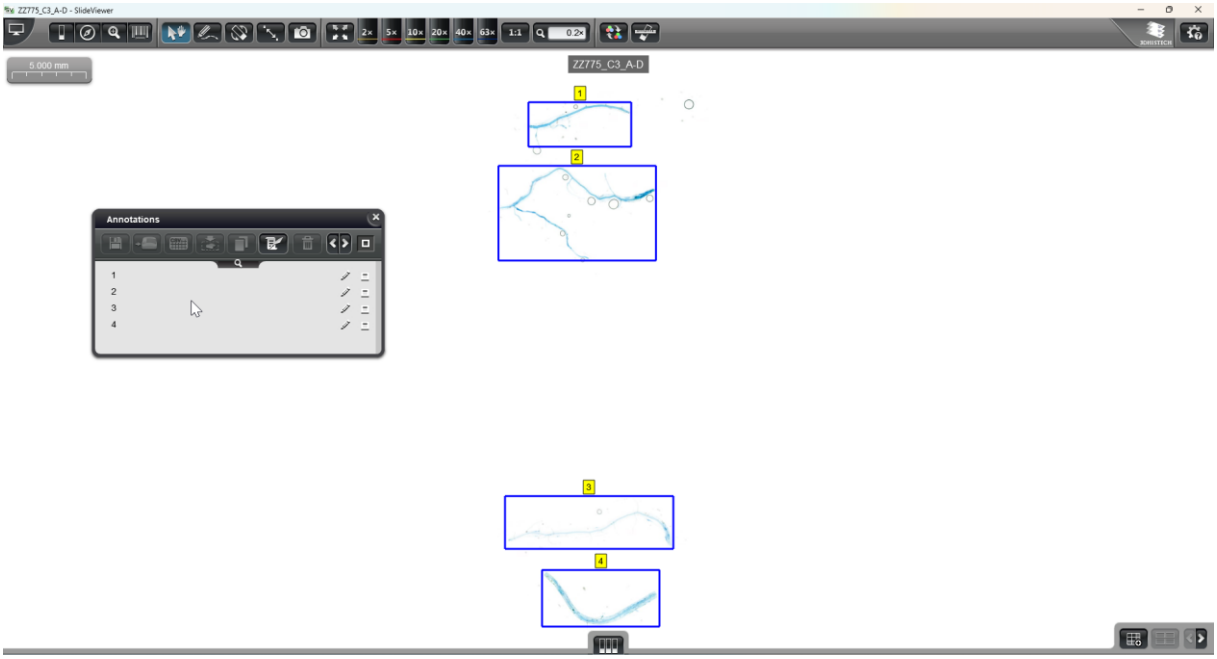
In this slide we can see we have four different root segments. For our analysis, we want to analyse each of these root segments, but before we can do that, they need to be converted into JPGs so that Mycorrhiza Finder can process them. There are several ways we can go about this conversion.

1. **Manual conversion** – historically, the workflow has involved converting these images to TIF files using SlideMaster and then manually cropping the images to JPGs. This naturally involves a lot of manual work and is not the optimal workflow, but it is worth noting that this is an option if one does not have access to slides in the correct format.
2. **Automatic Conversion** – one can also convert the MRXS slides into JPGs automatically. This workflow is detailed below.

The first step in the automatic conversion workflow is to annotate the roots with boxes. These boxes will denote the areas that we want to crop the slide to. We would recommend creating these boxes around each root segment.

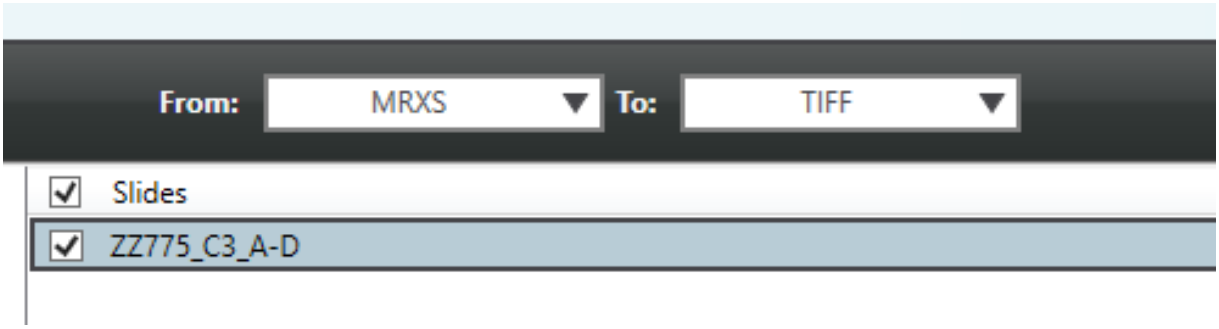
Note: *if the annotation is larger than 65500 pixels the automatic conversion to JPGs will not work. In this case, you will need to either split the root up into multiple segments, crop these manually or output the files to PNG.*

An example of an annotated slide is shown below.

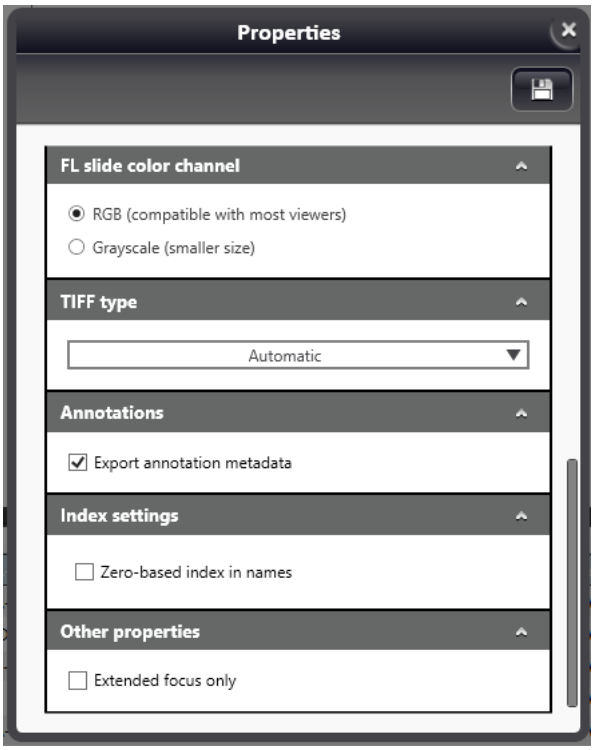


We can see that in the above we have created four annotations called 1, 2, 3 and 4, which are bounding boxes for the different segments of the root. You may want to use different naming conventions here – e.g. A, B, C and D. We will see why this is relevant later in the process.

To convert these annotations into JPGs, we now need to use SlideMaster to convert the above MRXS files into a TIF file. You need to open SlideMaster and find the relevant folder with your MRXS file in. You then need to select the conversion at the top from MRXS to TIFF, as below.

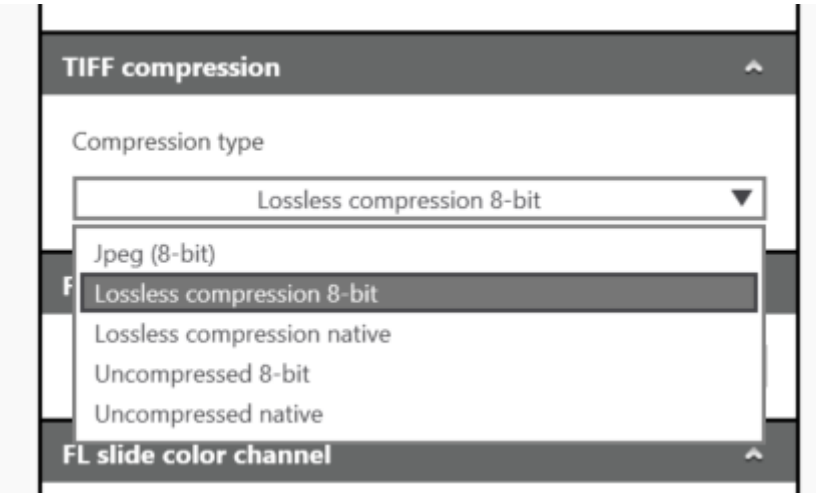


On the right-hand side, select any local folder as your output folder. Leave “Action for slide” as “No Action” and then click Properties. Under the “Annotations” section you need to tick the box “Export annotation metadata” – it will look like the below.



Note: *there is also an option to export the annotations separately using the dropdown at the top. If you use this method, the* ***cropping will not work****, as the offsets in the XML do not line up correctly. Please be aware of this when creating your annotations.*

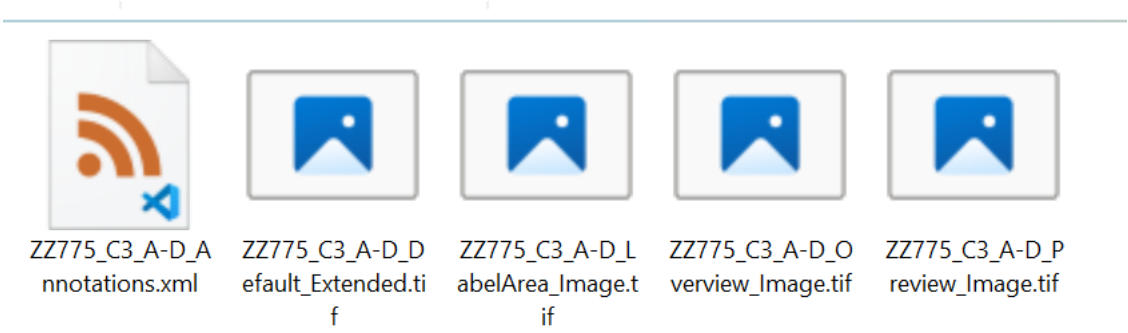
To improve the image quality, one can also change the compression to “Lossless compression 8-bit” as below.



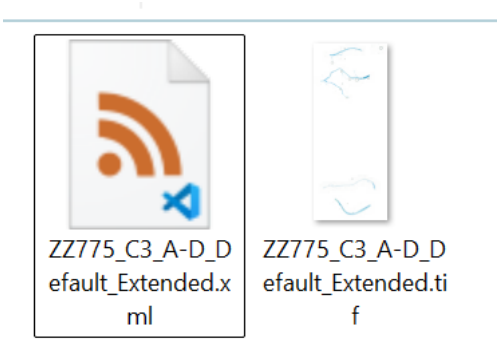
After changing the Properties as above, you can add the slide to the queue using the “Add slide(s) to queue” button. Note that you can do this for multiple slides at once if you want to batch process slides.

After a slide has been processed (which you will be able to see in the bottom pane of SlideMaster), you should get an output that looks like the below.

You need to drag the annotations file into the folder, and then open the folder. You should see something similar to the below.



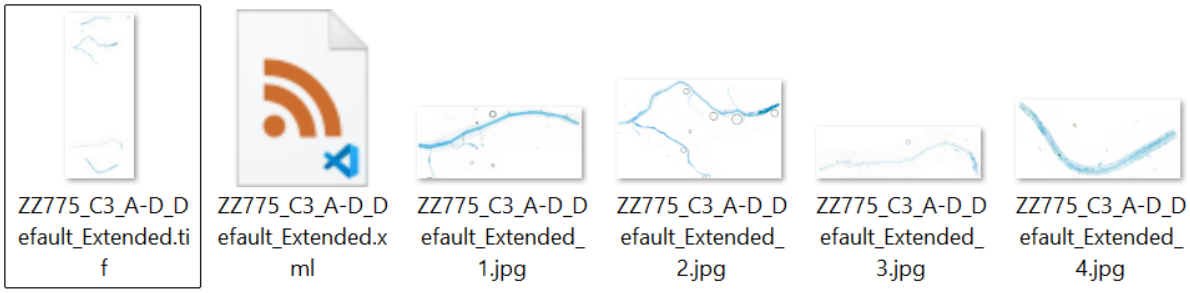
Here, we are **only** interested in the Default Extended image. Delete all the tif files except the one that ends in Default_Extended and rename the annotations file to have the same name as the renaming tif file. The final state should look like the below.



You are now ready to convert the annotations you specified in the tif image into JPGs. Take the file path of the folder that contains the TIF file and annotations file above, and copy/paste this into the input box for the image path. If the file path is valid, the tiles will become enabled.

Now, click the “TIF Convert” tile and click Submit. If the conversion is successful, then the notification in the bottom right will go green and tell you it was successful. If unsuccessful, the notification will be red and will tell you about the error. In this case you can debug the output via the command line – see here for more details on this.

If the conversion was successful, your directory from earlier should now look like the below.



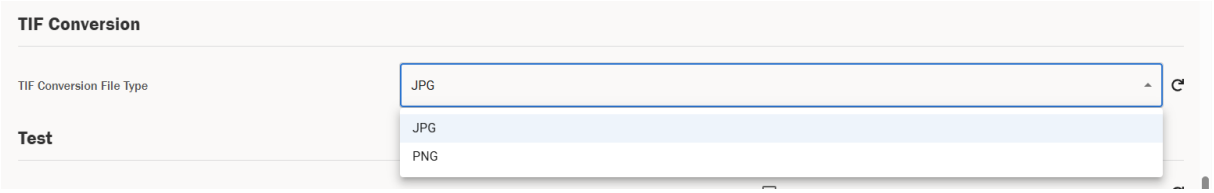
Each JPG should exactly match the annotation you made earlier in SlideViewer, and will be named using the convention "{*Image_name}_{Annotation_name}*", where Image_name is the name of the tif file, and Annotation_name is the name of the annotation you created in SlideViewer. You can now use these JPGs as input to the AMFinder tool.

***Note**: you can batch process the conversion of slides by selecting the parent folder under which multiple of the above TIFs, along with their corresponding annotation files, exist.*

There is also the option to change the conversion from JPGs to PNGs. This can be useful as the maximum supported size for a JPG is 65500x65500px, so if you have a root larger than this you could use PNGs.

WARNING: as PNGs and JPGs have different compression settings, using a PNG instead of a JPG can lead to very slightly different predictions to be output by the tool. The current version of the model was trained on JPGs, so it is best to use these when possible.

In order to change this, you need to navigate to the settings page using the right sidebar and change the TIF Conversion File Type setting, under the TIF Conversion section.



Note: as TIF images are generally quite large, this functionality can take up a lot of RAM. We expand on this in the error handling section – there is currently no fix for this, if you hit RAM limits you either need to use a machine with more RAM or use smaller TIF images.

04.2.07 COLONISATION PERCENTAGE

One of the main outputs of the tool is understanding the colonisation percentage across a root. Colonisation percentage is given as an output when testing a particular model, but as this is quite time consuming there is also a functionality to output just the colonisation percentages for a set of annotations.

To do this, you need to trigger the functionality using the Colonisation tool. As always, make sure to input the file path to the directory that contains the images that you would like to output the colonisation percentages for – these images must have annotations stored in the database for the fungus type you have selected (or as a CSV if you are instead using CSVs). Now, simply click the Colonisation tile and click Submit – this will now start to calculate the colonisation percentage for the enabled annotations. Once this is done, the final line of the console output will tell you where the results have been stored.

The results will be a zip file with a single file called file_metrics.csv inside. This file will contain the **actual percentage colonised metrics per root for the associated annotations**.

The default output directory will be:

```
{DEFAULT_HOME_DIRECTORY}/Documents/amfinder/{DATE_TIME}_col_test
```

where DEFAULT_HOME_DIRECTORY is your default home directory, e.g. C:\Users\your-user, and DATE_TIME will be a date time string in the format YYYYMMDD_HHMMSS.

Note: the colonisation metrics differ for AM and ErM images, and as such you need to make sure you have the correct colonisation mode selected for the image(s) you are calculating the colonisation percentage for.

The formulas for the summary statistics that are provided as part of this output are outlined here.

04.2.08 TRAIN

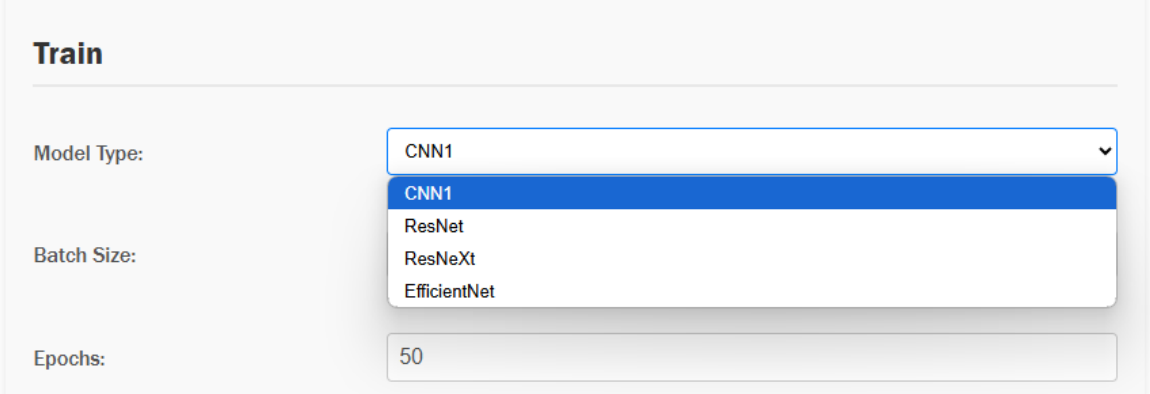
This section of the tool can be used to train new models and also retrain existing models with new data. This is a more complex area of the tool, potentially requiring development work and rebuilds in order to fully integrate any new models into the tool. The general process for training or retraining a new model is outlined below – this is also expanded in the Developer documentation that is included with the codebase of the tool.

The below is intended as a high-level overview on how to run model training, however, this is a much deeper topic. We have included a section on training best practices here to give a more in-depth view on how to successfully carry out training – please make sure to read both the below and the supporting Annex before training new versions of the models.

04.2.08.01 TRAINING A NEW MODEL

You can either train a new model from scratch, or you can retrain an existing model with extra data. The latter is the more likely scenario for most use cases – however, if you want to train a model entirely from scratch, you need to ensure that the "Model for AM" or "Model for ErM" setting in the General settings section is empty, depending on whether you are training a model for AM or ErM respectively. This will initialize a new model for training instead of using the existing model.

Note that you can train multiple different model architectures, including the original CNN1 architecture as found in the original Evangelisti paper [1], ResNet, ResNeXt and EfficientNet. In order to change the type of model you are training, you need to change the dropdown selector for Model Type.



If you want to retrain a model then you should populate the "Model for AM" or "Model for ErM" setting with the pth file for the model you want to retrain for AM or ErM respectively (which is determined using the toggle at the top of the screen as previously discussed) – in this case the model type selector will be ignored, and the model specified in the model field will be used as a base model.

In order to train a model, you need labelled training data that matches the fungal type you want to train – this means you need one or more images with annotations for AM or ErM. These annotations must be labelled by a human, as they need to be the exact ground truth, and are assumed to have entirely correct labels. In general, your training dataset should be a representative sample of the data you expect to feed into the model and have a balanced number of examples of each class where possible. They also need to match labels to the fungal type you are selecting – 6 classes for AM and 10 for ErM.

Once you have selected your training data, you need to put all this data into a folder called “train”. This will then automatically and randomly be split by the tool into a training and validation dataset. You will also want to keep approximately 20% of your labelled images reserved for testing – more on this here.

Once your images are all grouped together under a train folder, you will want to pass the image path of the parent folder of the train directory into the input box. For example, if the path to your train directory was “C:\Path\To\Directory\train”, the correct input path to the tool would be “C:\Path\To\Directory”.

Now, you are nearly ready to train – the last thing to do is to tweak the training settings accordingly to your needs. Note that each parameter will give networks with varying performance, and you should carry out hyperparameter sweeps where possible to select the best performing networks automatically. This is not directly supported in the UI of the tool, however it can be carried out using provided Jupyter notebooks in the code repository for the tool – these are used for AzureML sweeps.

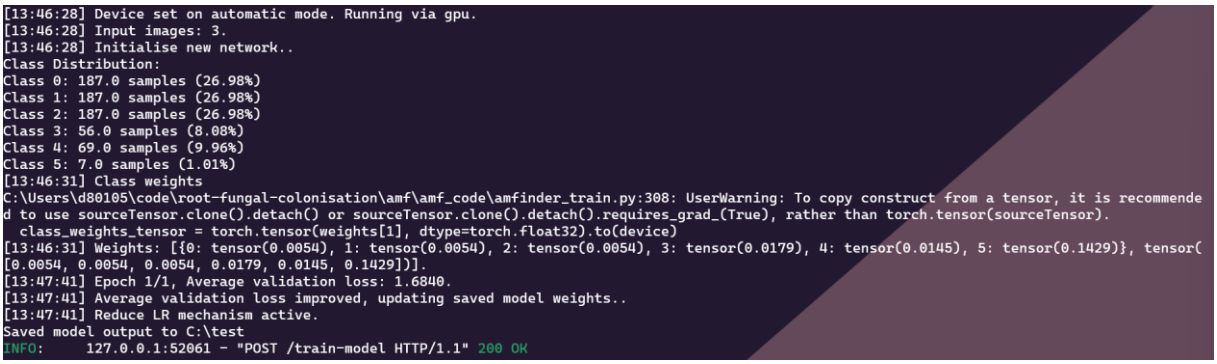
The settings that are available for you to train are as follows.

Name	Description	Default value	Required
Semi-supervised	Indicates whether semi-supervised learning is enabled.	False	False
ML Flow flag	Activates MLflow logging if set to true.	False	False
Batch Size	Size of each batch for training.	32	True
Adam Beta 1	The beta1 parameter for the Adam optimizer.	0.906450740008503	False
Adam Beta 2	The beta2 parameter for the Adam optimizer.	0.986390310777448	False
Balance factor	Factor to balance different aspects during training.	1.24895925434138	False
Epochs	Number of training epochs.	50	True
Vfrac	Validation fraction of the dataset as a proportion.	0.2	False
Data augmentation	If true, then augment data during training	False	False
Summary	Indicates whether to save the CNN model architecture summary.	False	False
Learning Rate	Learning rate for the Adam optimiser.	0. 000004218361045	True
Patience – Early Stopping	Patience value for early stopping.	10	False

Patience – Learning Rate Reduction	Patience value for learning rate reduction.	5	False
Model Type	Specify the model type you want to initialise - can select from cnn1, resnet, resnext and efficientnet.	efficientnet	True
Use pre-trained model	Loads ImageNet weights if ResNet, ResNeXt or EfficentNet is selected for Model type.	True	True

Note that the defaults are the exact hyperparameters that were used to train the model **am_252_efficientnet.pth,** which is the default AM model. These will provide a good baseline for retraining AM EfficientNet models, but would need to be tweaked if you wanted to train other model architectures, or if you want to train ErM models – we provide more detail on this here.

Once you have changed the settings accordingly, you can carry out a training run. Input the file path as described above and click the Train tile. In the console, you will see the below output.



Once training has completed, you will be able to access the final model in the folder specified above in the console, in the final line that begins with “Saved model output to…”. It will be saved as “model.pth” which you can rename accordingly. There is also a h5 version of the model saved – this cannot be used with the current version of the tool. In order to use this model in the tool, you can either specify the absolute path to the model, or you can add it to the trained_networks folder of the tool – more detail on this can be found here.

The default output directory will be:

{DEFAULT_HOME_DIRECTORY}/Documents/amfinder/{DATE_TIME}_train

where DEFAULT_HOME_DIRECTORY is your default home directory, e.g. C:\Users\your-user, and DATE_TIME will be a date time string in the format YYYYMMDD_HHMMSS.

We elaborate further on training best practices here.

04.2.08.02 AM VS ERM TRAINING

The process for training an AM or ErM model is the same – you need labelled training data for all your training images, in the form of AM or ErM annotations. You need to either set "Model for AM" or "Model for ErM" to blank to train a new model or set it to an existing model for retraining – the existing model needs to have the correct number of classes (six for AM, ten for ErM). The model you receive in the output will correspond to the fungal type you have trained for and will have six classes for AM or ten classes for ErM.

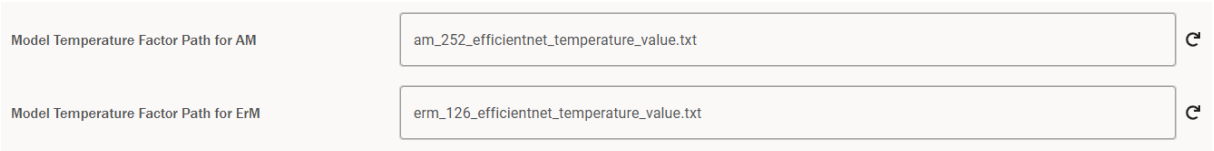
To use this model, you must update the corresponding model path in settings – for example, if you try to use an AM model for ErM images, it will give you an incorrect output. The relevant settings are below – all other training settings are fungal type agnostic.

04.2.08.03 CALIBRATING A TRAINED MODEL

Once a new model has been trained, **you must also calibrate the model** to update the temperature factor before you can test the model or use it for predictions – this is used for scaling probabilities so uncertainty measures can be calculated accordingly.

The process for calibration can be found here – this must be completed before you can use the trained model. You then also need to update the **Temperature Factor Path** – you can find instructions on how to do this here. **IMPORTANT:** this must be done before testing the model and calculating predictions, or the probabilities won't be scaled correctly.

Note that there are two fields for temperature factor path – one for AM and one for ErM. Naturally you need to update the one that corresponds to the fungal type you have trained a model for.



04.2.08.04 SEMI-SUPERVISED TRAINING

There is also the ability to carry out semi-supervised training, which will artificially construct labels to augment the dataset using weak and strong augmentation. To carry this out, one would train as normal using the usual folder structure but would tick the "Semi-supervised" option. This will then run training, augmenting the given images by taking unlabelled tiles and labelling them automatically.

Your data needs to be organised into "train", "test" and "val" folders. You should follow an 80/20 split between train & val/test, and then further split train/val using an 80/20 split respectively.

To run this training, you need to point towards your train/test/val folder as above to tick the "Semi-supervised" option in the Train settings, as below.

Your output will be saved to a folder called fixmatch_results inside the default train folder, which contains the best model and final version of the model as pth files (model_best.pth.tar and checkpoint.pth.tar respectively), a copy of the config (config.yaml) and a list of the labelled data that was used (labelled_data.csv).

Using this output, you can evaluate the output of semi-supervised training on the test directory – this is described in more detail here.

Note: *this functionality still needs development work to understand its efficacy. It is included into the tool but should be treated as an experimental option, and it should be understood that further development work would be necessary to understand if this is a beneficial route to take for model training. Also note that this is currently only implemented for AM models and would need further development work to use for ErM.*

04.2.08.05 ACTIVE LEARNING

Active learning denotes a procedure whereby the model autonomously selects images for labelling that are expected to be the most informative. Subsequently, the selected images are labelled, integrated into the existing labelled dataset, and used to retrain the model, with this iterative cycle repeating as needed. To facilitate this process, your images should be arranged into the following three directories *(**Note**: names of the directories are case specific):*

- **train:** Contains all initially labelled images, which will be employed to create both training and validation sets.

- **BALD:** Contains all unlabelled or partially labelled images. *(**Note**: Once images are acquired through active learning and manually labelled, they should remain in this folder.)*
- **test:** Contains all test images.

The subsequent sections elaborate on the two phases of sample acquisition and model training. Detailed descriptions follow in next sections.

04.2.08.05.01 ACQUIRING SAMPLES FOR LABELLING

To acquire images for labelling, you have to go to the “Active Learning” section as shown in the image below:

The “Output tiles for labelling” checkbox needs to be checked. You would then run the “Train” functionality, passing in the path to parent directory of the input directories as mentioned above.

The settings that are available for you to train are as follows.

Name	Description	Default value	Required
Active Learning Method	Mathematical method for acquiring new samples. Choose from BALD and BatchBALD. *(Note: BatchBALD seems to outperform BALD based on simulation results.)*	BALD	True
Number of samples for labelling	Number of tiles to acquire per file. *(Note: If a given file doesn’t have enough unlabelled tiles, all available unlabelled tiles will be selected.*)	10	True
Number of Monte Carlo Samples	Samples of potential outcomes to assess model uncertainty in predictions. *(Note: More samples improve accuracy but increase computation time)*	50	True

The output of this will be a .csv file in the output directory as specified by the user (or default otherwise) called output_data.csv. It will have the filename, with the row and column of the tile to be labelled within that file. An example of the output is shown below:

	A	B	C
1	file_name	row	col
2	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	11	2
3	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	22	21
4	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	5	23
5	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	23	16
6	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	13	24
7	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	22	5
8	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	39	25
9	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	41	30
10	C:\Users\d80105\Documents\Kew\BALD\BALD\BALD\ABM841_C2_A-B_Default_Extended_10.jpg	27	29

The user should then go and label these tiles in the images specified by file_name by using the Browser – once they are done, they can continue with the next step of retraining as discussed below.

04.2.08.05.02 TRAINING AFTER ACTIVE LEARNING

After active learning, the newly labelled data is added to the existing labelled data (handled by the code automatically). Subsequently the model should be trained for fewer rounds and with a smaller learning rate (step size during training). This helps the model smoothly learn from the new data without forgetting what it already knows. The idea is to improve accuracy without overdoing it or changing too much. *(Note: when training after active learning, make sure that the model path is identical to the one used during sample acquisition for labelling in the previous step.)*

Thus, when you want to train after labelling using the images acquired in the previous step, you need to check the “Train model after active learning” as shown below:

This will disable “Epochs” and “Learning Rate” and enable “Epochs after active learning” and “Learning Rate after active learning” as can be seen in the screenshots below.

This ensures that we train for a lower number of rounds/epochs with a lower learning rate. The **default value for "Epochs after active learning" is 5** in contrast to the default value for "Epochs" which is 50. Similarly, the **default value for "Learning rate after active learning" is 4e-7** in contrast to the default value for approximately 4e-6 *(Note: All other training settings are still valid and changeable as desired).*
The table below re-iterates the details of these two settings unique to training after active learning:

Name	Description	Default value	Required
Learning rate after active learning	The step size for training after acquired images are labelled after active learning. *(Note: lower than usual learning rate)*	4e-7	True
Epochs after active learning	Number of rounds to train after acquired images are labelled after active learning. *(Note: lower than usual learning rate)*	5	True

In general, the workflow around active learning should be that you first train on all the available images using conventional training settings/guidelines as mentioned in the earlier sections. Then if additional training is required and all labelled images have been exhausted, new images to label are acquired using active learning. Make sure to use the trained model for this step. After acquisition, the tiles are labelled by domain experts and the "Training after active learning" settings are used to train again. Again, make sure the same model is used for this. This cycle is repeated as desired.

We expand further on the intricacies of Bayesian Active Learning (BALD) later in the document.

04.2.09 TEST

Once you have trained and calibrated a model, you can calculate metrics for the model on the test dataset to understand its performance using test. This functionality will give summary statistics on the performance of the model.

04.2.09.01 MAIN TESTING WORKFLOW

To trigger a test run, you must specify the model you want to test and the directory of the images you want to test on. Similarly to train, the test set must all be grouped together under a folder called "test" – this should be composed of around 20% of your dataset, as outlined in the training section. Once your images are all grouped together under a test folder, you will want to pass the image path into the input box of the parent folder of the test directory. For example, if the path to your test directory was

"C:\Path\To\Directory\test", the correct input path to the tool would be "C:\Path\To\Directory".

Note: *the test dataset should be completely independent of the dataset used to train the model, to avoid introducing bias to the performance metrics.*

You now also need to specify the path to the model you want to test. If this is a model that is saved in the trained_networks folder of the tool (which one can find in the "_internal" folder that accompanies the exe), then you can just pass the name of the model. Otherwise, one must pass the absolute file path to the model. To summarise, your two options for specifying a model are as follows.

1. **Copying to trained_networks –** you can take a pth file (for example, the pth file that is the output of train) and copy to the "_internal/trained_networks" directory. The _internal folder comes packaged with the executable. Then, when specifying a model in settings, you can just pass the name of the file e.g. "model.pth".
2. **Specifying absolute file path** – if you do not want to copy the file to trained_networks, you can also pass the absolute file path to the model into the model setting e.g. "C:\Path\To\model.pth".

We describe this process in more detail here.

You need to be careful to only specify a model for testing that matches the fungal type you have selected. Then when you trigger testing, it will use the model specified in "Model for AM" or "Model for ErM" for AM and ErM fungal types respectively.

Now you have specified your model and your test images, you can trigger a testing run. Click the Test tile and then click Submit – you will get a notification telling you the testing is in progress. Once the testing is complete, the final line of the console output will specify where the test output is stored. In this folder, one can find the following.

1. **Incorrect predictions by file** – for each image in the test set, there will be a folder with the image name. Inside that folder there will be an image for each class, showing tiles that were misclassified for that particular class.
2. **Incorrect predictions by class** – there will also be several images with the file name incorrect_{class_name}.png, where class_name is the name of each class. This will show all the incorrect images per class.
3. **Confusion matrix** – there will be a file named confusion_matrix.png, which gives the confusion matrix for all tiles across the test set – this is explained further here.
4. **Class Metrics** – inside the included zip file, there will be an excel sheet called class_metrics.csv. This file includes the accuracy, precision, recall and F1 score

for every class, which can be used to inform on how well the model performed on the test set. These terms are described in detail here.

5. **File Metrics** – this file has detailed metrics for each image in the test set. This includes:
    a. Metrics on the percentage colonisation per image (actual and predicted),
    b. Accuracy, precision, recall and F1 score for each class within a particular image,
    c. The number of actual tiles and predicted tiles for each class within a particular image.
6. **Generic metrics** – this file gives the accuracy, loss, macro recall, macro precision and macro F1 score across the whole test set. These can be used to judge the performance of the model as a whole and are described further here.
7. **Number of tiles below certain threshold** – This csv file contains the percentage of tiles, i.e. number of tiles in test set divided by total tiles in test set, where the max probability of the tested model is below a certain threshold. The thresholds under consideration are 0.1, 0.2, 0.3, ..., 1.0.
8. **Class Metrics after manual labelling** – This folder contains csv files with class metrics for each threshold assuming that every tile with max probability output by the model, which is lower or equal to the threshold, was manually reviewed and labelled and hence would get the correct label.

The contents of all the above outputs are outlined in detail here.

The default output directory for the output of test will be:

```
{DEFAULT_HOME_DIRECTORY}/Documents/amfinder/{DATE_TIME}_test
```

where DEFAULT_HOME_DIRECTORY is your default home directory, e.g. C:\Users\your-user, and DATE_TIME will be a date time string in the format YYYYMMDD_HHMMSS.

Finally, you will also see the output in the console for test set accuracy, macro F1 and the precision, recall, accuracy and F1 score for each class – for an AM model, this will look like the below.

04.2.09.02 AM VS ERM TESTING

Naturally there are several differences when training an AM vs and ErM model. The first (and potentially most important!) one is that the training data itself will differ – naturally, to test an AM model, we need labelled AM images, and to train an ErM model we need labelled ErM data. We can label this data using the Browser.

Moreover, the outputs will differ for AM and ErM due to the different classes of the models. For each output, we will get metrics that relate to either the AM classes or the ErM classes. These can be interpreted in the same manner – it is worth noting that the ErM models have more classes than AM, and thus are slightly more complex to interpret.

04.2.09.03 USING CONTEXT IN TEST

Similarly to predict, during testing we can calculate contextual predictions for all tiles whose maximum confidence falls below a certain threshold, and we can then use these contextual predictions to calculate metrics for the model performance.

When we use context in test, the predictions will be carried out as normal, and then for every prediction that has a maximum confidence below the threshold we specify via the setting “Contextual Confidence Threshold” will be re-calculated using the maximum vote methodology including the four surrounding tiles – this is detailed here. This looks like the below in the console.

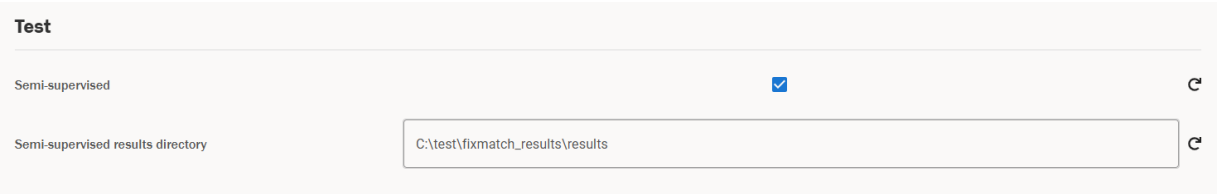
We can see that this recalculates predictions with max voting on an image-by-image basis. After this is completed, we get a summary of the class changes in the console – this shows the changes per class. There is a complimentary file called **tile_class_changes.csv** which tells you every individual tile that has been changed, including the image it is from, the row and column of the tile, the class it has changed from and to and the original confidence of the tile. The summary in the console looks like the below.

```
=== Summary of Class Changes ===.
[12:48:31] INFO: Background -> BlueCoils: 4 changes.
[12:48:31] INFO: Background -> BrownCoils: 3 changes.
[12:48:31] INFO: Background -> DSE: 2 changes.
[12:48:31] INFO: Background -> MainRoot: 3 changes.
[12:48:31] INFO: Background -> TypeTwo: 1 changes.
[12:48:31] INFO: Background -> Uncolonised: 5 changes.
[12:48:31] INFO: Background -> Unreadable: 3 changes.
[12:48:31] INFO: BlueCoils -> Background: 13 changes.
[12:48:31] INFO: BlueCoils -> BrownCoils: 5 changes.
[12:48:31] INFO: BlueCoils -> HybridErm: 1 changes.
[12:48:31] INFO: BlueCoils -> MainRoot: 8 changes.
[12:48:31] INFO: BlueCoils -> TypeTwo: 11 changes.
[12:48:31] INFO: BlueCoils -> Uncolonised: 41 changes.
[12:48:31] INFO: BlueCoils -> Unreadable: 12 changes.
[12:48:31] INFO: BrownCoils -> Background: 11 changes.
[12:48:31] INFO: BrownCoils -> BlueCoils: 8 changes.
[12:48:31] INFO: BrownCoils -> MainRoot: 9 changes.
[12:48:31] INFO: BrownCoils -> Uncolonised: 36 changes.
[12:48:31] INFO: BrownCoils -> Unreadable: 5 changes.
[12:48:31] INFO: DSE -> MainRoot: 1 changes.
[12:48:31] INFO: HybridErm -> Background: 1 changes.
[12:48:31] INFO: HybridErm -> BlueCoils: 5 changes.
[12:48:31] INFO: HybridErm -> BrownCoils: 4 changes.
[12:48:31] INFO: HybridErm -> Uncolonised: 1 changes.
[12:48:31] INFO: MainRoot -> Background: 12 changes.
[12:48:31] INFO: MainRoot -> BlueCoils: 8 changes.
[12:48:31] INFO: MainRoot -> BrownCoils: 14 changes.
[12:48:31] INFO: MainRoot -> TypeTwo: 4 changes.
[12:48:31] INFO: MainRoot -> Uncolonised: 24 changes.
[12:48:31] INFO: MainRoot -> Unreadable: 17 changes.
[12:48:31] INFO: TypeTwo -> Background: 3 changes.
[12:48:31] INFO: TypeTwo -> BlueCoils: 16 changes.
[12:48:31] INFO: TypeTwo -> MainRoot: 1 changes.
[12:48:31] INFO: TypeTwo -> Uncolonised: 4 changes.
[12:48:31] INFO: TypeTwo -> Unreadable: 5 changes.
[12:48:31] INFO: Uncolonised -> Background: 42 changes.
[12:48:31] INFO: Uncolonised -> BlueCoils: 55 changes.
[12:48:31] INFO: Uncolonised -> BrownCoils: 47 changes.
[12:48:31] INFO: Uncolonised -> MainRoot: 34 changes.
[12:48:31] INFO: Uncolonised -> TypeTwo: 20 changes.
[12:48:31] INFO: Uncolonised -> Unreadable: 6 changes.
[12:48:31] INFO: Unreadable -> Background: 24 changes.
[12:48:31] INFO: Unreadable -> BlueCoils: 19 changes.
[12:48:31] INFO: Unreadable -> BrownCoils: 5 changes.
[12:48:31] INFO: Unreadable -> MainRoot: 13 changes.
[12:48:31] INFO: Unreadable -> TypeTwo: 9 changes.
[12:48:31] INFO: Unreadable -> Uncolonised: 14 changes.
[12:48:31] INFO: Total changes: 589.
[12:48:31] INFO: ==============================
```

The rest of the output will then be the same format; however the metrics will use the relabelled contextual labels instead of the original label that is derived from the maximum confidence class of the tile.

04.2.09.04 SEMI-SUPERVISED TESTING

If you have carried out semi-supervised training, you can test the resulting model as follows. Pass in the parent directory of the test directory into the image input as usual, and now go to the test settings and add the results directory you received via training to the "Semi-supervised Results Directory" field in settings. Note that you will get this path as the last line in the console when running semi-supervised training. This will only be enabled if semi-supervised is ticked – see below.



Now hit "Test", and the testing functionality will be carried out as normal. You will get an output under a folder called test_results in the fixmatch results folder you specified earlier, which contains very similar metrics to the test functionality, including F1 score and incorrect prediction tiles for each class. This includes:

- Files names {class_name}_incorrect_predictions.png, which show you the incorrect predictions for each class.
- Confusion_matrix.png which shows you the confusion matrix for the training run.
- File_metrics.csv which will output the predicted colonisation metrics for each input image.
- Test_metrics.csv which ouputs per-class accuracies, macro F1 and test set accuracy.

Further details on semi-supervised training can be found here.

Note: *this functionality still needs development work to understand its efficacy. It is included into the tool but should be treated as an experimental option, and it should be understood that further development work would be necessary to understand if this is a beneficial route to take for model training. Also note that this is only available for AM testing and not ErM.*

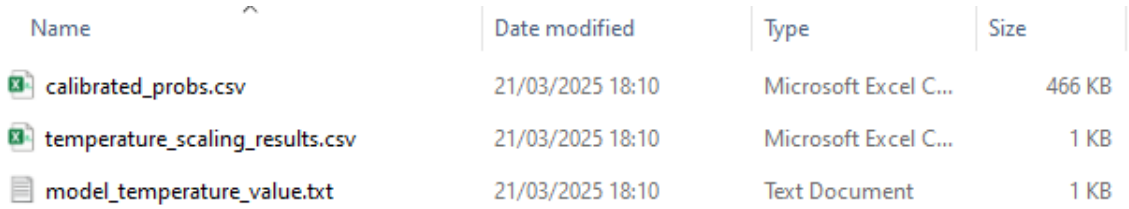
04.2.10 CALIBRATE

Calibration is a necessary technique that we carry out on a newly trained model to calculate a value called the temperature factor. This temperature factor is used to balance the probabilities that are output by the tool when calculating predictions, such that the final output probabilities match the confidence of the model closely. With the calibrated probabilities, we are able to capture uncertainty and calculate uncertainty thresholds for the colonisation metrics of a prediction.

To calibrate a model, we need the path to the relevant model pth file, which can either be a pth file inside the trained_networks folder, or it can be an absolute file path to a pth file – the difference is explained here.

Once you have specified the relevant model in the "Model for AM" or "Model for ErM" setting (depending on the type of model you are calibrating), all you need to do is point to your labelled data and hit calibrate. Note that in this case, **the more labelled data you can provide the model the better** – the code will look for a folder called "train" underneath the directory you pass to the tool and as mentioned above, you should add as many labelled images to this folder as possible. Naturally, the labelled data needs to match the fungal type you are processing – for AM we need AM annotations, and similarly for ErM we would need images with ErM annotations.

The output of the calibration (which is either saved under the directory you specified in the "Output directory" field, or the default directory) is seen below.



The content of each of these files is as follows.

1. **Calibrated_probs.csv -** This file contains the calibrated predictions for each tile in the dataset provided for the calibration procedure.
2. **Temperature_scaling_results.csv -** Here one can find the output of the calibration training procedure, meaning the loss and performance metrics.
3. **Model_temperature_value.txt -** This file contains a single scalar value, which is output of the calibration procedure of a model and called the temperature factor.

This is expanded on in more detail here.

The default output directory will be:

```
{DEFAULT_HOME_DIRECTORY}/Documents/amfinder/{DATE_TIME}_calibrate
```

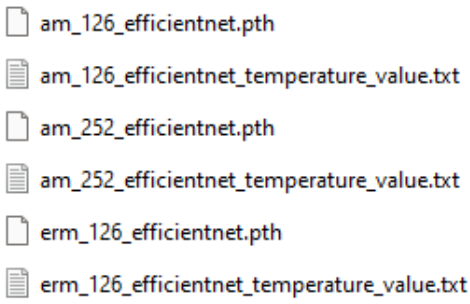
where DEFAULT_HOME_DIRECTORY is your default home directory, e.g. C:\Users\your-user, and DATE_TIME will be a date time string in the format YYYYMMDD_HHMMSS.

IMPORTANT: to use a model for predictions and also to correctly test the model, it must be accompanied by the relevant temperature factor. Once you have calibrated a model

as above, you must also update the “Model Temperature Factor Path for AM” or “Model Temperature Factor Path for ErM” in the settings to point to the **model_temperature_value.txt** file that you calculated above that corresponds to the model. Naturally you need to complete this process for AM and ErM models separately. You can update these values using general settings.

04.2.11 TRAINED NETWORKS

The existing models in the trained_networks folder are as follows.



- am_252_efficientnet.pth and the corresponding temperature value file are the best performing AM model for 252 tiles – this model is the default for AM predictions.
- erm_126_efficientnet.pth and the corresponding temperature value file are the best performing ErM model for 126 tiles – this model is the default for ErM predictions.
- am_126_efficientnet.pth and the corresponding temperature value file are the best performing AM model for 126 tiles. You will need to change tile size to 126 to be able to use this model for predictions and update the Model for AM and Model Temperature Factor Path for AM settings accordingly.

One can see that each trained network has the corresponding temperature factor, with the naming convention of {model_name}_temperature_factor.txt, where model_name is the name of the model.

Naturally the chosen pth file needs to match the requirements of the network, e.g. if you are using it for level one AM predictions, it needs to have been trained on AM data and have the correct number of classes.

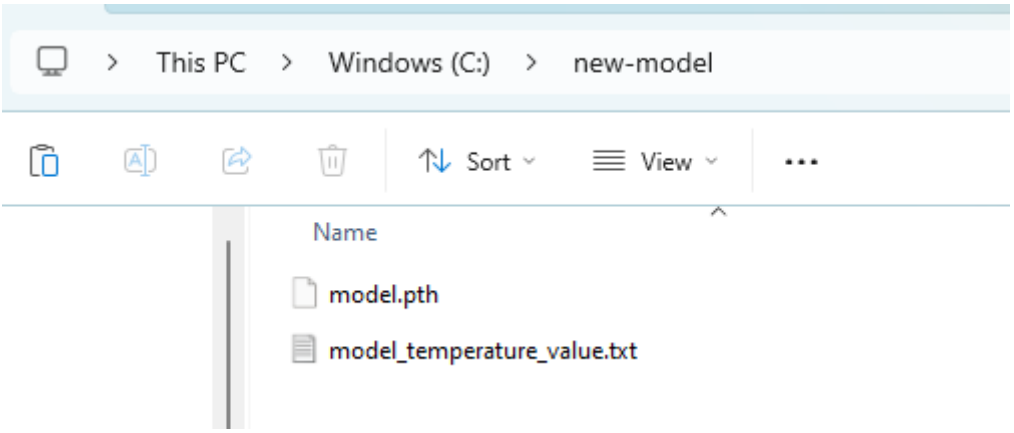
When specifying a model, a user has two options.

1. They can use a pre-defined model that is saved in the trained_networks folder.
2. They can specify an absolute path to a pth file that they have chosen, that e.g. might be the output of train.

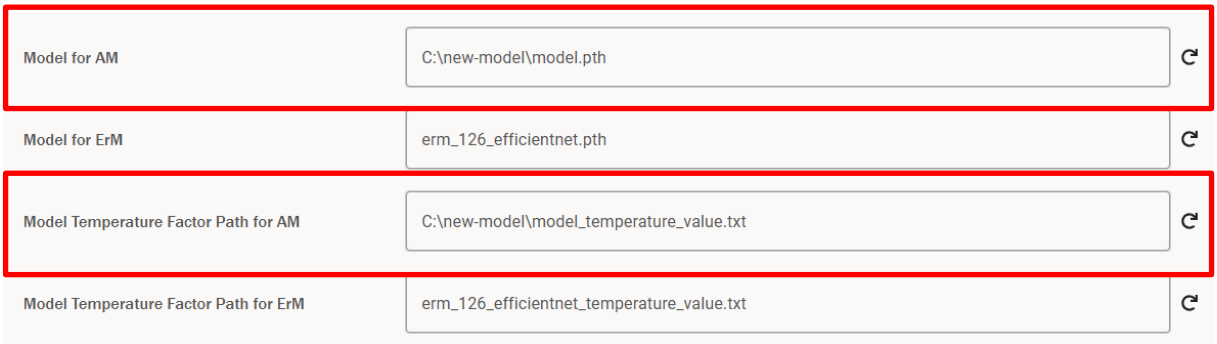
For option 2, the user needs to change the “Model for AM” or “Model for ErM” setting for AM and ErM respectively and the corresponding “Model Temperature Factor Path for

AM" or "Model Temperature Factor Path for ErM" setting respectively to the absolute file path to these values.

For example, let's assume you have a model pth file that you got via Train for AM fungus type, and a Temperature Factor file that you got via running calibrate on this model, and you have stored them both in the folder C:\new-model as below.



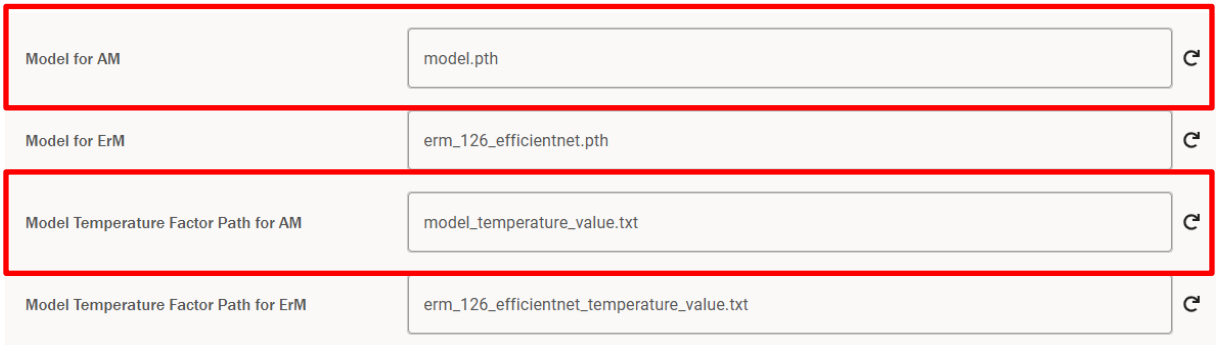
To use this model in the tool, you would have to update both the "Model for AM" and "Model Temperature Factor Path for AM" settings to point to these files – we can see this below.



However, if you wanted to use this model and temperature factor without having to specify the absolute file path, you can add these directly to the trained_networks folder for the tool. To do this, go to the directory where your exe is stored – there should be a folder named "_internal" next to it (looks like the below).

Click into _internal and find the directory called "trained_networks" as below.

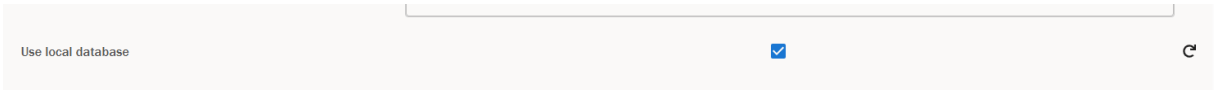
Now copy paste your model pth and temperature factor file into this folder. You can now just specify the name of the model and temperature factor file, instead of having to specify the absolute file path.



This would be a similar story for ErM – you would merely have to update the corresponding ErM fields instead of AM.

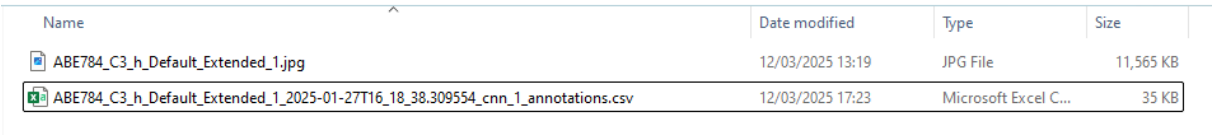
04.2.12 USING CSVS

The main day-to-day usage of the tool is intended to use the database for storing prediction and annotation data. However, there are certain situations where it is appropriate to use CSVs for training purposes, for example when training on detached infrastructure. To use CSVs for every functionality in the tool, there is a single setting that one needs to change – this is the “Use local database” setting, which can be found under the General Settings.



If this is checked (which is the default), each functionality of the tool will search for annotations/predictions in the database. If it is unchecked, the tool will check locally for annotations/predictions in the form of CSVs that **match the image name in question and are stored in the same directory.**

For example, if one tries to carry out a training run with this box unchecked, then the necessary directory structure would be directories including the jpgs, with corresponding csv annotations named as below.



In the above, the CSV file has the naming convention

{image_name}_{timestamp}_{level}_{type}

In the above, the variables in braces are defined as follows.

1. {image_name} – this is the name of the image the CSV relates to.
2. {timestamp} – this is the timestamp used to uniquely identify the set of annotations/predictions.
3. {level} – this corresponds to whether the csv corresponds to network 1 or 2 and will take value cnn_1 or cnn_2 accordingly.
4. {type} – this will either be annotations or predictions.

This is the **exact naming convention that you will get when you download CSVs from the tool database** – there is more detail on this in the next section. It is vital you have this naming convention, otherwise the tool will not link the image to the CSV.

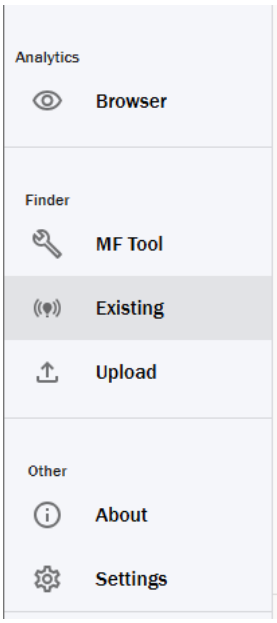
Note that all the output will also be in CSV format, i.e. predictions would be saved as a CSV and not in the database.

04.3 EXISTING PREDICTIONS AND ANNOTATIONS

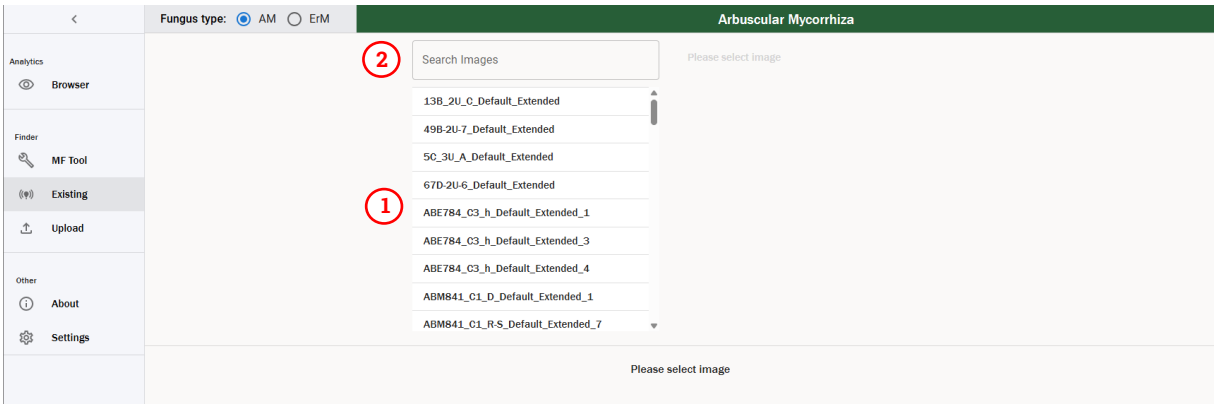
In the section of the Finder labelled Existing, you can view predictions and annotations that are stored in the Postgres database. You can also download any of these predictions and annotations to CSV files.

04.3.01 VIEWING AND DOWNLOADING EXISTING PREDICTIONS AND ANNOTATIONS

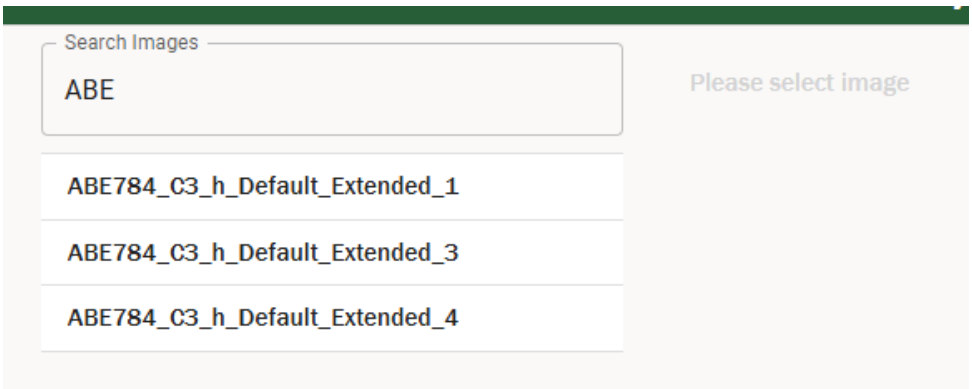
In order to access this functionality, you need to navigate to the Existing section via the sidebar:



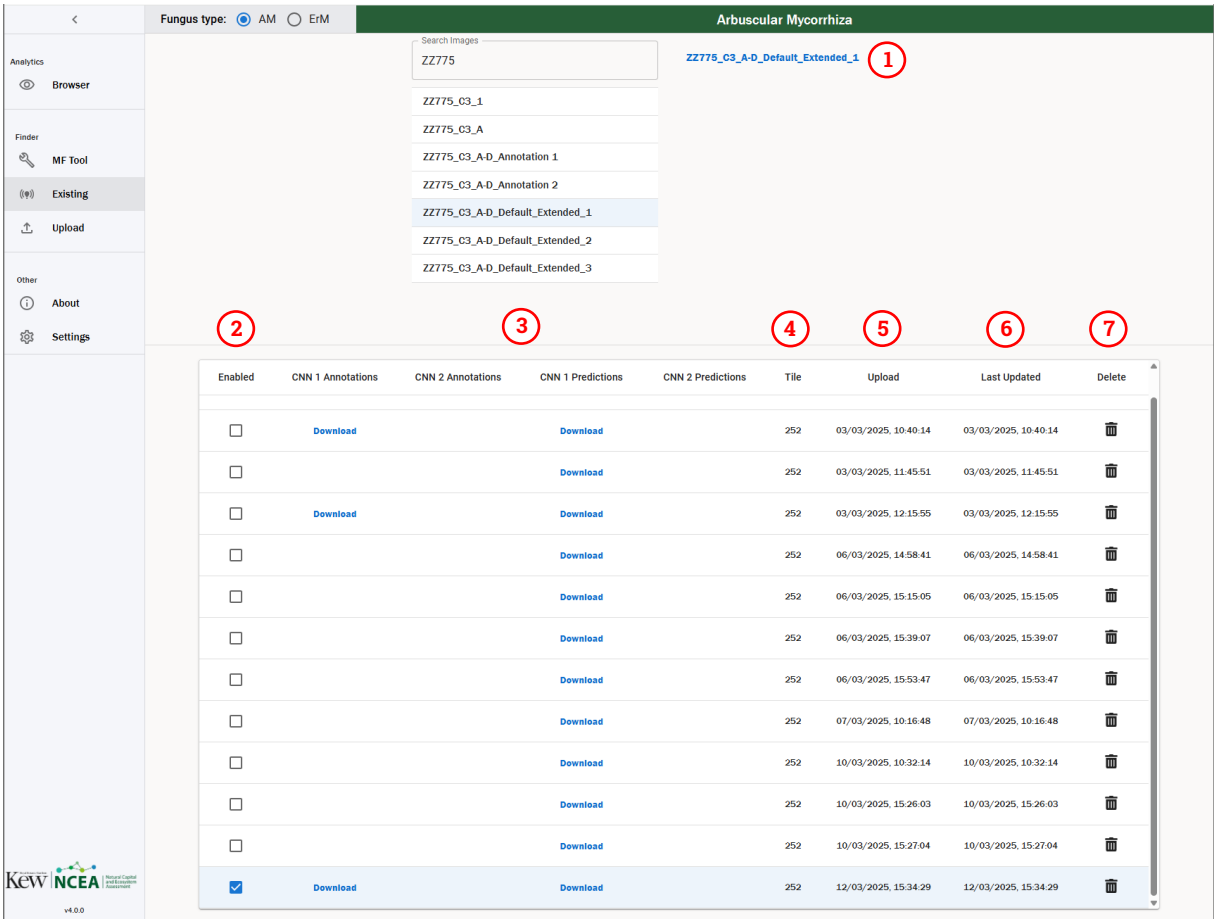
When you click this, you will be greeted with a screen as below.



1. Each of the names in the list correspond to an image that has predictions and/or annotations stored in the database. These should match the name of the image in your local folder structure.
2. This is a search bar that can be used to filter the image names – we see this below.



You can select any image to view the annotations and predictions that are saved down for that particular image – for this example, we will choose ZZ775_C3_A-D_Default_Extended_1. The table looks like the below.



1. **Image name** – this specifies the name of the image you are viewing.
2. **Enabled** – for each image that has annotations stored in the database, one such set of annotations must be enabled. This is then the set of annotations that is used by the tool for any functionality that ingests annotations, for example testing models, training new models, calculating colonisation percentages or upscaling annotations. **It is vital that before carrying out any of these functionalities, you have enabled the set of annotations that you want to use.** You can find more information on this here.
3. **Existing values** – these columns tell what annotations/predictions exist in the database for a particular entry. By clicking the corresponding Download button, you can download a CSV of annotations or predictions for CNN1 or CNN2. This will download a CSV of the annotations or predictions to your Downloads folder. You can use this CSV for non-local training/testing and to transfer annotations or predictions between machines.
4. **Tile Edge** – this shows the tile edge for the set of predictions/annotations.

5. **Upload Timestamp** – this specifies the time stamp of the *first time you saved* this set of predictions or annotations. It is used as the unique identifier for a set of predictions or annotations for a particular image.
6. **Last Updated Timestamp** – this specified the time stamp of the time this entry was last updated, e.g. if you altered the annotations and saved them to the database.
7. **Delete** – this can be used to delete a particular entry in the database, removing all annotations and predictions associated with that row. When you click this, a modal will pop up to confirm that you want to delete that row. This is an irreversible action, so be careful!

Note that when you download predictions or annotations, the file naming convention is as follows.

{image_name}_{timestamp}_{level}_{type}

In the above, the variables in braces are defined as follows.

1. {image_name} – this is the name of the image you are downloading predictions or annotations for, which corresponds to 1) in the above image.
2. {timestamp} – this is the timestamp used to uniquely identify the set of annotations/predictions, which corresponds to 2) in the above image.
3. {level} – this corresponds to whether you are downloading a csv for network 1 or 2 and will take the value cnn_1 or cnn_2 accordingly.
4. {type} – this will either be annotations or predictions, depending on whether you are downloading annotations or predictions respectively.

Note that the annotations and predictions shown will depend on the fungal type selected – e.g. if an image has AM annotations/predictions but not ErM annotations/predictions, then if we change the fungal type to ErM we will just see an empty table.

04.3.02 ENABLED PREDICTIONS AND ANNOTATIONS

As mentioned above, for each image there will be one set of enabled annotations/predictions. This set of annotations/predictions is the entry that the tool will use for any functionalities that need annotations/predictions to run. This includes converting predictions to annotations, aggregating tiles to a larger size, training models, testing models, and calibrating models.

You can change the enabled set of annotations/predictions by changing the row that has enabled ticked. Note that you can only enable one row.

Also, if you save a set of annotations via the Browser or if you upload a set of annotations, this set of annotations **automatically becomes enabled.**

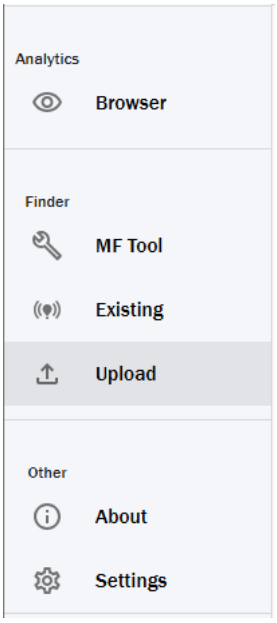
We would highly recommend checking which set of annotations/predictions is enabled before you carry out any functionality that relies on these things, as otherwise you could get unexpected results.

04.4 UPLOAD PREDICTIONS AND ANNOTATIONS

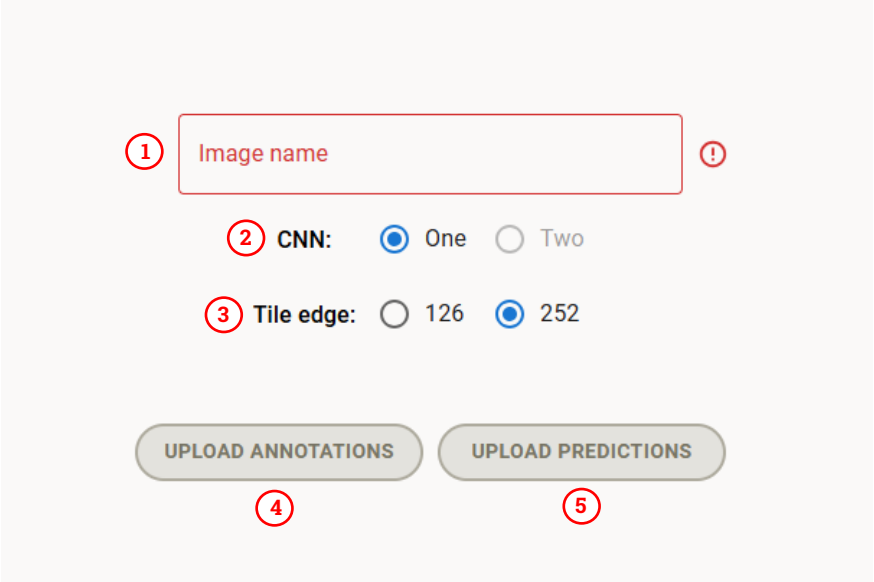
This functionality is used to upload CSVs of predictions or annotations to the tool. It can be used to transfer predictions and annotations between different machines, and to upload legacy predictions and annotations from previous versions of the tool (which come in the form of tsv files).

04.4.01 UPLOADING

In order to access this functionality, you need to navigate to the Upload section in the sidebar:



When you click this, you will be greeted with the below screen.



1. **Image name** – this is the input box where you specify the image name you are uploading predictions or annotations for. This must be the **exact image name without file path or extension.** So, for example, if I wanted to upload annotations for an image stored at C:\Path\To\Folder\image_name.jpg, I would enter image_name into this text input. If it has a red border, it means it is invalid, and there will be a tooltip on the right hand side.
2. **CNN selector** – this specifies whether you are uploading annotations/predictions for CNN1 or CNN2. Currently CNN2 is disabled.
3. **Tile size** – this specifies the tile size you are uploading images for. By default, this is set to 252 for AM and 126 for ErM.
4. **Upload predictions** – click this button to upload predictions. The predictions must be in csv or tsv format and must have the following header format for AM (case sensitive and order matters!). Note that ContextualLabel is optional.

row	col	AMColonised	Uncolonised	Background	Unreadable	DSE	Hybrid	ContextualLabel

And for ErM, the format is as follows (split over two rows).

row	col	BlueCoils	BrownCoils	TypeTwo	Uncolonised	Background

MainRoot	Unreadable	DSE	HybridErM	HybridDSE	ContextualLabel

5. **Upload annotations** – this button can be used to upload annotations. You can use this to either upload legacy annotations or transfer annotation files between machines. The annotations must be in csv or tsv format and must have the

following header format for AM, where Question and QuestionComment columns are optional.

row	col	AMColonised	Uncolonised	Background	Unreadable	DSE	Hybrid	Question	QuestionComment

And similarly, for ErM (once again split over two rows).

row	col	BlueCoils	BrownCoils	TypeTwo	Uncolonised	Background

MainRoot	Unreadable	DSE	HybridErM	HybridDSE	Question	QuestionComment

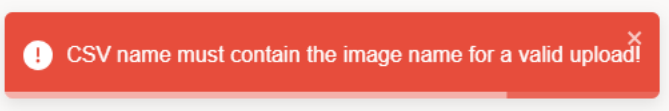
After inputting your image name, selecting your tile size and clicking either upload predictions or annotations, you will be greeted with a file explorer. From here, you must select the relevant csv or tsv format as outlined in 3) and 4) above. Select the relevant file, and you should be greeted with the output of "Upload successful!" on the button text. You can now view the uploaded predictions and annotations via the Browser.

Naturally the type of annotation or prediction you are uploading needs to match the fungus type you have selected in the tool. If you try to upload AM annotations whilst ErM is selected, then you will get an error that you are missing mandatory columns. These errors are discussed further in the next section.

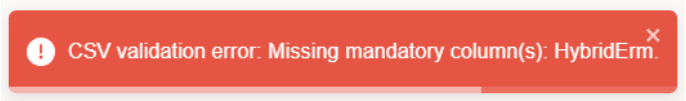
04.4.02 DEBUGGING ERRORS

There are several errors that can occur here – we discuss them below.

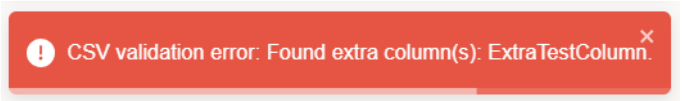
1. **Image name does not match** – the annotations/predictions file you are uploading must contain the image name in the name of the CSV, as when you download annotations or predictions it contains the image name. If your CSV does not contain the image name, you will get the following error. The solution is to update your CSV to contain the image name – however you should check you are uploading the correct thing, as if you have not renamed it then chances are that there is a mistake!



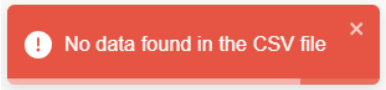
2. **Missing mandatory columns** – if you upload a CSV that does not have all the mandatory columns for the relevant fungus type then you will get an error, which will tell you what columns are missing. Double check and make sure the headings in your CSV match the fungus type you are trying to upload.



3. **Extra columns** – if you have a column that does not belong in the classes for the relevant fungus type you are uploading, you will throw an error. Double check and make sure the headings in your CSV match the fungus type you are trying to upload.



4. **No data in CSV –** if you try to upload a CSV that has the correct headings but no data, you will get the following error.



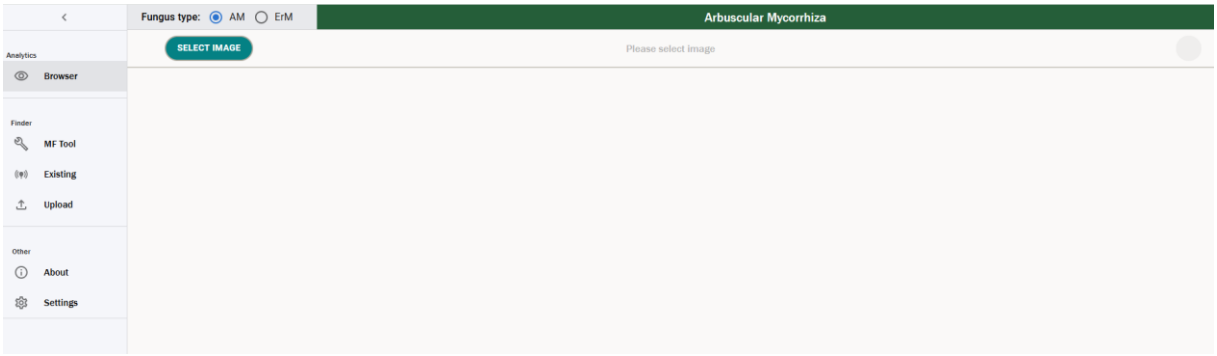
If your upload fails, you should first check the console output for an error. More details on how to debug this are here. You should also double check that the columns of your file match the above. Moreover, if you accidentally upload predictions to annotations or vice versa, this will fail. Make sure you are using the correct button to upload the relevant file type.

05. BROWSER

The Browser is the user interface for viewing predictions and annotations. This is the main place where a user can analyse the output of the MF Tool, converting predictions to annotations and verifying the output, and create new labelled training data for further model training.

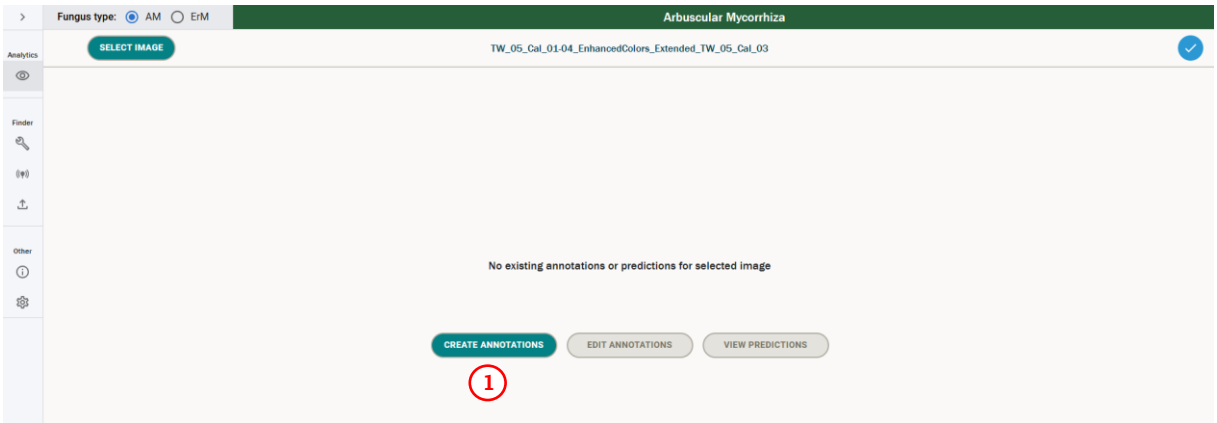
05.1 OPENING AN IMAGE

To access the browser, one must click on the “Browser” tile on the home screen, or on the Browser section in the sidebar. You will be greeted with the following screen.

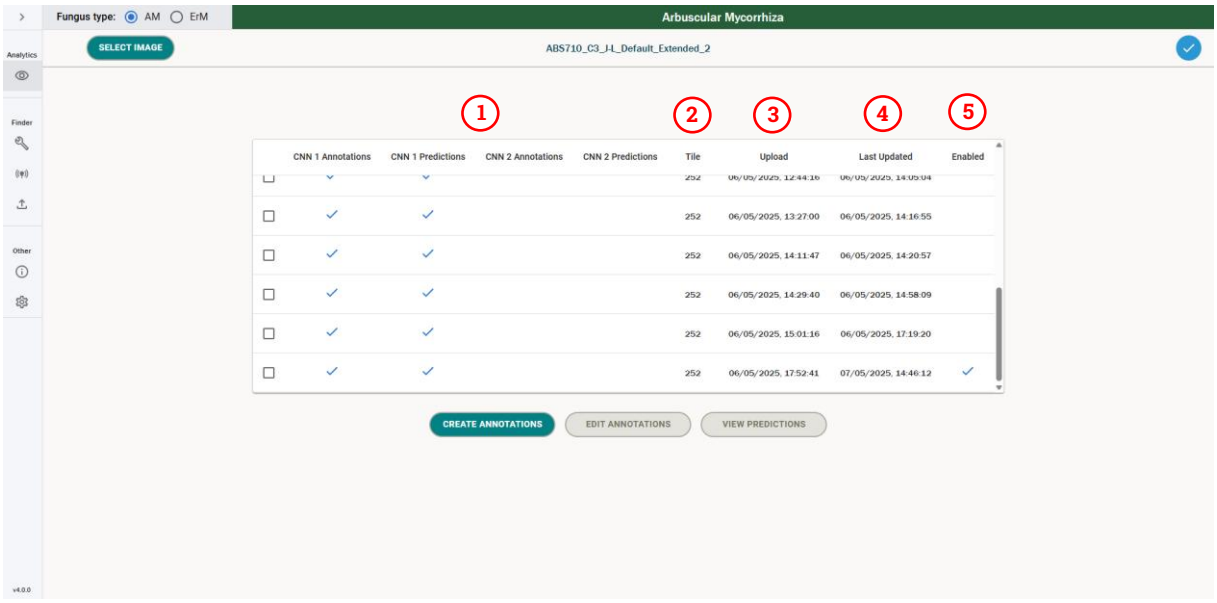


Click Select Image in the top left – this will open a file explorer. Navigate to the image you want to view and open it. The file explorer will then close, and you will see one of two screens.

- If there are no predictions or annotations stored for an image for the fungus type you have selected, then you will see the below screen. You can then click the create annotations button (1) to generate a new set of annotations for that image – you can find more detail on this here.



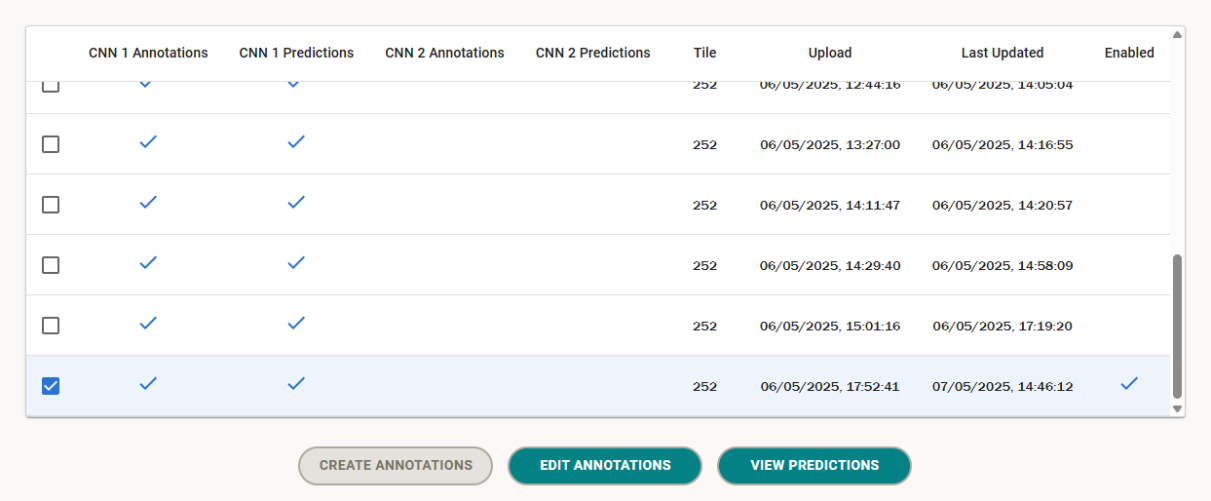
- If predictions or annotations exist for the image you have selected for the particular fungus type you have selected, you will see the below screen.



You will notice that this is a very similar view to the one we saw earlier in the Existing Predictions & Annotations screen. The meanings of each column are much the same – they are outlined below.

1. **Contains** – this shows you what predictions and annotations the image in question has saved in the database.
2. **Tile Edge** – this shows the tile edge for the set of predictions/annotations.
3. **Upload** – this specifies the time stamp of the *first time you saved* this set of predictions or annotations. It is used as the unique identifier for a set of predictions or annotations for a particular image.
4. **Last Updated** – this specifies the time stamp of the *most recent time you saved* this set of predictions or annotations. It could differ from the upload by e.g. you updating the annotations.
5. **Enabled** – this denotes which set of annotations/predictions is enabled for the image – you cannot update this here, you can only update enabled via the Existing Predictions & Annotations screen.

You now have two options – by selecting a row from the table, you can view that set of predictions and/or annotations for the image. When you select the row, the "Edit Annotations" and "View Predictions" will become enabled, depending on whether annotations or predictions exist for that image respectively – this looks like the below.



You can also select the “Create new annotations” button, which will open an empty set of annotations for you start labelling. This will only be enabled if you have not selected a row.

05.1.01 AM VS ERM IMAGES

When selecting images, you will only see existing predictions/annotations for the colonisation type you have selected. For example, if I have calculated an AM prediction for an image, but I have selected the image whilst I have ErM selected as my fungus type, I will not see that prediction as part of the grid. Make sure you have selected the correct colonisation type that you want to view.

The main difference between AM and ErM images are the classes that are available – for AM, the classes are as follows.

- AM Colonised
- Uncolonised
- Background
- Unreadable
- DSE
- Hybrid

And for ErM they are as follows.

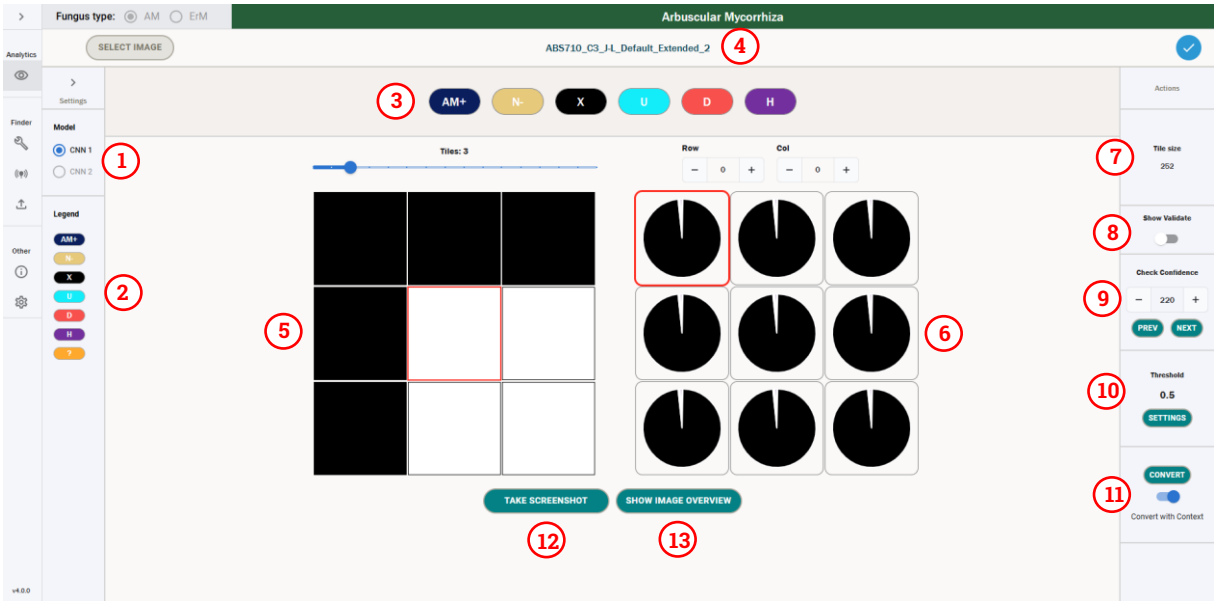
- Blue Coils
- Brown Coils
- Type Two Colonised
- Uncolonised
- Main Root
- Background
- Unreadable

- DSE
- Hybrid DSE
- Hybrid ErM

The formal definitions of these classes are provided in [4] and [6]. We see how the UI differs for AM vs ErM images in the following sections.

05.2 VIEWING PREDICTIONS

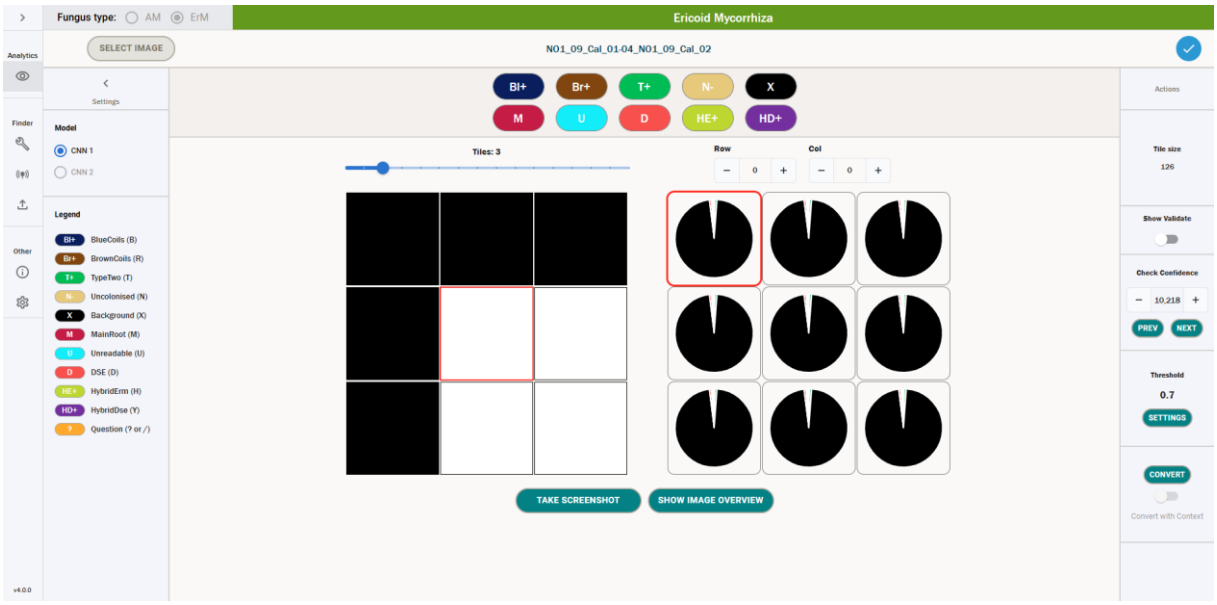
If you select a row from the above that has CNN1 predictions and then click “View Predictions” you will be greeted with the below screen.



1. **Network selector** – this section is used to switch between CNN1 and CNN2. Currently CNN2 is disabled for predictions, so you can only select CNN1.
2. **Legend** – this details the different classes for the fungus type you have selected. You can expand the sidebar with the arrow at the top to show the full names and key bindings for each class.
3. **Icon selector** – this can be used to manually add an annotation. This will override the prediction value. This can be used when reviewing predictions to manually override the models prediction with a human label. In particular, this is useful when reviewing uncertain predictions – the annotation functionality is described in more detail here. Note that the icons will differ based on whether you are viewing an AM or an ErM prediction.
4. **Image name** – this specifies the name of the image currently selected.
5. **Image grid** – this shows the tiles of the image you are viewing. The tile with the red border corresponds to the tile with the red border in the predictions grid. The image grid can be zoomed in and out with the slider above it.

6. **Predictions grid** – this grid is used to show individual tile predictions. It consists of pie charts for each tile, with the probabilities that that tile belongs to a certain class as the slices of the chart. This is detailed further here. Note that the prediction classes will differ based on whether you are viewing an AM or ErM prediction.
7. **Tile size** – this specifies the size of the tiles for the image you are currently viewing. By default, this will be 252 for AM and 126 for ErM images.
8. **Show Validate** – this toggle will open a section that can be used to click through tiles that are ranked from least certain to most certain, using ascending standard deviation of the predictions as an uncertainty measure. By default, this is not shown, as the confidence is a more valid measure of uncertainty for our use case. More details on this can be found here.
9. **Check Confidence** – this section of the tool can be used to click through tiles from lowest confidence to the highest confidence, where confidence is the value of the prediction for the class with maximum confidence for a particular tile. Tiles with a maximum confidence less than the threshold will have a red background and will not be automatically converted when a user hits convert. More detail on this can be found here.
10. **Threshold** – this specifies the threshold value for which the maximum confidence of a prediction must be higher than to be automatically converted to an annotation. More details on this can be found here.
11. **Convert –** this button can be used to automatically convert predictions to annotations that are above a certain threshold. Annotations can be converted with or without context – this process is described further here.
12. **Screenshot** – this will take a screenshot of the prediction viewer, which is saved down to your local machine – more details can be found here.
13. **Image Overview –** using this, you can see an overview of the entire image, where the tiles are coloured according to either the maximum prediction value or a manual annotation. More details can be found here.

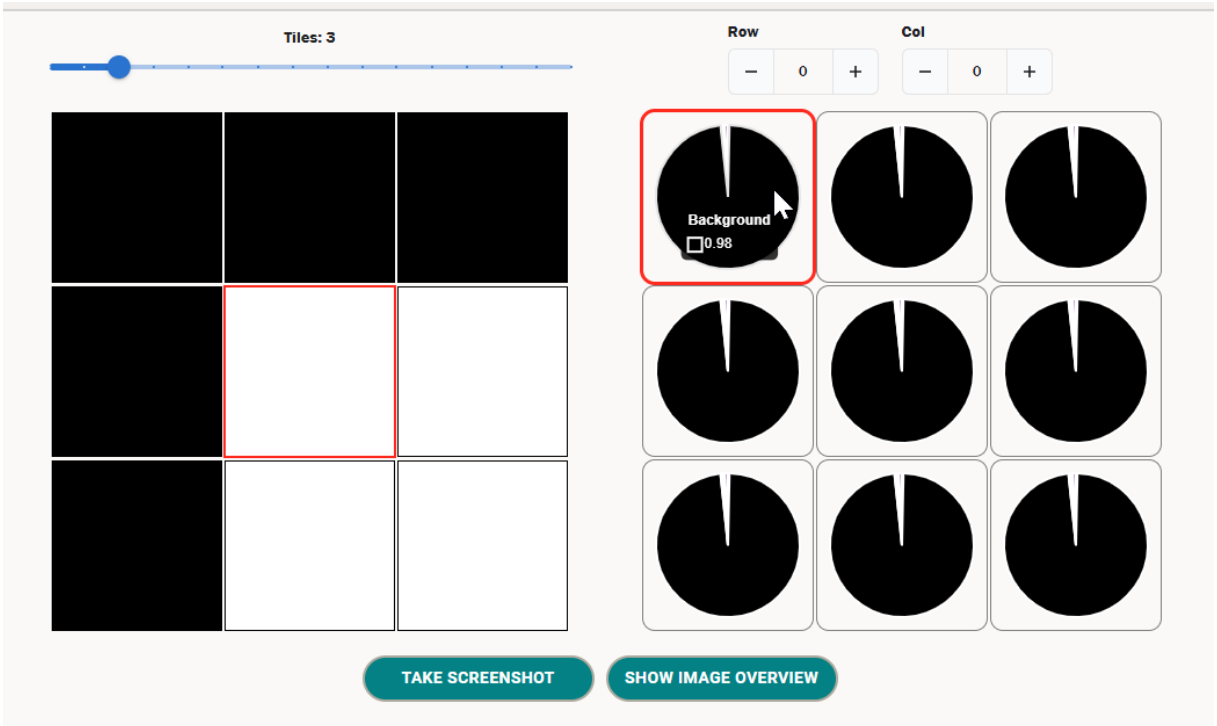
One can also see what the prediction grid looks like for ErM images below. The functionality is the same – we merely have ErM classes instead of AM classes.



05.2.01 WHAT IS A PREDICTION?

A prediction is a list of probabilities that a tile belongs to each class that is calculated by a computer vision model as described here. The higher the probability associated to a particular class, the more likely that the model believes that that tile belongs to that class.

These predictions are displayed in the user interface as pie charts. By hovering over the different segments of the pie chart, one can see the probabilities for each class for a particular tile. We can see below that for the tile we have selected, the probability that it belongs to the Background class is 0.98.



All the prediction values will be between 0 and 1 inclusive.

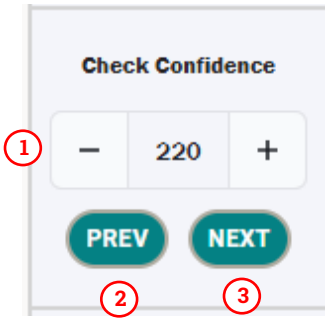
05.2.02 LOW CONFIDENCE PREDICTIONS

The predictions the tool outputs will range between very confident predictions, where the tool has a clear preference of a single class, and less confident predictions, where the tool assigns similar probabilities to a number of classes.

In order to deal with uncertain predictions, we would expect an expert to validate cases where the model has low confidence predictions. To accommodate this, the tool has two functions – one to check the confidence and one to check the standard deviation.

05.2.02.01 CHECKING CONFIDENCE

The first (and main) method for checking confidence is using the "Check Confidence" section. This allows a user to scroll through predictions from the lowest confidence (index 0) to the highest confidence (highest index), where confidence is the value of the maximum prediction for that tile. The section looks like the below:

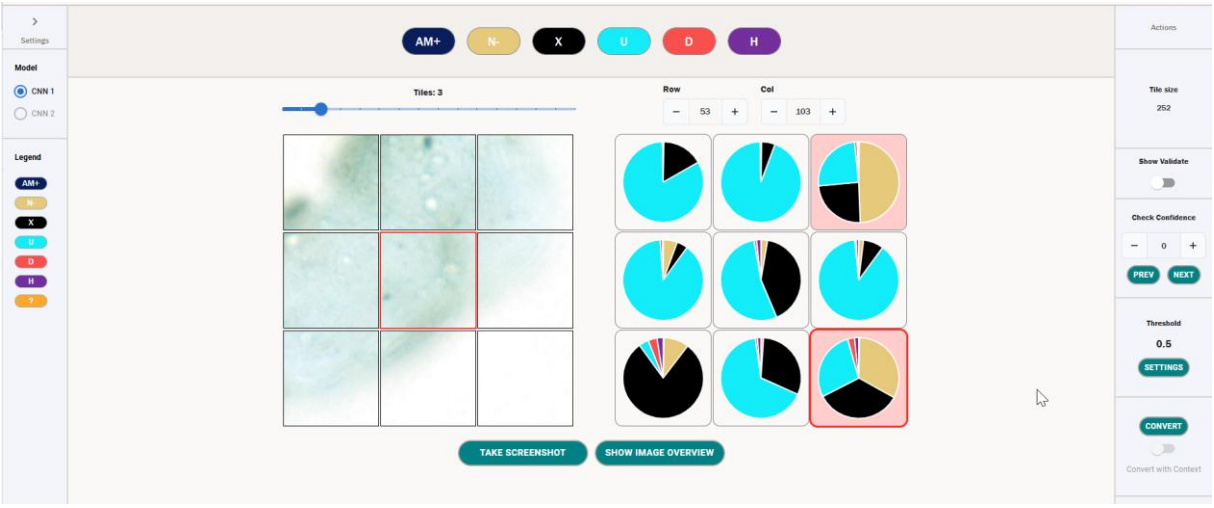


1. **Index** – this is the rank of uncertainty where **the tile with index 0 is the most uncertain prediction**, and the **tile with the highest index is the least uncertain prediction**.
2. **Validate prev** – this will move on to the previous most uncertain tile. If you are at index 0, this will circle back to the least uncertain tile.
3. **Validate next** – this will move on to the next most uncertain tile, in descending order of uncertainty.

One can use the prev and next buttons to scroll through the most uncertain tiles, from index 0 onwards. For each uncertain tile, one can manually add an annotation to the tile, which overwrites the prediction and can be used as a true label for the tile. This is a method of validating the output of model and is good practice as we assume the human labelling to always be more accurate than the predictions given by the model.

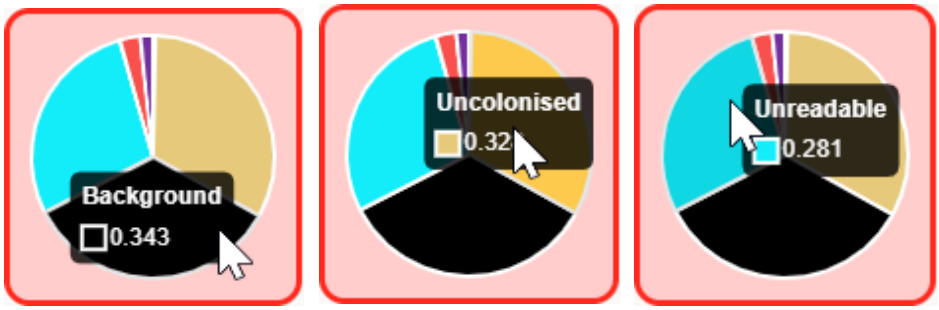
To manually annotate the tile, one can either use the icons at the top of the page, or the keyboard shortcuts associated to those icons.

Below, consider the tile of Index 0.

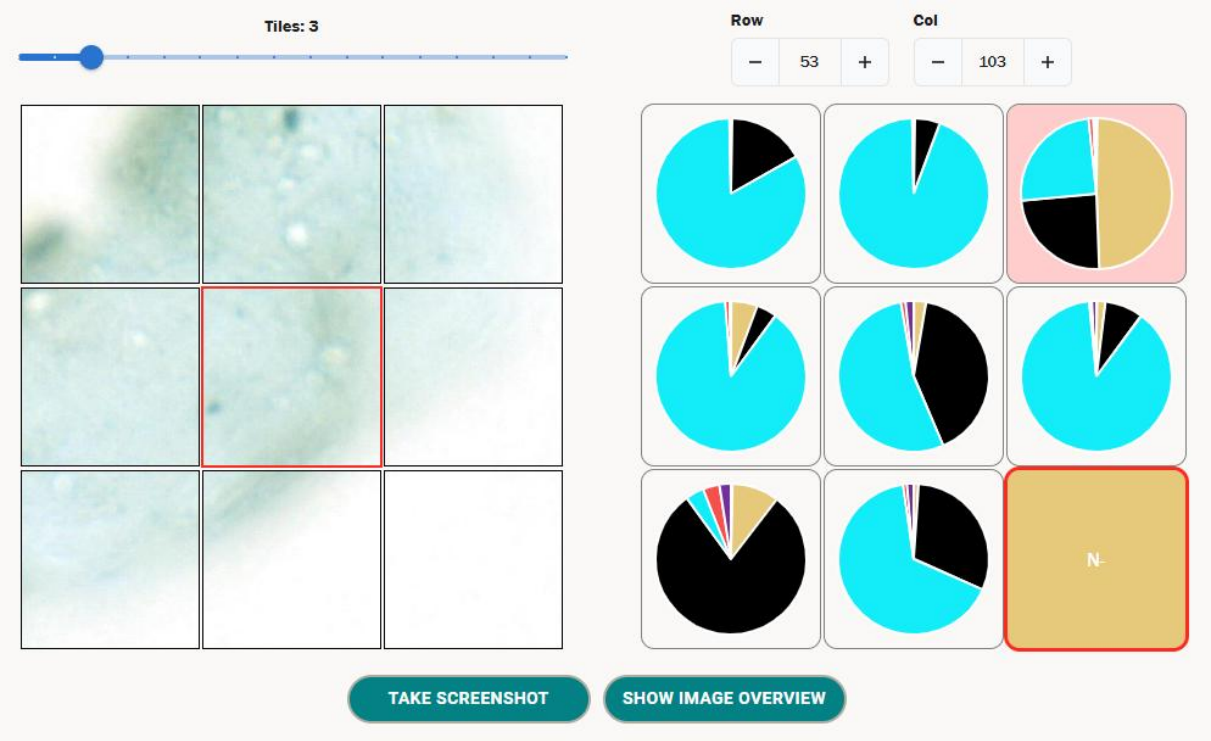


We can see that the prediction the model has made has various probabilities that are close together, and as such it is confused about what label to give the tile. We can also see that as this tile has a maximum prediction below the conversion threshold of 0.5 it has a red background, which denotes that the tile will **not be automatically converted** when we hit convert.

We can see the values of these probabilities by hovering over the pie chart as below.



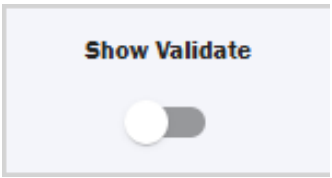
In this case, we can either allow the tile to be automatically converted by the tool with respect to the conversion threshold (detailed further here), or we can manually annotate this tile. If we choose to annotate this with the class Uncolonised, the tile will look like the below.



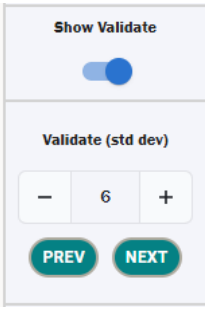
When converting this tile to annotations using the Convert button, this particular tile will now take the value Uncolonised, regardless of the prediction – i.e. the manual annotation takes precedence, and also ignores the conversion threshold.

05.2.02.02 VALIDATING TILES WITH STANDARD DEVIATION

We can also validate tiles using the standard deviation. In order to do this, we need to toggle on the “Show Validate” section on the right sidebar (this is false by default).



When this is toggled on, you will see the below section.



This looks very similar to the Check Confidence section, however the measure that is now used to click through tiles is standard deviation in ascending order and not

maximum confidence. This can be useful for understanding the uncertainty of a tile, as the lower the standard deviation, the closer the predictions are to one another, and generally the more uncertain the prediction is.

We would only recommend using this over confidence if there is a specific reason, as the confidence fits the conversion workflow better.

05.2.03 BEST PRACTICE FOR LABELLING LOW CONFIDENCE PREDICTIONS

The validation metrics above gives us a good idea of which tiles are uncertain and which aren't, but how can we handle these uncertain tiles? As previously mentioned, we would recommend manually labelling a certain proportion of the tiles with the highest uncertainty, as this will improve the accuracy of the output when you convert your predictions to annotations, especially with respect to the colonisation metrics.

In order to understand how many tiles one should manually relabel, we need to consider the CSV output for the prediction. This was saved in the same directory that you ran the prediction and will be titled {date}_{time}_results.csv.

In the last 10 columns of this CSV, you should see columns labelled "Tiles with confidence <= n", where n ranges from 0.1 to 1.0 inclusive in steps of 0.1 – this will look something like the below.

AP	AQ	AR	AS	AT	AU	AV
Tiles with confidence <= 0.4	Tiles with confidence <= 0.5	Tiles with confidence <= 0.6	Tiles with confidence <= 0.7	Tiles with confidence <= 0.8	Tiles with confidence <= 0.9	Tiles with confidence <= 1.0
9	46	142	246	376	572	8120

These rows denote the number of tiles that have the maximum prediction less than n. As such, to replace all tiles with the highest prediction of <0.5 with a manual label, you would have to manually label the 46 most uncertain tiles in the prediction. The more uncertain tiles you label, the stronger your overall prediction will be.

As part of the test functionality, there will be a folder called class_metrics_after_manual_labelling which contains multiple CSV files that contain metrics on the performance of the model after all the tiles below each threshold above have been given the true label – this mimics an expert manually labelling the tiles after a prediction. For your model, you should set a threshold based on the level of performance you are happy with. For example, if you want predictions with an F1-score of 80%, you should choose the lowest threshold that has this F1-score as output by test.

We can see the output of one such threshold csv below – this corresponds to the "threshold_0.8.csv" file, which means these are the model metrics if one were to convert every tile to the correct annotation with confidence less than or equal to 0.8.

class_name	Accuracy	Precision	Recall	F1 Score
AMColonised	0.862379	0.790218	0.862379	0.824723
Uncolonised	0.899	0.903336	0.899	0.901163
Background	0.989285	0.997186	0.989285	0.99322
Unreadable	0.787431	0.694943	0.787431	0.738302
DSE	0.802013	0.737654	0.802013	0.768489
Hybrid	0.697674	0.849057	0.697674	0.765957
Average	0.83963	0.828732	0.83963	0.831976

Then, for whichever threshold you choose (in our example it is 0.8), you should label the corresponding number of uncertain tiles from the prediction results file – in our example this means you should label all the tiles with confidence <= 0.8, which corresponds to labelling 376 uncertain tiles to reach an F1 score of 83.2%. In practice this means you should label all the uncertain tiles up until index 375 using the validation functionality.

Note that as the test set is different to the image you are calculating predictions on, this is only a rule of thumb. In reality, the more uncertain tiles you label manually, the more accurate your resulting annotations will be – its up to you what trade-off you want for prediction accuracy to human effort.

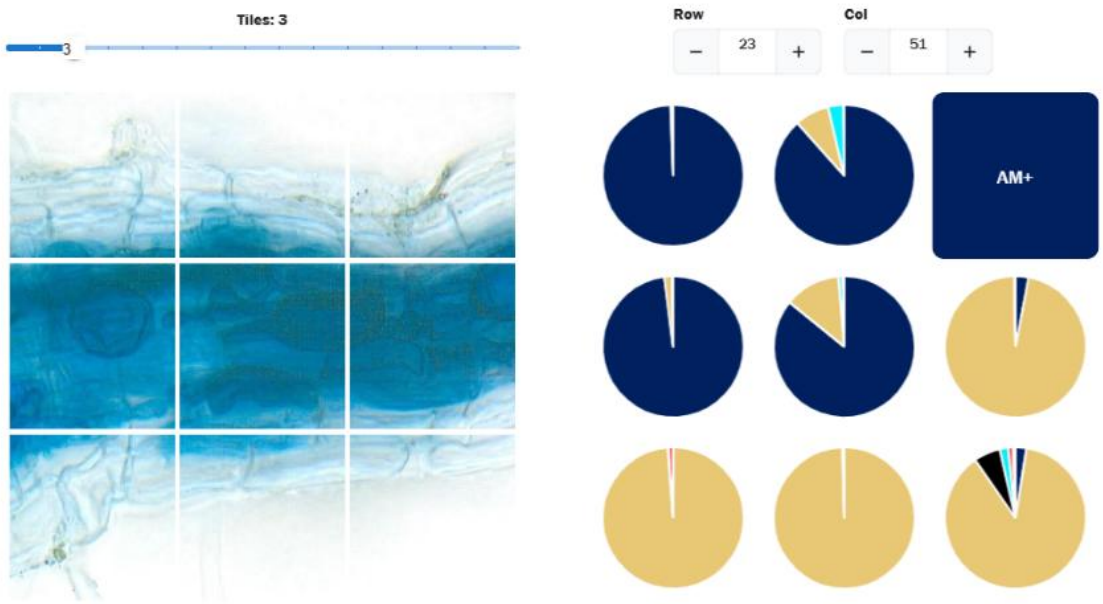
Finally, you should then set your conversion threshold to the same as the above threshold before converting predictions to annotations, which should ensure there are no tiles left as unlabelled once you convert – this is detailed further here.

When you are labelling your predictions, you should use the "Check Confidence" section to click through the tiles from index 0, manually relabelling each one until the background of the tiles is no longer red. This means you have manually labelled every tile under the conversion threshold, and when you convert the predictions to annotations there shouldn't be any blank values.

05.2.04 SCREENSHOTS

One can also take a screenshot of the predictions grid using the "Take screenshot" button at the bottom of the page. This will download a screenshot with the name mfbrowser-screenshot-{date_time}, where {date_time} is the current date and time.

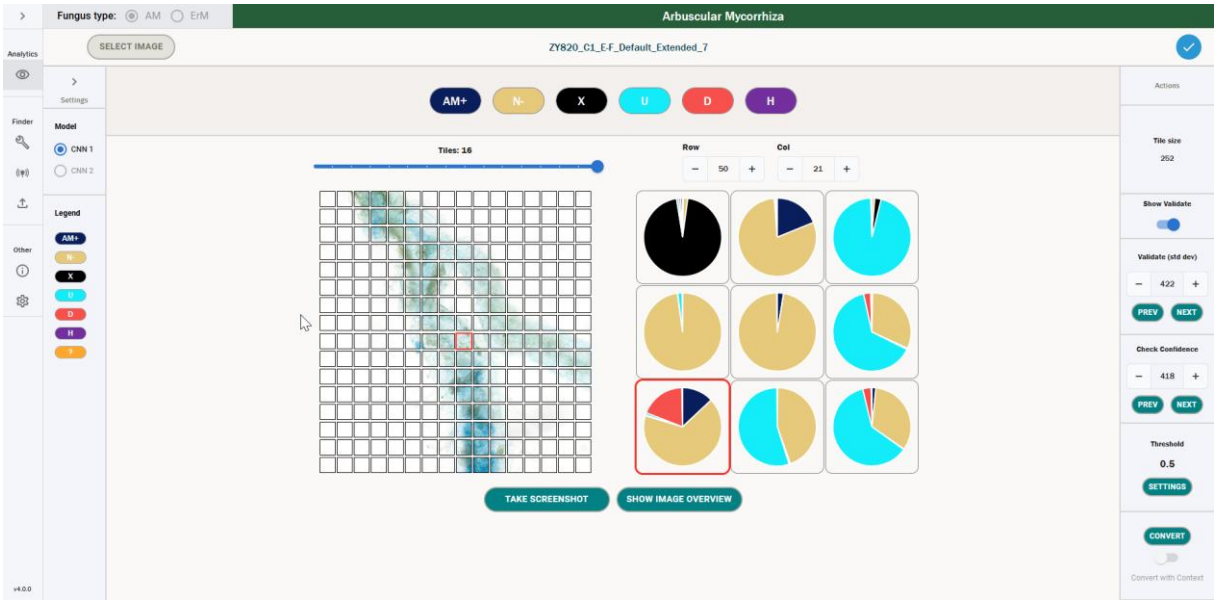
The screenshot will look like the below.



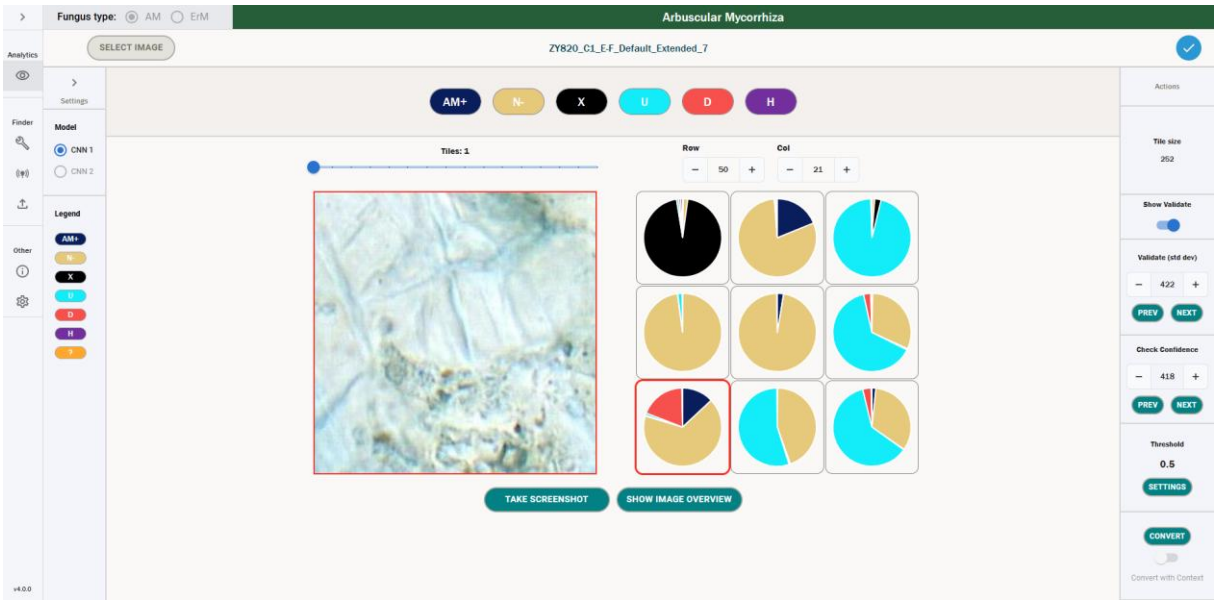
05.2.05 ZOOM FUNCTIONALITY

The default setting for the image grid on the left-hand side is a 3x3 grid. However, one can change this zoom level by dragging the bar above the image grid to the left and right, which will reduce and increase the number of tiles in the grid respectively.

The maximum number of tiles varies based on the size of your browser window – we include a screenshot below of the maximum number of tiles in a particular browser window size.

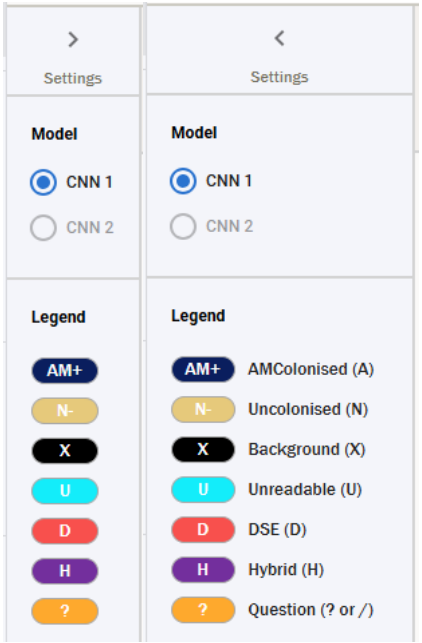


Moreover, below we also include a screenshot of the maximum zoom level, which is one single tile.



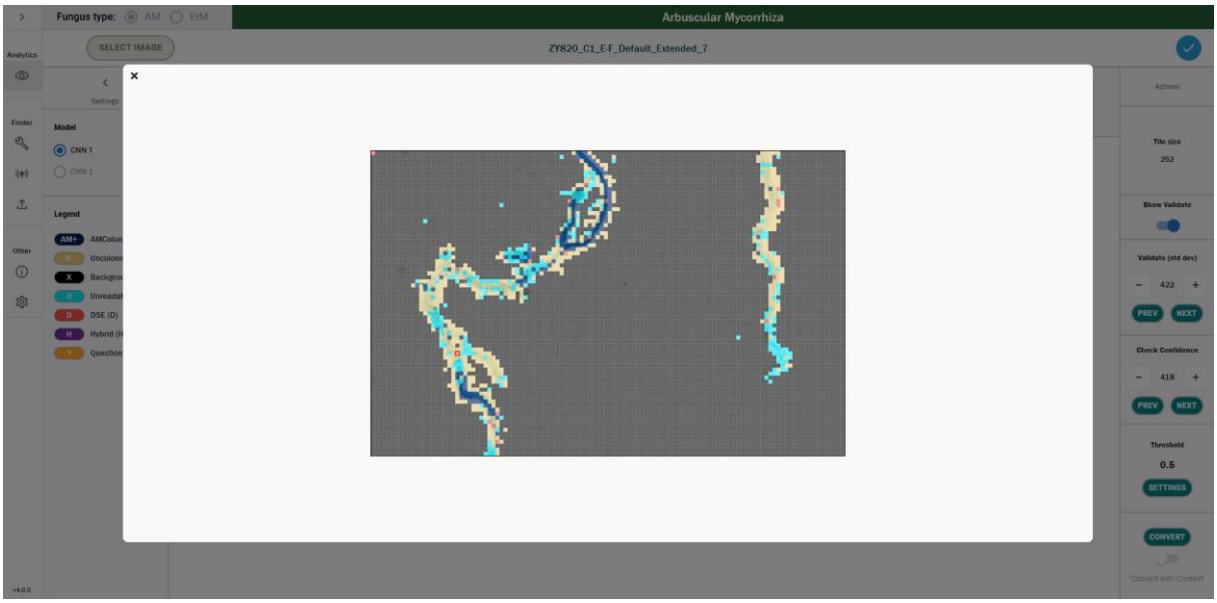
05.2.06 SHOWING A LEGEND

You can see the legend for the icons by opening the sidebar on the left-hand side titled “Settings”. This shows the name of the classes, along with the key binding shortcuts one can use to manually label that tile with a particular annotation.



05.2.07 IMAGE OVERVIEW

In addition to the zoom functionality, one can also click the button labelled “Image Overview” to see a high-level overview of the entire image. We can see an example of this below.



We can see in the above that the root is overlaid with coloured squares. These colours match the colours of the icons in the icon selector, and they denote either:

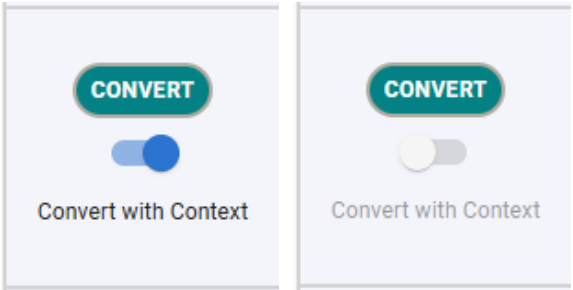
1. The label corresponding to the value of the maximum prediction for a particular tile.
2. The label corresponding to the contextual label for that tile.
3. The label corresponding to the manual annotation added to a tile if this exists.

So, for example, the beige tiles in the above correspond to Uncolonised tiles, either by prediction, contextual label or manual annotation.

Moreover, one can click anywhere in the image, and this will change the selected tile, so the image overview can also be used for easy navigation of the whole image. The currently selected tile is denoted by a small red border around a particular square.

05.2.08 CONTEXTUAL LABELS FOR A PREDICTION

If you have calculated your predictions including context, then certain predictions will come with a contextual label attached. If there are any predictions with a contextual label, then the toggle underneath “Convert” called “Convert with Context” will be enabled and will be set to on by default. Conversely, if no predictions are found with contextual labels, then this toggle will be disabled and will be set to off by default.



If this is toggled on, then if a prediction has a contextual label, you will be able to see this label for said prediction (details on how this is calculated can be found here). This looks like the below.

We can see in the bottom right corner of the tile, there is a label denoting N- (Uncolonised) for this tile. This is the Contextual Label for that tile. If you have "Convert with Context" toggled on, then instead of being converted to AM+ (which is the class with the highest confidence), this class will be converted to N-. If you have "Convert with Context" toggled off, then this will be converted to the maximum confidence class, i.e. AM+.

Note that you will only be able to see these labels where the contextual label differs from the maximum confidence class – we only show contextual labels for the tile where the conversion would be different if you converted with a Contextual Label.

If a tile has a contextual label but the maximum confidence class for that tile is underneath the conversion threshold, this tile will **not be automatically converted** when you hit convert. This is to avoid converting uncertain tiles – in this case, manual labelling is still necessary.

This will also change the image overview as outlined in Image Overview.

05.2.09 CONVERTING PREDICTIONS TO ANNOTATIONS

Once you have gone through and manually added annotations to the uncertain tiles, you can automatically convert predictions to annotations using the Convert button. In order to do this, you need to understand the **conversion threshold**.

The conversion threshold is specified in the "Threshold" box on the right-hand side. This value defaults to 0.5 and should take a value between 0 and 1. Predictions will only be converted to annotations if the **maximum prediction value is greater than or equal to this threshold.** For example, if you set this threshold to 0 then all predictions will be converted to annotations. If you set to it to above 1, then none of the predictions would be converted.

One should be careful when setting this threshold, as if you set it too low you risk converting low confidence predictions, and if you set it too high you increase the amount of manual labelling necessary to have a completely labelled root.

To set this threshold, you can click the “Settings” button under the threshold. This will take you to the settings page, where you can scroll down to the Convert settings and update the threshold value. When you navigate back to the predictions (which you can do by hitting the back button), you will see the threshold value has updated.

As outlined in the best practices for labelling uncertain tiles, you should endeavour to set this threshold for the confidence value that you have manually labelled uncertain tiles up to.

Once you are ready to convert predictions to annotations, you merely need to click the convert button, and all predictions will be converted to annotations via the following rules.

1. If the prediction has a **manual annotation**, then this tile will be converted to the manually added annotation label.
2. If a tile **has a contextual label, convert with context is turned on** and **the highest prediction value is above the conversion threshold**, then this tile will be given the label that matches the contextual label.
3. If a tile **does not have a manual annotation or contextual label (or convert with context is turned off)** and **the highest prediction value is above the conversion threshold**, then this tile will be given the label of the class with the highest prediction.
4. If a tile **does not have a manual annotation** and the **highest prediction value is below the conversion threshold**, this tile will not be given a label when converting.

Once you have clicked convert, you will be taken to the annotations screen – details on how to interact with this are provided in the next section.

05.3 VIEWING & SAVING ANNOTATIONS

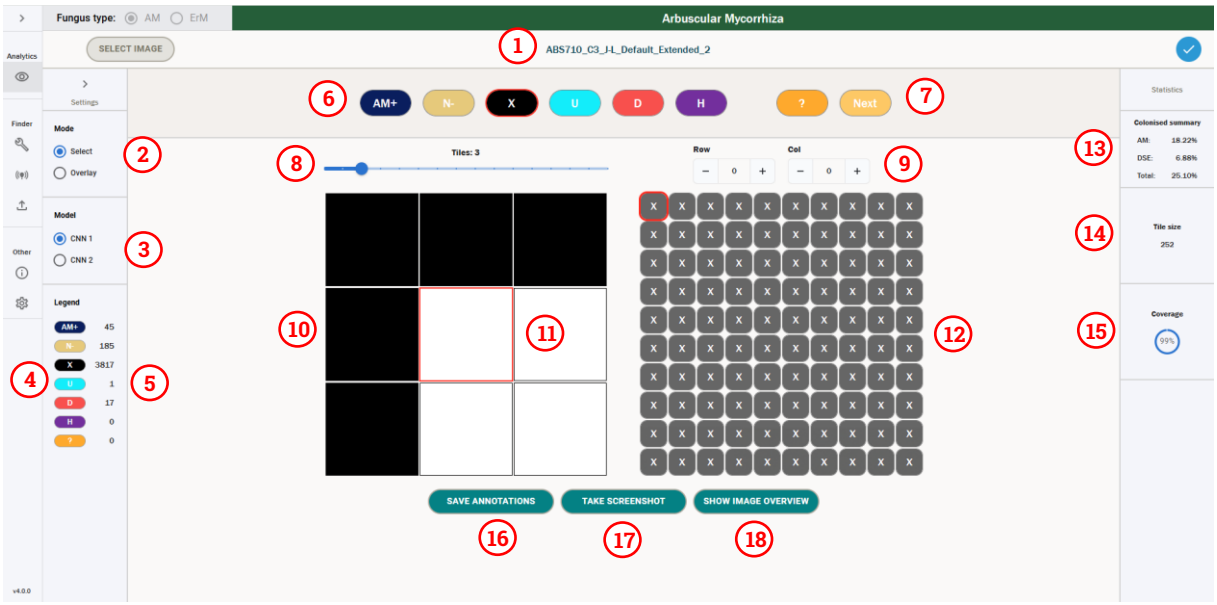
The annotations section of the Browser is the main user interface for viewing and editing the labels associated to a particular tile. A label denotes the contents of that tile – for example, if the tile has the label AM+, then we expect this tile to have a cell or cells that display AM Colonisation.

This screen can be used to assess colonisation on a granular level, by splitting up the root into small tiles and assessing the colonisation in each tile. We can then use these annotations to generate summary statistics about the root, including the percentage colonisation across the whole root.

In the below section, we outline in detail each functionality of the Annotations browser.

05.3.01 BROWSING ANNOTATIONS

When you open the annotations screen, the view you will be greeted with will look like the following.



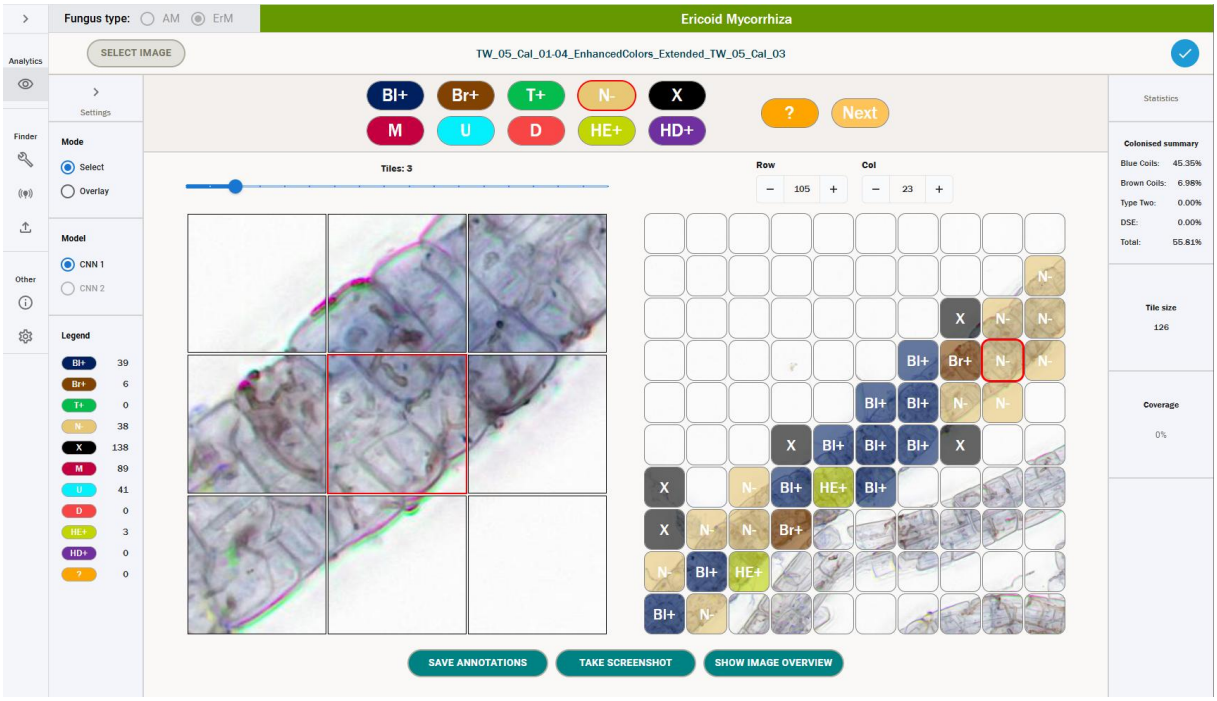
1. **Image name** – the title displays the name of the image you have selected.
2. **Mode** – this specifies the mode used when labelling annotations. The default functionality for the icon selector is “Select”, which allows users to assign classes to tiles – one can also select “Overlay”, which allows users to toggle only seeing one label at a time in the annotations grid. More detail on the overlay functionality can be found here.
3. **Network selector** – you can use this toggle to switch been viewing annotations for colonisation (CNN 1) and annotations for intraradical hyphal structures (CNN 2). CNN 1 is the main functionality of the tool and is currently the only network that supports predictions. CNN 2 is currently not supported for calculating predictions; however, one can use the CNN 2 setting to manually label structures in colonised root tiles. More information on CNN 2 can be found here – outside of this section, we exclusively focus on CNN 1 annotations.

4. **Legend** – this details the different classes for the fungus type you have selected. You can expand the sidebar with the arrow at the top to show the full names and key bindings for each class.
5. **Counts** – these are the current counts of each class, which denote the number of annotations that class has in the current set of labels. More information on counts can be found here.
6. **Icon selector** – the icon selector is how a user can edit annotations. To change the label of a tile they have selected, a user can either click the icon that corresponds to the label they want to give the tile, or they can use the corresponding keyboard shortcut, which can be viewed by expanding the left-hand sidebar. More details on how to change annotations can be found here.
7. **Question marks** – if a user is not sure about what label to give a tile, they can use this functionality to add a question mark to a tile. They can also add a comment to the question mark by double clicking on a question tile. They can transfer these questions between machines using the export and upload functionality, which allows them to easily discuss these cases with other researchers. This functionality is detailed further here.
8. **Zoom functionality** – the zoom functionality can be used to zoom the image grid in and out, increasing the number of visible tiles. The maximum grid size is dependent on the browser window size – the minimum grid size is a 1x1 grid. More detail on the zoom functionality can be found here.
9. **Row and column selector** – the row and column selectors can be used to quickly jump to a particular row and column in the image grid. This will move the selected tile (which is detailed in 12) to the selected row and column.
10. **Image grid –** the image grid is the main method for viewing the image that you are annotating. The central tile will always be the selected tile that you are annotating – this has a red border. You can use the image grid to assess the labels assigned to a particular tile. You can also click any image tile in the grid to move the selected tile to that particular tile.
11. **Selected tile –** the tile with a red border is the currently selected tile. This is reflected in the annotations grid by a smaller tile which also has a red border. When you are selecting a label for a tile, it is this tile that you are changing the label for.
12. **Annotations grid** – this grid displays all the annotations that are currently assigned to the various tiles. This is a fixed 10x10 grid, and the tile with a red border is the currently selected tile. Unlike the image grid, the selected tile is not fixed in the centre, and it can roam freely around the grid. You can click any tile in the annotations grid to select it. When you change annotations, you will see the label and colour of the selected tile change accordingly. In the above example, all the tiles are labelled “Background”, which is denoted by a black tile with an X.

13. **Summary statistics** – these show the percentage colonisation for the current set of annotations. The calculation for these is outlined here.
14. **Tile Size** – this denotes the size of the tiles in the image you have selected. The tiles are square, so 252 refers to the grid being split into tiles of 252x252px. 252 is the default tile size for AM, and 126 is the default tile size for ErM.
15. **Total labelled percentage** – this displays the current percentage of tiles that have been assigned a label – 0% denotes that no tiles have a label, whereas 100% denotes that all tiles have been assigned a label.
16. **Save annotations** – to save your annotations, you need to click this button – annotations are **not saved automatically.** When you click this button, the annotations will be saved down to the database. A more detailed description of how to save your annotations can be found here.
17. **Take screenshot** – one can take a screenshot of the image and annotations grid by clicking this button. This will save a screenshot to your default Downloads folder of the current state of the grids. An example of what this screenshot looks like can be found here.
18. **Show image overview** – by clicking this button, one can see an overview of the entire image, overlaid by the current annotations. These are denoted by coloured boxes, where the colours of the annotations match the colours of their label in the annotations grid. An example of the image overview can be seen here.

You can easily scroll around the grid by hovering over the annotations grid and using your mouse wheel to scroll the grid. This will mean you can't scroll the page – move your mouse back outside the grid to scroll the page again. If you hold shift while scrolling, you will move horizontally instead of vertically.

One can also see how the annotation grid looks for an ErM image below – the functionality is the same, but the labels change. One can see that the summary statistics in the top right have changed to report on ErM colonisation – these statistics are defined here. CNN2 annotations are disabled for ErM, as the tool would need further development work to include this.



05.3.02 CHANGING ANNOTATIONS

To change an annotation, a user has two options – one can either use the icons displayed near the top of the screen, or one can use the keyboard shortcuts associated with a particular label. These keyboard shortcuts can be shown by opening the left-hand setting sidebars. Note that these shortcuts are case-insensitive. These classes, for both AM and ErM, can be seen below and are outlined in detail in the accompanying documentation for best practice labelling guidelines [4].

Beyond these labels, there are two further keyboard shortcuts:

1. Delete annotation – keyboard shortcut(s): delete key.
2. Label all empty tiles as background – keyboard shortcut(s): Shift + *.

Note: *option 2 for labelling all empty tiles as background is irreversible, so only label all tiles as background with this functionality once you are sure there are no non-background tiles remaining in the image. If you did this by mistake, then make sure to close the image without saving your annotations.*

Note that if the selected tile already has an annotation, a user can either use the delete key or merely click the same annotation again in the icon selector to delete the annotation.

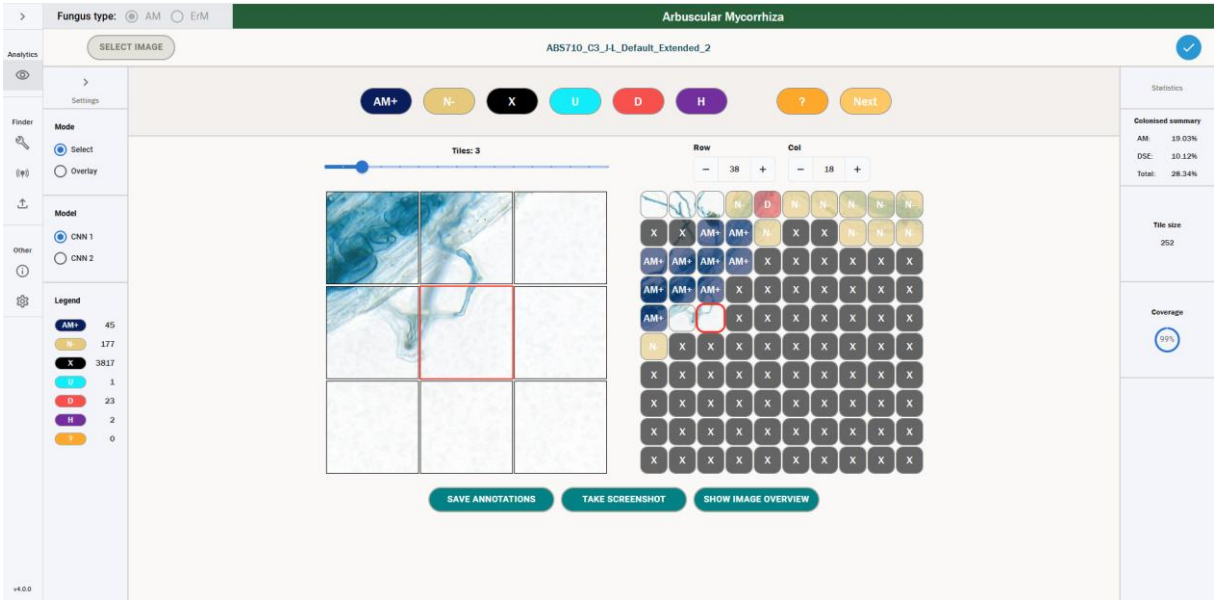
If a user wants to use the Icon selector to change the annotations, they must have the Icon selector in “Select” mode – if it is in “Overlay” mode, the icons will not change. However, if one is in Overlay mode, one can still change the annotations using the keyboard shortcuts.

If the selected tile has an annotation, then the icon that corresponds to that annotation in the icon selector will have a red border. This denotes that this is the selected annotation for the currently selected tile.

To change multiple annotations at once, a user can hold down one of the keyboard shortcuts above, and then by clicking and holding, they can drag their mouse over tiles to annotate them quickly.

05.3.03 CONVERTED ANNOTATIONS

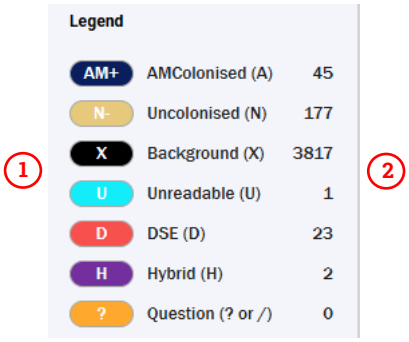
As outlined here, a user can automatically convert all CNN1 predictions to annotations that have a maximum prediction above a certain threshold. Naturally, this means that for converted predictions, the tiles that have a prediction below the threshold will be left empty. As such, the usual picture that we would see if one automatically converted predictions to annotations with no manual labelling would look like the below.



One can see that there are several empty tiles (including the selected tile). If a user wants a completely labelled root, they will have to go through these empty tiles and manually label them before saving their annotations, as discussed in the predictions section.

05.3.04 LEGEND AND COUNTS

One can view the legend and counts for each class by opening the left-hand sidebar. We can see this below.



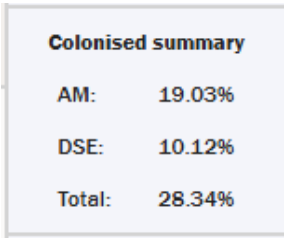
1. **Legend** – the legend is displayed to the right of each icon. This gives the full name for the class that is associated with each icon. It also shows the key binding in brackets next to the class name.
2. **Counts** – these are displayed to the right of the legend. They display the current number of tiles that have each respective label. In the above image, the counts denote that there are 45 AM Colonised tiles, 177 Uncolonised tiles, etc.

05.3.05 SUMMARY STATISTICS

The summary statistics for a particular set of annotations are shown in the right-hand sidebar. They differ for AM and ErM annotations – the differences are summarized below.

05.3.05.01 AM COLONISATION METRICS

The summary statistics for AM roots are shown below.



They denote three figures – AM colonisation percentage, DSE colonisation percentage and total colonisation percentage. These figures are calculated as follows. First, let the total root be the following.

$$Total\ Root = AMColonised + Uncolonised + DSE + Hybrid$$

So, the total root is defined as the sum of all the tiles, ignoring Unreadable and Background. The summary statistics are then defined as follows.

AMColonised percentage is defined as the summation of the AMColonised tiles plus Hybrid tiles, over the total root.

$$AMColonised\ Percentage = \frac{AMColonised + Hybrid}{Total\ Root} \times 100$$

DSE colonised percentage is defined as the summation of the DSE tiles plus Hybrid tiles, over the total root.

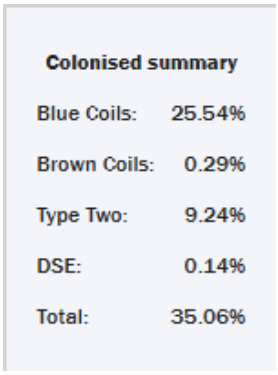
$$DSE\ Colonised\ Percentage = \frac{DSE + Hybrid}{Total\ Root} \times 100$$

The total colonisation percentage is defined as the summation of AMColonised tiles, plus DSE tiles, plus Hybrid tiles, over the total root.

$$Total\ Colonised\ Percentage = \frac{AMColonised + DSE + Hybrid}{Total\ Root} \times 100$$

05.3.05.02 ERM COLONISATION METRICS

For ErM roots, the summary statics include the below.



They denote five figures – Blue Coils colonisation percentage, Brown Coils colonisation percentage, Type Two colonisation percentage, DSE colonisation percentage, and total colonisation percentage. These figures are calculated as follows. First, let the total root be the following.

$$Total\ Root = BlueCoils + BrownCoils + TypeTwo + Uncolonised + DSE + HybridErM + HybridDSE$$

So, the total root is defined as the sum of all the tiles, ignoring Unreadable, Main Root and Background. The summary statistics are then defined as follows.

Blue Coils colonised percentage is defined as the summation of the Blue Coils tiles over the total root.

$$Blue\ Coils\ Colonised\ Percentage = \frac{Blue\ Coils}{Total\ Root} \times 100$$

Brown Coil colonised percentage is defined as the summation of the Brown Coils tiles over the total root.

$$Brown\ Coils\ Colonised\ Percentage = \frac{Brown\ Coils}{Total\ Root} \times 100$$

Type Two colonised percentage is defined as the summation of the Type Two tiles over the total root.

$$Type\ Two\ Colonised\ Percentage = \frac{Type\ Two}{Total\ Root} \times 100$$

DSE colonised percentage is defined as the summation of the DSE and Hybrid DSE tiles over the total root.

$$DSE\ Colonised\ Percentage = \frac{DSE + HybridDSE}{Total\ Root} \times 100$$

The total colonisation percentage is defined as the summation of Blue Coil, Brown Coil, Type Two and Hybrid ErM tiles over the total root.

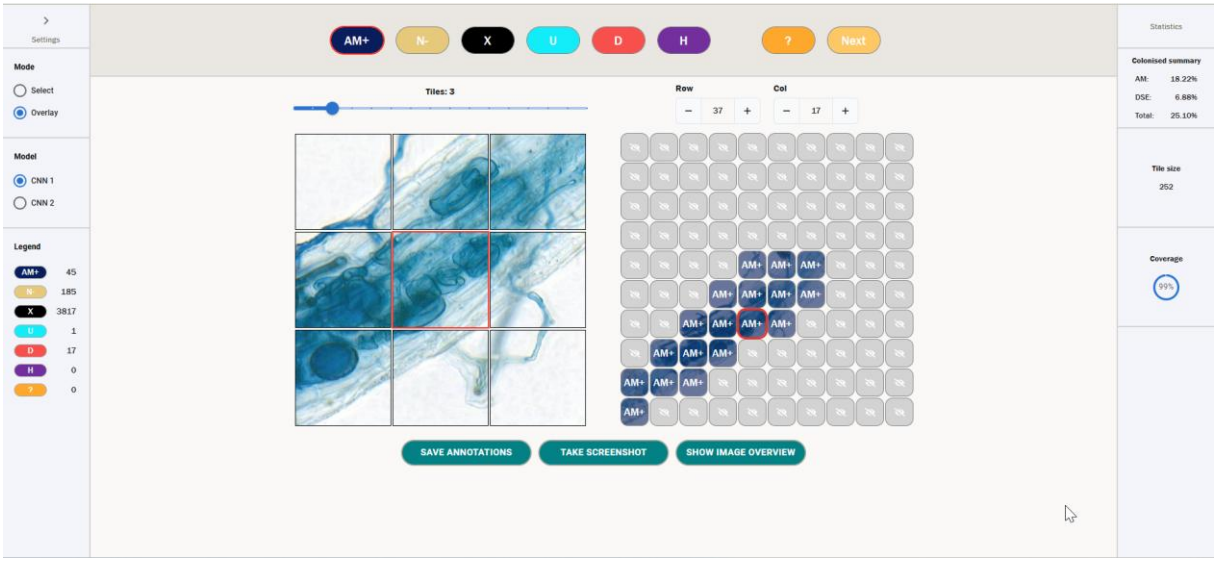
$$Total\ Colonised\ Percentage = \frac{Blue\ Coils + Brown\ Coils + TypeTwo + HybridErM}{Total\ Root} \times 100$$

These colonisation percentages can also be given in a CSV output via colonisation – the process for this is defined here.

05.3.06 OVERLAY

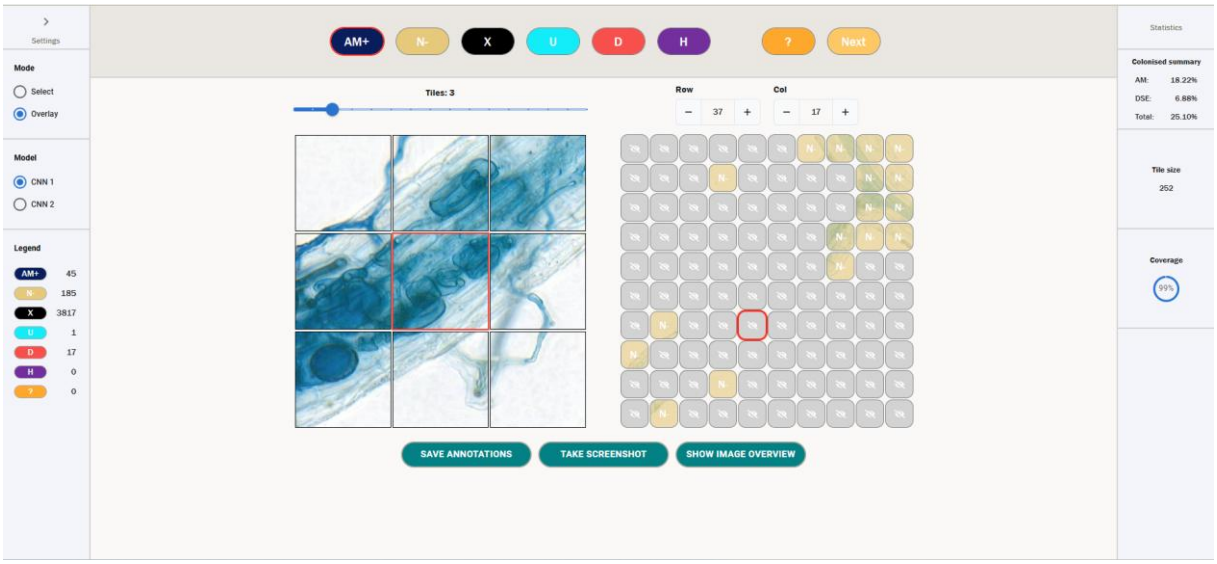
The default behaviour for the icon selector described here is to change annotations. However, if a user changes the toggle on the right-hand side to “Overlay”, the behaviour of this selector changes.

When “Overlay” is selected, if a user clicks an icon, then instead of the annotation for the selected tile changing, all the tiles that do not have that icon will change to having a grey icon, and only the selected icons will be viewable. One can see an example of this below – in the below image, Overlay is turned on and the AM+ icon has been selected.



To go back to the normal view, the user can either click the AM+ icon again or switch back to Select mode.

Below, we see the same screen but with an N- overlay.

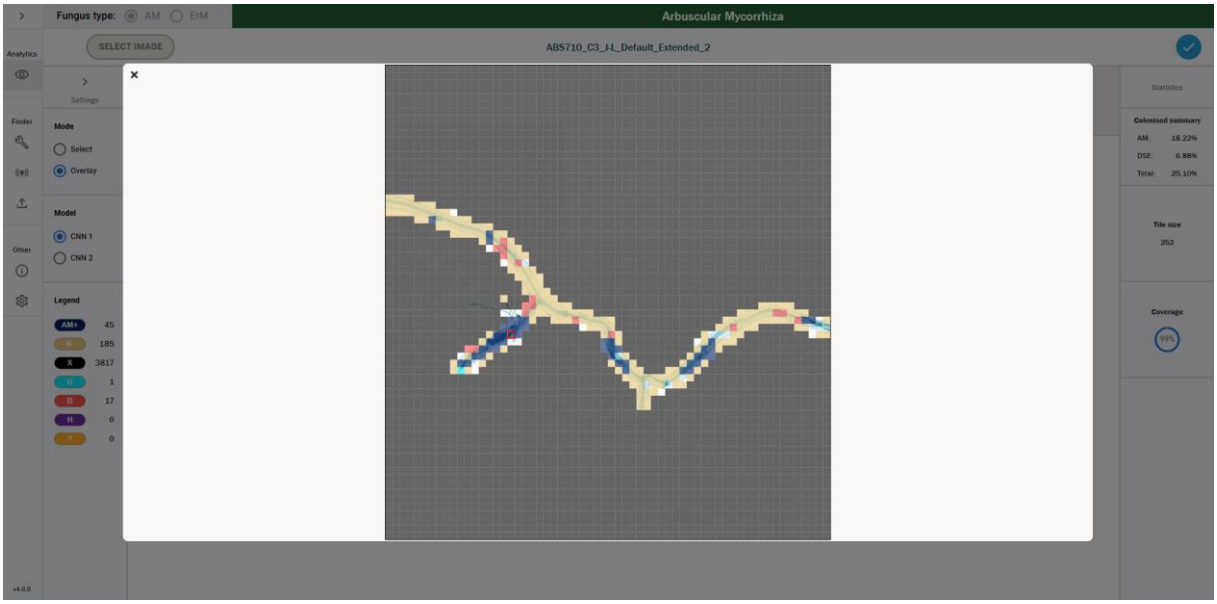


Note that if one uses the keyboard shortcuts outlined here while in Overlay mode, this will still change the value of the icons. Thus, the keyboard shortcuts can be used to change the annotation value of a tile, regardless of what mode you are in.

05.3.07 IMAGE OVERVIEW

Often when labelling, it is useful to be able to check what section of the root you are currently labelling – as the image and annotation grid have a fixed small context window, it can be difficult to understand what part of the root you are labelling purely by the normal image and annotation grid.

To be able to see an overview of the entire root, one can open the image overview. If you click the button at the bottom of the screen denoted “Show image overview”, this will open a modal that looks like the below.



This is a picture of the entire root image that the user is currently viewing annotations for. The coloured boxes that overlay the root denote the current annotation for that tile – the colours will match the colour of the icon for that particular annotation (so for example, in the image above we see a lot of beige tiles – these correspond to N-, or Uncolonised, tiles). Note that if a particular section of the root does not have a coloured overlay, it means those tile(s) are unlabelled.

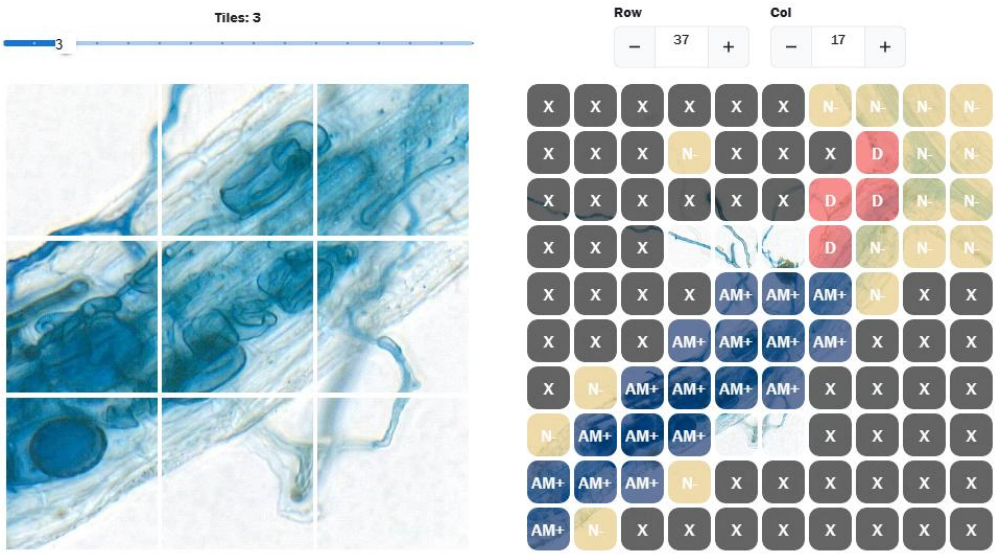
One can click on any section of the root to move the selected tile to that section. When you then close the modal, the image and annotation grid will have moved accordingly.

In order to close the modal, you can either click outside of the modal, or click the x button in the top left corner.

05.3.08 SCREENSHOTS

You can take a screenshot of the current state of the image and annotations grid by clicking the “Take screenshot” button at the bottom of the screen. This will download a screenshot with the name mfbrowser-screenshot-{date_time}, where {date_time} is the current date and time.

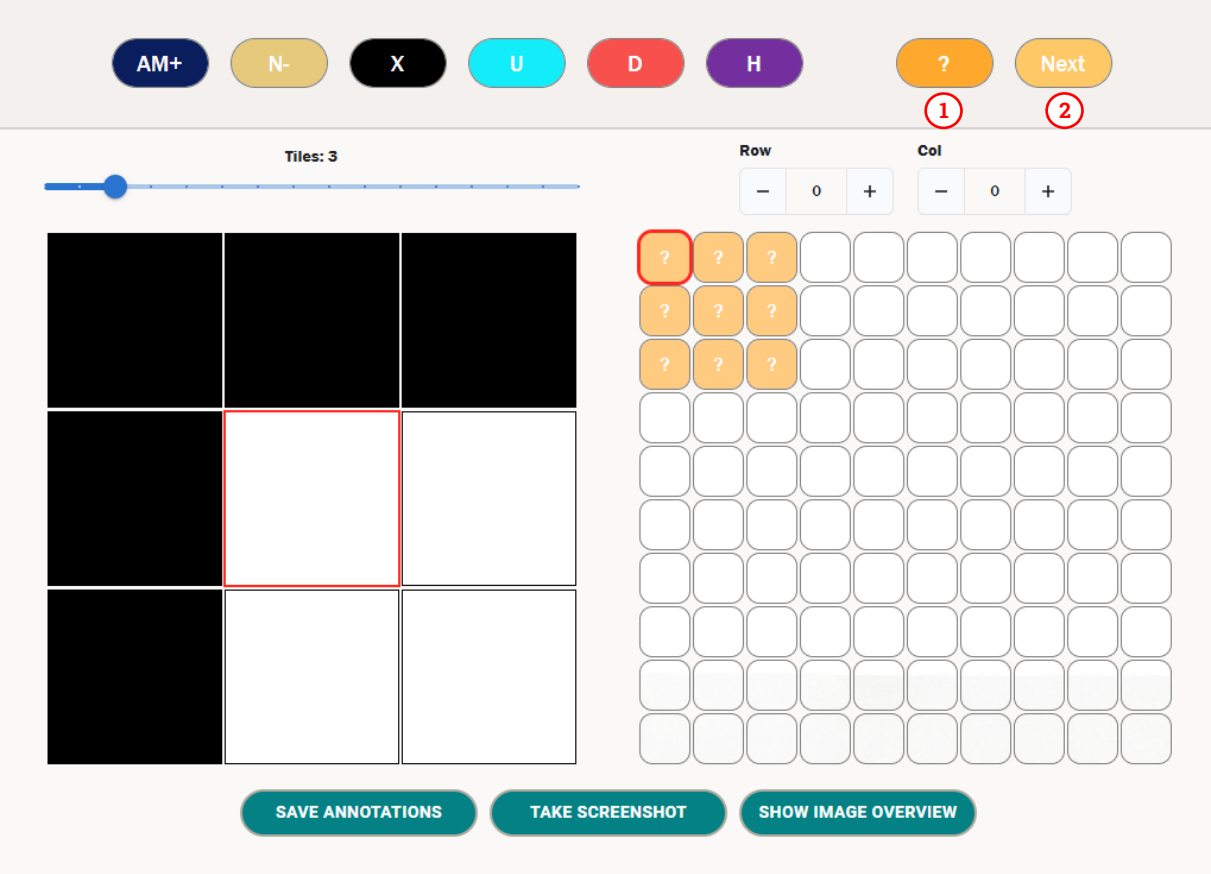
The screenshot will look like the below.



05.3.09 ADDING QUESTION MARKS

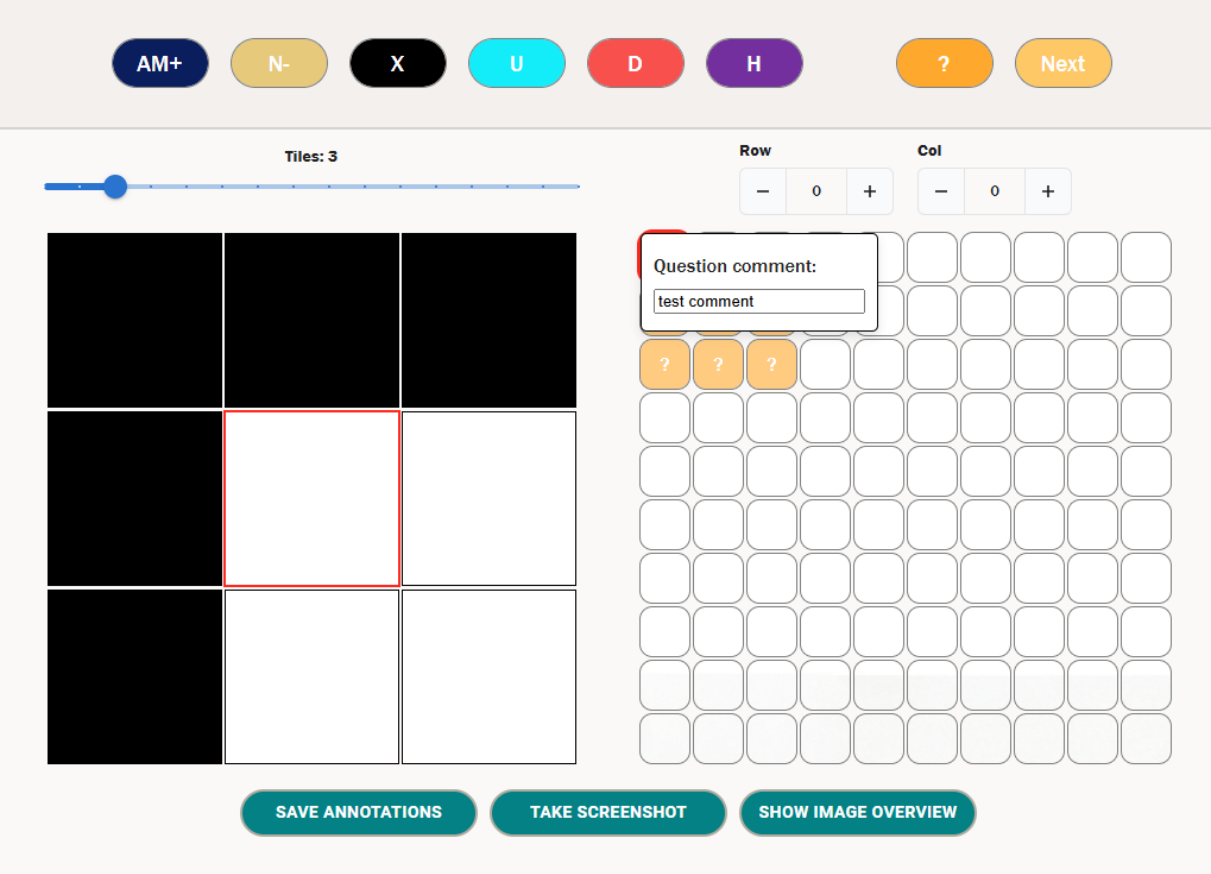
If there is a tile that requires further thought or potentially warrants a wider discussion before being given a label, one can label this tile with a question mark. This makes it easy to return to later, and gives a user the option to add a comment to the tile, allowing them to note down any potential comments for discussion relating to that tile.

In the image below, we can see several tiles marked with a question mark.



1. **Add question mark** – clicking this icon will add a question mark to a tile. You can also use the keyboard shortcuts of ? or / to add a question mark to a tile. If a tile has a question mark, then repeat this to delete the question mark, or use the delete key keyboard shortcut.
2. **Next question mark** – clicking this button will jump you to the next tile labelled with a question mark in the grid. This gives users an easy way to look at all potentially confusing tiles quickly. Note that the question marks are navigated in order of addition.

If a tile has a question mark, a user can also double click on the tile to add a comment. When a user double clicks on a question tile, it will open the following dialogue.



You can enter any comment in the box – above, we enter the value “test comment”. When we save the annotations, this comment will be saved down and associated to the tile with a question mark at Row 0 and Column 0. You can click outside this box to close it.

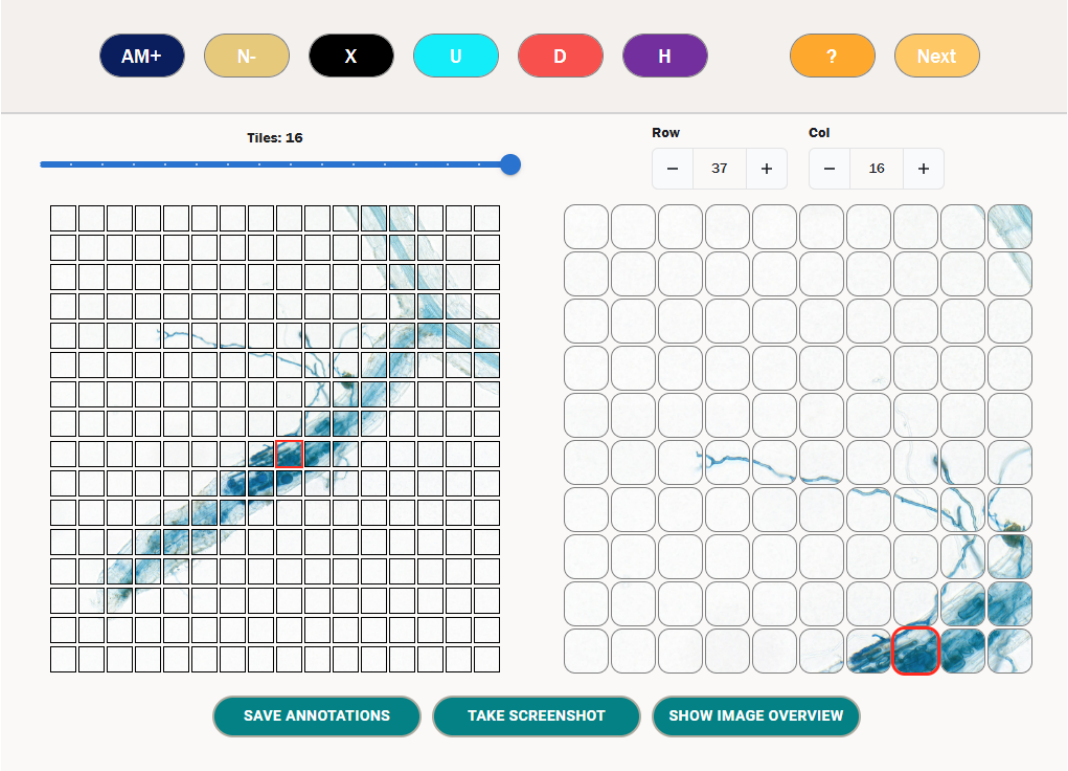
Note that if you delete the question mark from a tile, if it has an associated comment then this will also be deleted.

If you save a set of annotations that has question marks, a modal will appear telling you that you have question marks and confirming that you want to continue with saving. This functionality is shown in more detail here.

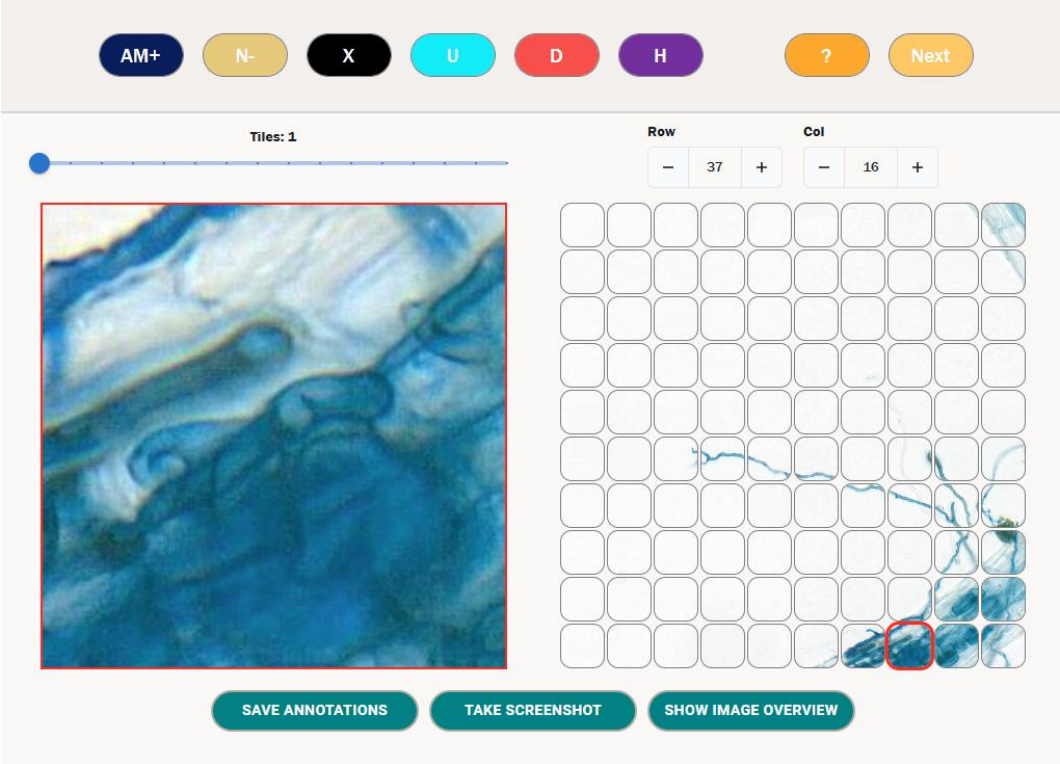
05.3.10 ZOOM FUNCTIONALITY

A user can zoom in and out of the image grid to give more context for the surrounding root. By dragging the zoom bar to the left, the image grid will zoom in, reducing the number of tiles shown. By dragging the zoom bar to the right, the image grid will zoom out, increasing the number of tiles shown. The maximum grid size varies based on the browser window size between 16x16 and 18x18. The minimum grid size is a 1x1 grid, i.e. a single tile.

We see an example of a fully zoomed out grid below.



We see an example of a fully zoomed in image grid below.



05.3.11 SAVING ANNOTATIONS

You can save annotations down to the database at any point during your session. Note that annotations **do not autosave** – the only way to save your annotations is by

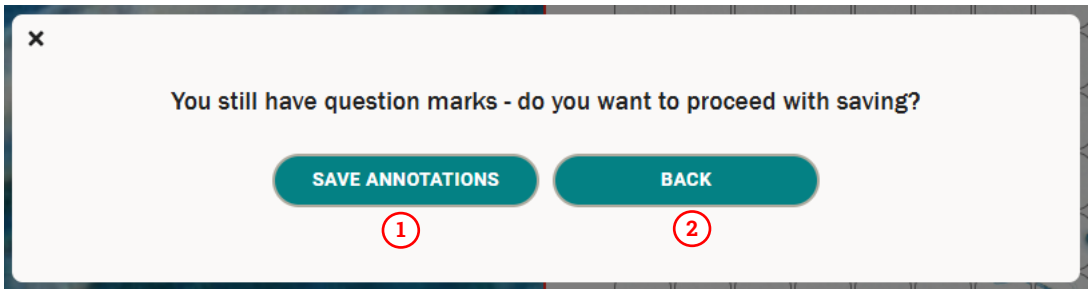
clicking the “Save Annotations” button below the image grid. **In order to save your progress, it is recommended you do this on a regular basis.** You must make sure you click the Save Annotations button before navigating away from your image or closing the tool if you want to save your progress – if you don’t, it will be lost.

Note that you can switch between annotations and predictions without losing your progress – you can use the back arrow to do this.

In order to save your annotations, you need to click the Save Annotations button. If this succeeds, you will get a green notification saying they have been saved successfully. If not then you will get a red notification telling you there was an error, as shown below. If the annotations are not saved successfully, you will need to debug the error via the console – the process for this is described here.

Then when you next select the same image as described in section 05.01, the relevant set of annotations will include your changes.

If there are question marks as part of the annotations, then when you click the save button, the annotations will not immediately save – rather you will see the following modal.



1. **Save annotations** – clicking this button will save the annotations as normal, including the question marks, to the database.
2. **Back** – clicking this button will close the modal and your annotations will not be saved.

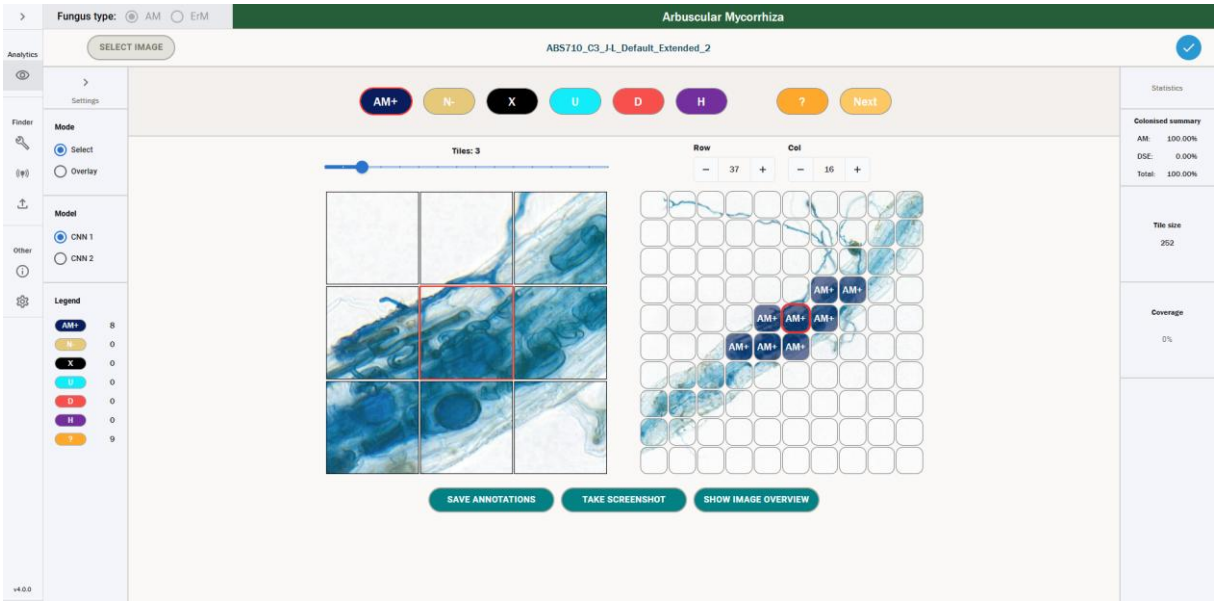
05.3.12 CNN2 FUNCTIONALITY

The tool is currently split into two main functionalities – CNN1 and CNN2. CNN1 is a network intended to assess the colonisation of a particular root and is the currently supported functionality of the tool from a predictive modelling perspective.

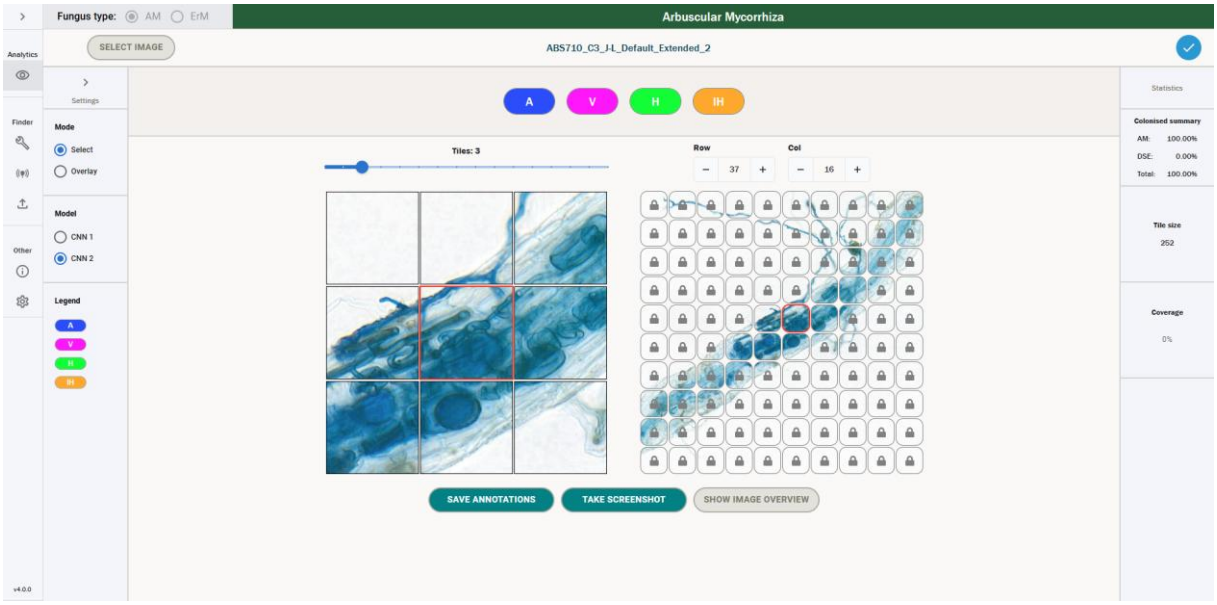
CNN2 is intended to assess the structures of colonisation within colonised tiles. There is currently not a productive model for this – more development work needs to be

carried out before predictions can be calculated using CNN2. However, we have included the ability to annotate colonised tiles with CNN2 categories in the current version of the tool to aid future development.

Consider the picture below, where one has several tiles that come under the category of AMColonised.



In this case, the future intention for CNN2 would be to assess the colonisation structures of these tiles. Therefore, when one clicks on CNN2, the user would see the following screen.

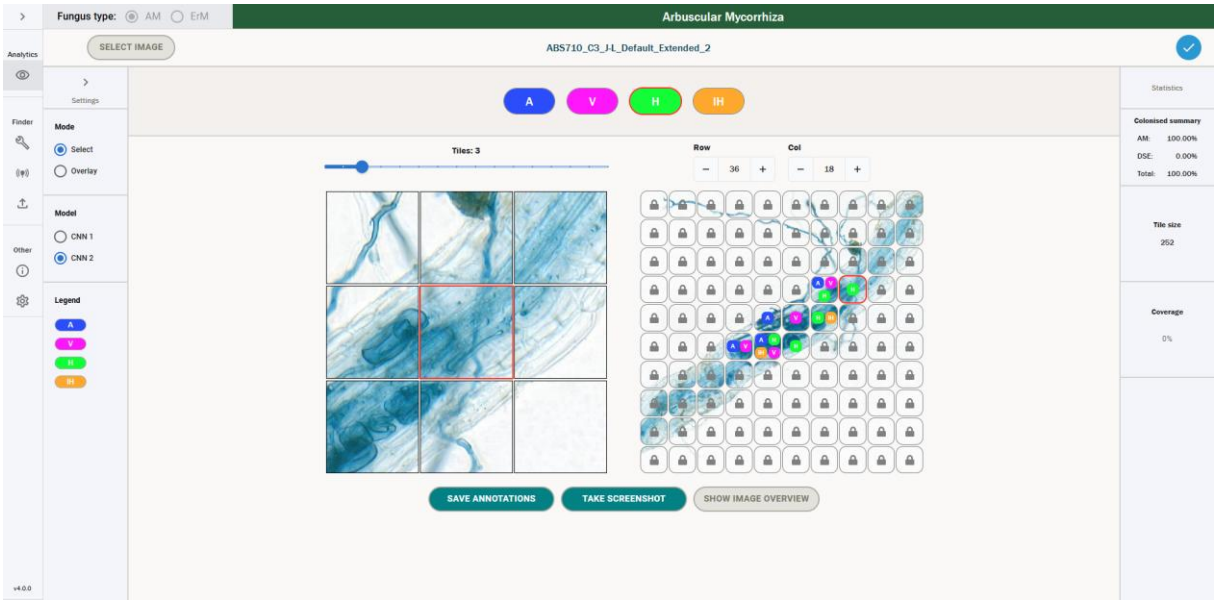


We can see that every tile besides the ones that had an AM+ label for CNN1 are “locked”. This is because a user can only add CNN2 annotations for colonised tiles.

In order to add CNN2 annotations for colonised tiles, one has four options, which all correspond to varying types of colonisation within a colonised tile.

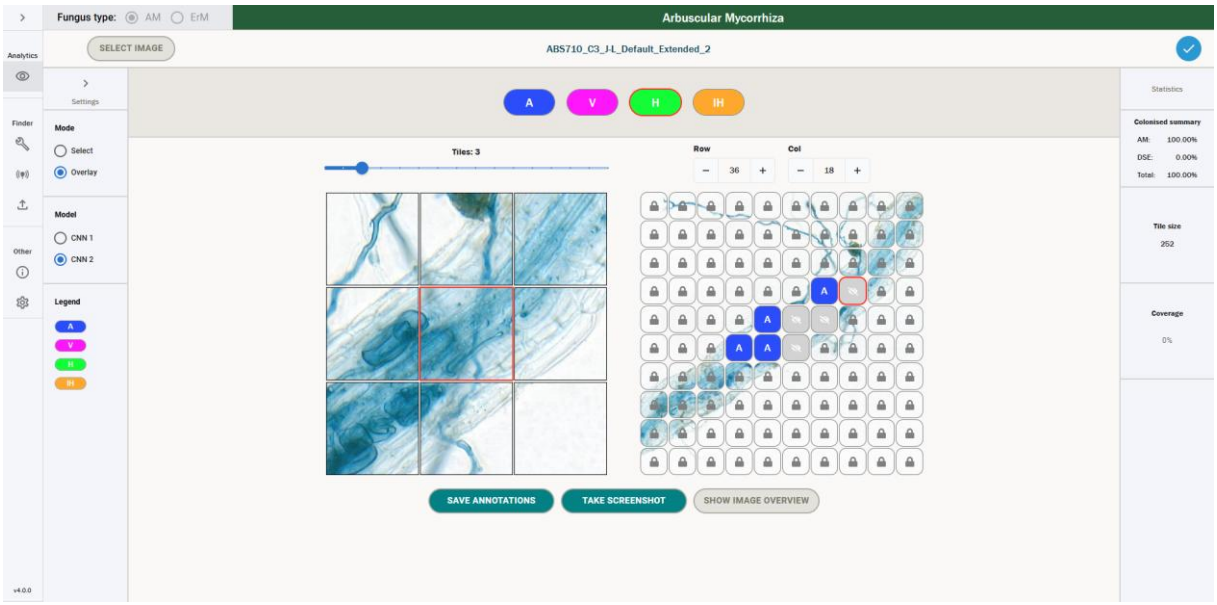
1. **Arbuscule (A)**
2. **Vesicle (V)**
3. **Hyphodium (H)**
4. **Intraradical Hyphae (IH)**

It is worth noting that multiple types of colonisation can appear in a single tile, and thus a user can add multiple labels per tile. This is shown below.



These annotations will be saved down along with CNN1 annotations when you click “Save annotations”.

It is worth noting that there are no question marks for CNN2 annotations, and the image overview will only currently show CNN1 annotations. All other functionalities should work the same – for example, the overlay functionality has a similar output to CNN1 where single labels are shown. We can see an example of this below.



To reiterate, this functionality can currently only be used for labelling annotations – there is no way to calculate predictions for CNN2. However, this functionality could be integrated in a future version of the tool.

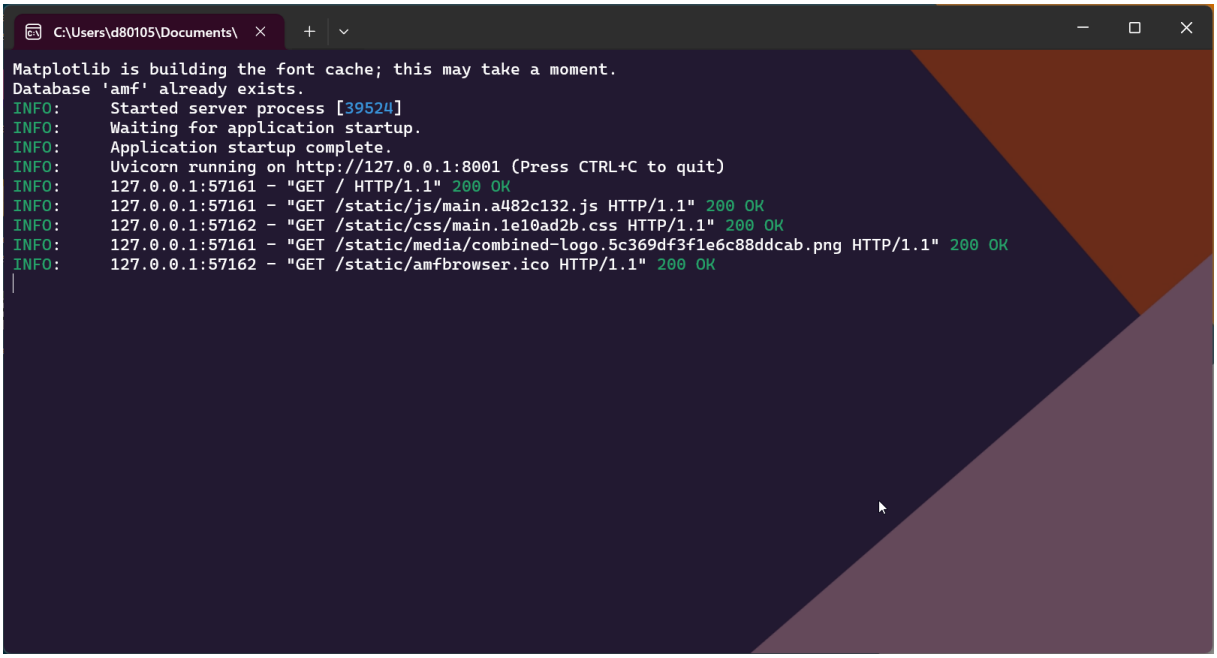
06. DEBUGGING ERRORS

To debug errors, the immediate first port of call is looking at the output the UI gives you. However, this can be limited – for example, you may know that a training run failed, but not have specific information on how or why it failed.

In order to dive into the possible causes for errors further, one can use the command line to understand what has gone wrong in the tool.

06.1 COMMAND LINE OUTPUT

To access the command line, open the window that pops up when you started running the executable – you should see something like the below.



This is where you can view the console output for the tool. Usually, you will never need to access this, as all the main output of the tool will be given to you via the user interface. However, if there is an issue that is not immediately obvious from the UI output, the best thing to do is to come to the console and read the error message to see if this gives you any insight.

06.2 COMMON ERRORS

Below is a list of potential common errors that you might come across when using the day-to-day functionality of the tool. This list is not exhaustive but may help you resolve some common errors.

Note that if the app fails on startup, if you have opened the tool by double clicking on the executable then the command line window will automatically close before you can read the error. In this case, you will need to start the tool via a cmd prompt – navigate to the directory the exe is in and run "`./amf.exe`". You will then be able to see the error displayed in the cmd prompt.

06.2.01 GENERAL ERRORS

Error: No images found.

```
  File "C:\Users\d80105\code\root-fungal-colonisation\amf\amf_code\amfinder_config.py", line 332, in find_files_in_directory
    assert len(img_files) > 0, f"no images found in {directory}"
AssertionError: no images found in C:\Users\d80105\Documents\Kew\test-output
```

Solution: you are pointing at a directory that does not contain images. Please point at a directory that contains the images you want to use for a particular functionality.

Error: password authentication failed for user "postgres".

```
[11:41:43] ERROR: Cannot connect to database, consider restarting your postgres service: connection to server at "localhost" (::1), port 5432 fa
iled: FATAL:  password authentication failed for user "postgres"
```

Solution: in this case, you are not entering the correct password for the postgres user. As explained in Tool Installation, the executable assumes the default postgres user has the password postgres. If you have changed this password, you will need to start the tool using the below command, replacing "example-password" with the password for the user postgres. Note that there is no option to change the user – you must use the postgres user.

```
./amf.exe --db-password example-password
```

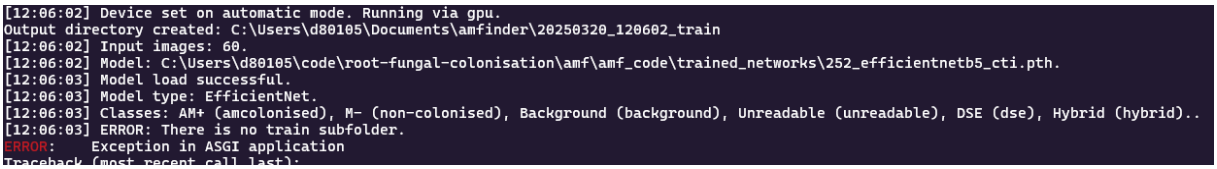
Error: Cannot connect to database

```
[11:50:42] ERROR: Cannot connect to database, consider restarting your postgres service: connection to server at "localhost" (::1), port 5432 fa
iled: Connection refused (0x0000274D/10061)
        Is the server running on that host and accepting TCP/IP connections?
connection to server at "localhost" (127.0.0.1), port 5432 failed: Connection refused (0x0000274D/10061)
        Is the server running on that host and accepting TCP/IP connections?
```

Solution: in this case, your postgres service is not started. If you are on Windows, navigate to "Services" via the start menu, and then look for the service named "postgresql-x64-17 - PostgreSQL Server 17" (note that the version could be different if you have not downloaded version 17). Right click on this and click Start (you will need administrator privileges to do this). Note that this should start automatically – check the Startup Type, and if it is not already set to Automatic, then change to Automatic.

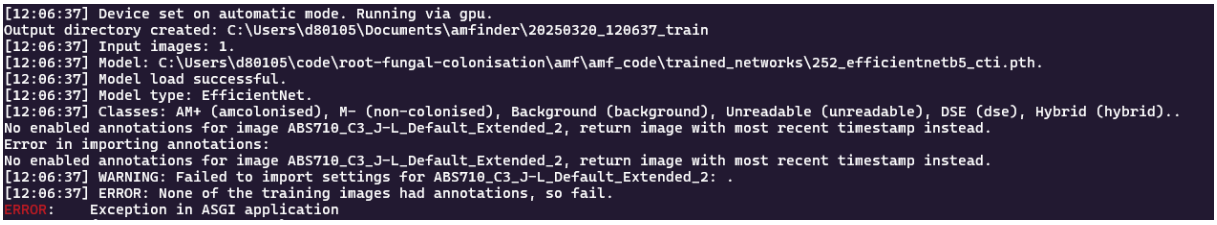
06.2.02 TRAINING ERRORS

Error: There is no train subfolder.



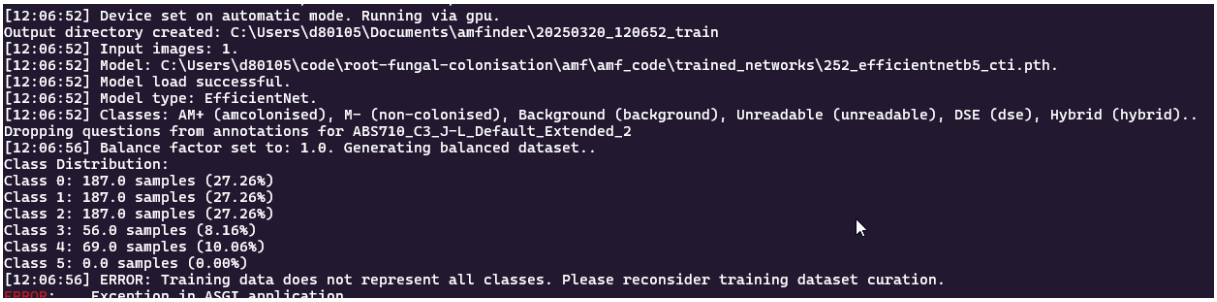
Solution: you are attempting to run training without the correct folder structure. Make sure the folder you have passed has a folder “train” as a subdirectory.

Error: None of the training images had annotations, so fail.



Solution: you have attempted to carry out training on images that do not have corresponding annotations. Make sure that your training dataset has at least one image that has corresponding annotations.

Error: Training data does not represent all classes. Please reconsider training dataset curation.



Solution: There was a class in the training dataset that did not have any samples included. You need to make sure every class has a minimum representation in the dataset – consider the class distribution values in the output and add more examples of the offending class (in the above case, Hybrid) to satisfy the minimum necessary samples for training.

06.2.03 TESTING ERRORS

Error: Input images do not contain tile annotations. Use amfbrowser to annotate tiles before training.

```
[12:58:51] Device set on automatic mode. Running via gpu.
Output directory created: C:\Users\d80105\Documents\amfinder\20250320_125851_test
[12:58:51] Input images: 3.
[12:58:51] Model: C:\Users\d80105\code\root-fungal-colonisation\amf\amf_code\trained_networks\252_efficientnetb5_cti.pth.
[12:58:51] Model load successful.
[12:58:51] Model type: EfficientNet.
[12:58:51] Classes: AM+ (amcolonised), M- (non-colonised), Background (background), Unreadable (unreadable), DSE (dse), Hybrid (hybrid)..
Results Directory: C:\Users\d80105\Documents\amfinder\20250320_125851_test
[12:58:51] Tile extraction.
No enabled annotations for image ABS710_C3_J-L_Default_Extended_2, return image with most recent timestamp instead.
Error in importing annotations:
No enabled annotations for image ABS710_C3_J-L_Default_Extended_2, return image with most recent timestamp instead.
[12:58:51] WARNING: Failed to import settings for ABS710_C3_J-L_Default_Extended_2: .
[12:58:51] ERROR: Input images do not contain tile annotations. Use amfbrowser to annotate tiles before training.
```

Solution: the images in your test dataset do not have corresponding annotations. Label the images and then retry testing.

Error: inhomogenous array

```
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\d80105\code\root-fungal-colonisation\amf\amf_code\amfinder_load.py", line 54, in __init__
    self.x = np.array(x) if not isinstance(x, np.ndarray) else x
             ^^^^^^^^^^^
ValueError: setting an array element with a sequence. The requested array has an inhomogeneous shape after 2 dimensions. The detected shape was
(37392, 3) + inhomogeneous part.
```

Solution: in this case, you have likely triggered a training run where the annotations for images have different tile edges, e.g. one image has annotations enabled with 252 tile edge and another has annotations enabled with 126. Go and check each annotations file edge and make sure they all match the edge size of the model you are testing.

Error: CUDA error

```
    untyped_storage.copy_(self, non_blocking)
RuntimeError: CUDA error: device-side assert triggered
CUDA kernel errors might be asynchronously reported at some other API call, so the stacktrace below might be incorrect.
For debugging consider passing CUDA_LAUNCH_BLOCKING=1.
Compile with `TORCH_USE_CUDA_DSA` to enable device-side assertions.
```

Solution: in this case the CUDA driver has become corrupted, and you need to restart the application by closing and reopening the executable.

06.2.04 CONVERSION ERRORS

Error: Maximum supported image dimension is 65500 pixels when converting TIFs to JPGs.

```
Maximum supported image dimension is 65500 pixels
Failed to crop due to error: OSError: encoder error -2 when writing image file
```

Solution: the maximum supported size for JPG conversion in Python is 65500 pixels. As such, if any of your annotations in SlideViewer (or the TIF itself if you are converting the entire image) are larger than this, the tool will not be able to automatically convert them. In this case, you can either manually crop the image, or use PNG mode when converting your images, as these do not have a maximum size.

Error: No space left on device.

```
Failed to crop 449_5 due to error: OSError: [Errno 28] No space left on device
```

Solution: you have run out of storage space on your device. You need to clear space to be able to download further images.

Error: MemoryError

```
Failed to crop 548_2 due to error: MemoryError:
```

Solution: your machine has run out of RAM – this can happen when converting very large TIFs. You should close as many programs as possible and try to run again – if you still get this error, you need to use a machine with more RAM or crop your TIFs to have smaller annotations, as your machine does not have the capacity to load such a large image.

06.2.05 CALIBRATION ERRORS

Error: unable to allocate memory.

```
[10:44:16] 56 images in filtered dataset.
Traceback (most recent call last):
  File "C:\Users\d80105\code\root-fungal-colonisation\amf\amf_code\amf", line 144, in <module>
    main()
  File "C:\Users\d80105\code\root-fungal-colonisation\amf\amf_code\amf", line 137, in main
    AmfCalibrate.run(input_files)
  File "C:\Users\d80105\code\root-fungal-colonisation\amf\amf_code\amfinder_calibrate.py", line 364, in run
    cal_normalised_dataset = AmfLoad.CustomNormalisedDataset(x_cal, y_cal)
                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\d80105\code\root-fungal-colonisation\amf\amf_code\amfinder_load.py", line 54, in __init__
    self.x = np.array(x) if not isinstance(x, np.ndarray) else x
             ^^^^^^^^^^^
numpy.core._exceptions._ArrayMemoryError: Unable to allocate 75.1 GiB for an array with shape (423287, 3, 252, 252) and data type uint8
```

Solution: this is a known bug in the tool where during calibration, if the dataset is too large for your computer, then it won't be able to load the images. This bug should be fixed in a future version of the tool – for now you would either need to reduce the dataset size or use a machine with higher RAM.

07. ANNEX A – METRICS AND THEIR MEANING

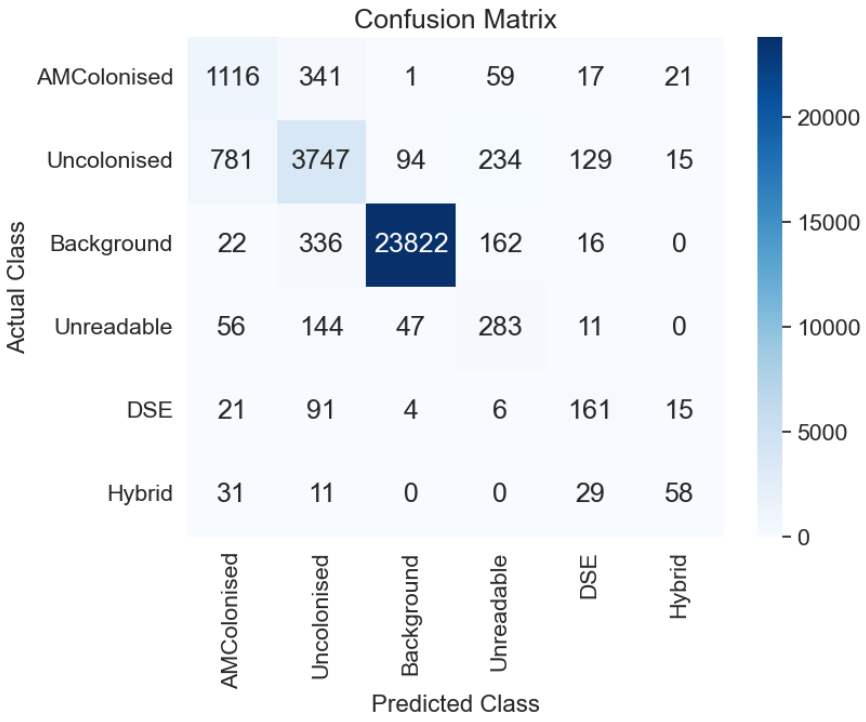
In this section, we introduce the metrics that are used to evaluate the performance of a model on a test set. An introduction to these metrics is also provided in [5].

To fully understand evaluation metrics, we must first grasp the concept of a **confusion matrix** and its key components.

A **confusion matrix** is a fundamental tool for assessing the performance of a classification model by comparing predicted and actual outcomes. It consists of the following elements:

- **True Positives (TP):** Correctly predicted positive instances. For example, if a tile is predicted as AM+ and is indeed AM+ in reality.
- **False Positives (FP):** Incorrectly predicted positive instances. For example, if a tile is predicted as AM+ but is actually N- in reality.
- **False Negatives (FN):** Incorrectly predicted negative instances. For example, if a tile is predicted as N− but is actually AM+ in reality.
- **True Negatives (TN):** Correctly predicted negative instances. For example, if a tile is predicted as N- and is indeed N- in reality.

An example confusion matrix for our case is shown below.



As can be seen above, a confusion matrix organises the true labels and predicted labels for analysis.

*(**Note**: It is important to note that for a class, say AM+, if the tile is predicted as anything but AM+, and it is AM+ in reality, it's a FN. This goes for all classes.)*

This matrix serves as the foundation for deriving various evaluation metrics as discussed below, ensuring a comprehensive model assessment.

07.1 ACCURACY

Accuracy can be calculated on both an overall and a per-class basis. We will first look into overall accuracy.

07.1.01 OVERALL ACCURACY

Accuracy quantifies the proportion of correct predictions across all classes in a classification model. This metric is mathematically defined as

$$Accuracy = \frac{Number\ of\ correct\ predictions}{Total\ number\ of\ predictions} = \frac{\sum_{i=1}^{n} TP_i}{\sum_{i=1}^{n}(TP_i + FP_i)}$$

Where TP_i represents true positives for class i, and FP_i represents false positives for class i.

The example confusion matrix presented yields an overall accuracy of 87.1%, calculated by dividing the **sum of diagonal elements** (29,227) by the **total number of samples** (33,549).

Despite its intuitive interpretation, accuracy presents significant limitations when applied to datasets with class imbalance. This occurs because the metric is **disproportionately influenced by the majority class, potentially obscuring poor performance on minority classes.**

Note that this overall accuracy is reported as "Accuracy" in the generic_metrics file provided by the test functionality.

07.1.02 PER-CLASS ACCURACY

Per-class accuracy provides a more granular evaluation by measuring the proportion of correct predictions for each individual class:

$$Accuracy_i = \frac{TP_i}{TP_i + FN_i}$$

Where TP_i represents true positives and FN_i represents false negatives for class i. *(**Note**: the denominator is the sum across all columns for a given class in the confusion matrix above)*

For example for the background class, $Accuracy_X$ = 23822/(23822 +336 + 94 + 162 + 16 + 0) = 97.5%.

07.2 PRECISION:

Precision quantifies the proportion of correctly identified positive instances among all instances predicted as positive for a specific class. Mathematically, per-class precision is defined as:

$$Precision_i = \frac{TP_i}{TP_i + FP_i}$$

Where TP_i represents true positives for class i, and FP_i represents false positives for class i. *(**Note**: the denominator is the sum across all rows for a given class in the confusion matrix above).*

For example, calculating the precision for the AMColonised class:

$$Precision_{AM+} = 1116/(1116 + 781 + 22 + 56 + 21 + 31) = 55.1\%$$

To evaluate the precision performance on an overall basis, we can calculate the **Macro-Precision which is the average of the precision value across all classes.**

07.3 RECALL

Recall measures the ability of a classification model to identify all relevant instances of a particular class. It represents the **coverage or completeness** of the model's predictions. Mathematically, **per-class** recall is defined as:

$$Recall_i = \frac{TP_i}{TP_i + FN_i}$$

Where TP_i represents true positives for class i, and FN_i represents false negatives for class i. *(**Note**: this is **identical to per-class accuracy!**).*

For example, calculating the recall for the AMColonised class:

$$Recall_{AM+} = 1116/(1116 + 341 + 1 + 59 + 17 + 21) = 71.8\%$$

To evaluate the precision performance on an overall basis, we can calculate the **Macro-Recall which is the average of the recall value across all classes.**

07.4 F1-SCORE

The F1 score addresses the inherent trade-off between precision and recall. While precision emphasizes the correctness of positive predictions, and recall focuses on comprehensive coverage of positive instances, neither metric alone provides a complete picture of model performance. The F1 score harmonises these

complementary aspects by calculating their harmonic mean, creating a balanced metric that is particularly valuable for imbalanced datasets.

07.4.01 PER-CLASS F1 SCORE

The F1 score for each class is mathematically defined as:

$$F1_i = 2\frac{Precision_i \times Recall_i}{Precision_i + Recall_i}$$

For example, calculating the F1 score for the AMColonised class:

$$F1_{AM+} = 2 \times 55.1 \times 71.8/(55.1 + 71.8) = 62.4\%$$

Similarly, the F1 scores for other classes are:

- Uncolonised: **77.4%** (2 × (80.0% × 75.0%)/(80.0% + 75.0%))
- Background: **98.5%** (2 × (99.4% × 97.7%)/(99.4% + 97.7%))
- Unreadable: **44.1%** (2 × (38.0% × 52.4%)/(38.0% + 52.4%))
- DSE: **48.6%** (2 × (44.2% × 54.2%)/(44.2% + 54.2%))
- Hybrid: **48.7%** (2 × (53.2% × 45.0%)/(53.2% + 45.0%))

07.4.02 MACRO-AVERAGE F1 SCORE

The macro-average F1 score calculates the unweighted mean of per-class F1 scores, treating all classes equally regardless of their frequency:

$$F1_i = \frac{1}{n}\sum_{i=1}^{n} F1_i$$

For the given confusion matrix:

$$Macro - F1 = (62.4\% + 77.4\% + 98.5\% + 44.1\% + 48.6\% + 48.7\%)/6 = 63.3$$

The macro-F1 score of 63.3% reveals an overall performance that accounts for both the precision and recall dimensions. The substantial variation in F1 scores across classes—ranging from 44.1% for the Unreadable class to 98.5% for the Background class—highlights the classification challenges posed by certain classes. This balanced metric effectively exposes performance disparities that might remain concealed when considering accuracy, precision, or recall independently.

07.5 NUMBER OF TILES BELOW CERTAIN THRESHOLD

The number of tiles below a certain threshold gives an idea about how many tiles the model is confident about. As described in the section Colonisation Metrics for a Prediction, we look for each tile at the maximum probability that is output by the

model. Then, given this max probability, this tile is counted towards one of the confidence thresholds buckets, where the borders range from 0.1 to 1.0 in steps of 0.1. The buckets are cumulative, meaning that all the tiles that are in the bucket with threshold 0.2 are also in the bucket with threshold 0.3 and above. As an example, we consider a tile, where the model predicts a class with max probability of 0.42. This probability is lower or equal to 0.5. Hence, the tile counts towards the buckets “Tiles with confidence <= 0.5”, “Tiles with confidence <= 0.6”, ..., “Tiles with confidence <= 1.0”. That way one gets an idea of how many tiles it is worth looking at for manual labelling.

We can see the output of the number of tiles below a threshold for test – this is reported as a percentage, whereas for predictions it is reported as an absolute value.

	F	G	H	I	J
	Tiles with confidence <= 0.6 in %	Tiles with confidence <= 0.7 in %	Tiles with confidence <= 0.8 in %	Tiles with confidence <= 0.9 in %	Tiles with confidence <= 1.0 in %
9	3.87064	6.587	9.67034	13.9801	100

07.6 CLASS METRICS AFTER MANUAL LABELLING

As already described above, one has an idea about how many tiles the model is how confident about with its prediction. The natural question to ask is then, if the tiles below are certain confidence threshold would be manually labelled by a researcher, how would the model performance change. This is exactly what class metrics after manual labelling tells us. It converts every annotation below a threshold to its correct annotation and then recalculates based on that the metrics mentioned above. The values for the threshold are again 0.1, 0.2, ..., 1.0.

We can see the output of one such threshold csv below – this corresponds to the “threshold_0.8.csv” file, which means these are the model metrics if one were to convert every tile to the correct annotation with confidence less than or equal to 0.8.

class_name	Accuracy	Precision	Recall	F1 Score
AMColonised	0.862379	0.790218	0.862379	0.824723
Uncolonised	0.899	0.903336	0.899	0.901163
Background	0.989285	0.997186	0.989285	0.99322
Unreadable	0.787431	0.694943	0.787431	0.738302
DSE	0.802013	0.737654	0.802013	0.768489
Hybrid	0.697674	0.849057	0.697674	0.765957
Average	0.83963	0.828732	0.83963	0.831976

08. ANNEX B – TRAINING BEST PRACTICES

- When defining training and test datasets, an 80%-20% split is a common and sensible choice. Make sure that training and test data sets are mutually exclusive in order to get a fair and realistic performance estimate on the test dataset.
- Retraining of the model only is advisable if a significant loss of performance in predicting classes for new images is detected. To achieve an improvement of performance in retraining a good rule of thumb is that the number of images for retraining should be at least about 25% of the size of the original training dataset – in addition, the images used for retraining should look different, in the sense that they add new information/features for the model to learn.
- The default values for each hyperparameter are based on the best performing model for Efficient Net, which is provided in trained_networks as efficientnet_252_6class_D2.pth. We should note here that these are a good starting point for retraining this model but may not generalize well if you are training other types of models, e.g. ResNet, ResNeXt or CNN1. We have provided the best hyperparameter values we have seen for both 126 and 252 tiles for each of these model types below (based on maximum Macro F1 score) – these should give you a good starting point if you want to train other types of models.
- Parameters of the "Adam Optimiser" should be close to beta1=0.9, beta2=0.999, since they are good choices in general. It is only advised to tweak them when large scale hyperparameter sweeps are carried out, which should not be necessary when retraining on new images.
- Since most retraining scenarios will be fine-tuning for EfficientNet, it is advisable to start with low learning rates (approx. 5e-6). Track the validation loss for each epoch, which should decrease monotonically until it reaches a point where it will rise again.
  - If the validation loss is decreasing only minimally, try **increasing** the learning rate.
  - If the validation loss is fluctuating a lot between epochs, try **decreasing** the learning rate.
- Set the patience_r (currently 3) and patience_e (currently 10) parameters, which control automatic learning rate reduction, if a rise in validation loss is detected, and early stopping, if no model improvement is detected anymore before the maximum number of epochs is reached in order to save resources, respectively.

We note that the values from the hyperparameter sweep for EfficientNet were as follows, where the 252 hyperparameters correspond to the am_252_efficientnet.pth model in trained_networks, and the 126 to am_126_efficientnet.pth.

Tile Size	Balance Factor	Batch Size	Learning Rate	Adam Beta 1	Adam Beta 2
252	1.2490	32	0.0000042	0.90645074	0.9863903

126	1.0274	32	0.0000043	0.9076909	0.9909789

Moreover, the best parameters for ResNeXt for 126 and 252 models were as follows.

Tile Size	Balance Factor	Batch Size	Learning Rate	Adam Beta 1	Adam Beta 2
252	1.1446	32	0.000004734	0.893003810	0.968032358
126	1.2456	32	0.000000521	0.895399930	0.994679281

Similarly, for ResNet:

Tile Size	Balance Factor	Batch Size	Learning Rate	Adam Beta 1	Adam Beta 2
252	1.2444	32	0.000004846	0.910678232	0.997845726
126	-	-	-	-	-

And finally, for CNN1:

Tile Size	Balance Factor	Batch Size	Learning Rate	Adam Beta 1	Adam Beta 2
252	1.1742	32	0.00009188	0.90678225	0.95423917
126	0.9383	32	0.000079789	0.911281288	0.975138196

You can use these values as starting points for (re)training each type of model. A detailed analysis of every hyperparameter sweep can be found in the included excel sheets [2] and [3].

09. ANNEX C – ACTIVE LEARNING

Active learning is a machine learning paradigm that aims to achieve high accuracy with minimal labelling effort by selectively querying the most informative unlabelled instances for annotation. Unlike traditional supervised learning approaches that rely on large volumes of labelled data, active learning algorithms strategically identify which data points would most improve model performance when labelled. This approach is particularly valuable in domains where obtaining labelled data is expensive, time-consuming, or requires expert knowledge.

The active learning process typically follows an iterative cycle: (1) train a model on the currently available labelled data, (2) apply an acquisition function to evaluate the informativeness of unlabelled instances, (3) select the most valuable instances for annotation, (4) obtain labels from an oracle (typically a human expert), and (5) update the labelled dataset and retrain the model. This process continues until a predefined stopping criterion is met, such as reaching a target performance level or exhausting the labelling budget.

Various acquisition strategies have been developed to identify informative instances, including uncertainty-based sampling, diversity-based methods, expected model change, and Bayesian approaches. Among these, Bayesian methods have gained significant attention due to their principled treatment of model uncertainty, particularly in deep learning contexts.

09.1 BAYESIAN ACTIVE LEARNING BY DISAGREEMENT (BALD)

Bayesian Active Learning by Disagreement (BALD) is an information-theoretic approach that selects instances which maximize the mutual information between model predictions and model parameters. Conceptually, BALD identifies samples where the model is uncertain about its predictions, but this uncertainty is expected to be reduced significantly with additional information about the model parameters.

Mathematically, BALD quantifies the mutual information $I(y, \omega \mid x)$ between the prediction y and model parameters ω, conditioned on the input x:

$$I(y, \omega \mid x) = H(y \mid x) - E[H(y \mid x, \omega)]$$

Where $H(y \mid x)$ represents the entropy of the predictive distribution, and the second term is the expected entropy of the predictions across different model parameters.

In practical implementations with Bayesian neural networks or ensemble methods, BALD can be approximated using Monte Carlo dropout (dropout randomly deactivates neurons during training to prevent overfitting and approximate Bayesian inference.).

By performing multiple forward passes with dropout enabled during inference, we generate T different predictions for each input x:

$$I(y, \omega \mid x) \approx H(1/T \sum p(y \mid x, \hat{\omega}\_t)) - 1/T \sum H(p(y \mid x, \hat{\omega}\_t))$$

BALD effectively identifies instances where the model predictions vary significantly across different dropout masks, indicating regions of high epistemic uncertainty that would benefit most from additional labelled data.

09.2 BATCHBALD

While BALD offers a principled framework for active learning, it selects instances independently without considering the potential redundancy among them. This becomes problematic in batch acquisition scenarios where multiple instances are selected simultaneously, as commonly required in practical applications to amortize the cost of retraining.

BatchBALD extends BALD to address this limitation by jointly selecting a batch of points that maximizes the mutual information between the model parameters and the joint predictions for all points in the batch. For a batch B = {x_1, x_2, ..., x_k} with corresponding labels Y = {y_1, y_2, ..., y_k}, BatchBALD computes:

$$I(Y, \omega \mid B) = H(Y \mid B) - E[H(Y \mid B, \omega)]$$

Computing this quantity exactly becomes intractable for large batch sizes due to the exponential growth of possible label combinations. BatchBALD implements an efficient greedy approximation algorithm that incrementally builds the batch by selecting points that maximize the conditional mutual information given the previously selected points:

$$x_i = argmax\, I(y, \omega \mid x, B_{i-1})$$

Where B_{i-1} represents the batch constructed in the previous $i - 1$ iterations, and y is the label corresponding to input x.

Empirical evaluations demonstrate that BatchBALD outperforms other batch acquisition methods, particularly in challenging scenarios with redundant data or class imbalances. By accounting for correlations between the uncertainties of different data points, BatchBALD avoids selecting redundant examples that provide similar information about the model parameters, resulting in more efficient use of the labelling budget and faster convergence to optimal performance.

10. ANNEX D - SEMI-SUPERVISED TRAINING

The semi-supervised training functionalities in the tool are based on FixMatch and were implemented by Cambridge Consultants in a previous version of AMFinder. In this version of the tool, we make these functionalities available via the UI of the tool – however, it is worth noting that further development work may be needed to assess the efficacy of this approach for training the tool.

The disclaimer provided by Cambridge Consultants is provided below.

"This is an unofficial PyTorch implementation of FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence, extended by Cambridge Consultants. The official Tensorflow implementation is here.

This project is an extension provided by Cambridge Consultants and is not associated with the original creators of the FixMatch model. All modifications and improvements made by Cambridge Consultants are intended to enhance functionality on the dataset provided by Kew Gardens."

Semi-supervised training is used to artificially label tiles in an incomplete training dataset, in order to boost the available data used for training the tool. When running FixMatch, the tool will consider labelled and unlabelled training data, and will augment the labelled data. It is worth noting that every image still needs to have annotations associated with it to load correctly, but these annotations do not need to be complete. Further details can be found in the linked paper above.

11. ANNEX E – EXPLANATION OF CONFIDENCE INTERVALS

In our classification problem, it's important to not only predict which class a tile belongs to, but also to understand how confident the model is in these predictions. In order to impose this uncertainty measure, we need to both calibrate the model's predictions and use statistical techniques to estimate the reliability of those predictions for any metric in an image.

Before we delve into the details, we define the general term 'metric' in the section as any of the following values (reported below for AM – the metrics are similar for ErM):

- am_colonised_percentage
- dse_percentage
- total_colonisation_percentage
- AMColonised
- Uncolonised
- Background
- Unreadable
- DSE
- Hybrid

For the following, it is important to highlight that the above metrics are all derived based on **converting the max probabilities to class labels** and are then calculated based on these labels. For the calculation of the confidence intervals, we change the perspective and rather than considering the max probabilities, we look at the whole distribution output by the model for a prediction of a tile. With this in mind, we can continue with the model calibration procedure and derivation of confidence intervals.

As a first step, we delve into the model calibration. Our model, after training, might be good at predicting classes but could be **overconfident** or **underconfident** about those predictions. Calibration helps adjust the model's probability scores to better reflect true confidence levels. For example, if the model predicts a tile to be AMColonised with 100% confidence, then we would expect that there is no uncertainty involved. However, if it only predicts it with 80%, we expect 80 out of 100 such samples to be AMColonised. Hence, calibration tries to match the model's confidence to its accuracy. To achieve this, we use a method called temperature scaling. It adjusts the logits (raw model outputs before the softmax layer) using a single scaling parameter, known as the "temperature." The logits are divided by this temperature before applying the softmax function to convert them into calibrated probabilities. If the temperature is greater than one, it reduces the confidence of the predictions. Conversely, a temperature less than one increases the confidence. This adjustment helps ensure that the predicted probabilities better reflect the true likelihoods observed in the data provided.

Regarding the output of the calibration procedure, we have three files. The final predicted calibrated probabilities for each tile across an image are always saved to calibrated_probs.csv. The temperature value can be found in Model_temperature_value.txt and the results of the optimization procedure to get the temperature value are saved to Temperature_scaling_results.csv.

Once we've calibrated the model, we approach the task of estimating metrics and the inherent uncertainty across an image. Instead of relying on just the maximum probability for a given tile, we look at the whole probability distribution for a tile to understand the uncertainty in the predictions. To get an estimate of one of the metrics within the image, we run through all the tiles in the image and sample for each tile the predicted class according to the probability distribution given by the calibrated model for that tile. By conducting numerous sampling runs across these tiles, we generate multiple estimates of the considered metric within the entire image. These repeated estimates form a distribution, which helps us understand the range and variability in our model's predictions.

Once we collect these estimates, we calculate the mean to find an average representation, along with the standard deviation to measure the spread of estimates around this mean. The confidence interval is then constructed using this mean and standard deviation, by considering the mean plus or minus **twice the standard deviation** to account for most of the variation. Hence, we get three values for each metric listed above:

- {Metric name}_MEAN
- {Metric name}_LowerConfidence = Mean – 2 * standard deviation
- {Metric name}_UpperConfidence = Mean + 2 * standard deviation

This interval offers a statistical range where we expect the true proportion of each class to fall, effectively quantifying how confident we can be in our model's predictions, despite the uncertainties inherent to each individual tile's prediction. Eventually the mean and the upper and lower bounds for the confidence intervals are the three additional metrics given per colonisation metric.

12. ANNEX F – ADDING CONTEXT TO PREDICTIONS

The default approach to calculating predictions in Mycorrhiza Finder involves segmenting high-resolution specimen images into small, fixed-size tiles (typically 252x252 pixels for AM and 126x126 for ErM). Each tile is processed in isolation by a deep learning model, which assigns a predicted class label based on the tile's local features. The class with the highest confidence is selected as the final label unless overridden by human annotation. This method, while efficient, ignores spatial dependencies and can produce inconsistent labels in regions of ambiguity.

In contrast, human experts often rely on the broader visual context to interpret ambiguous regions—such as surrounding structures or colour gradients that span multiple tiles. This discrepancy motivates the integration of contextual information into automated classification systems. The goal is to bridge the gap between human and machine interpretation by improving tile-level prediction consistency and accuracy using overlapping tiles and contextual aggregation methods.

We incorporate context by having four overlapped tiles for every concerned tile. The illustration below depicts this.

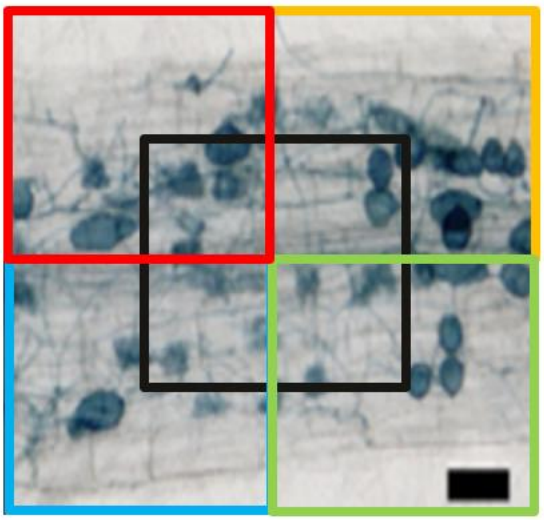


Figure 1: Example of Diagonal Overlap: tiles overlapping diagonally to capture broader spatial context across neighboring regions. The black tile is the central tile; red, blue, yellow, and green tiles show overlapping neighbors.

Diagonal overlap has the overlapping regions along diagonal directions, ensuring that each tile shares content with its neighbours both vertically, horizontally, and diagonally. This method provides rich contextual coverage by including corner-adjacent information, which can be beneficial for detecting patterns or objects that span across the diagonal and the cardinal boundaries of the image. Based on empirical evidence, we have discovered that 75% overlap provided the best results when

considering contextual information in the predictions. The picture below shows 75% overlap for visual appreciation.

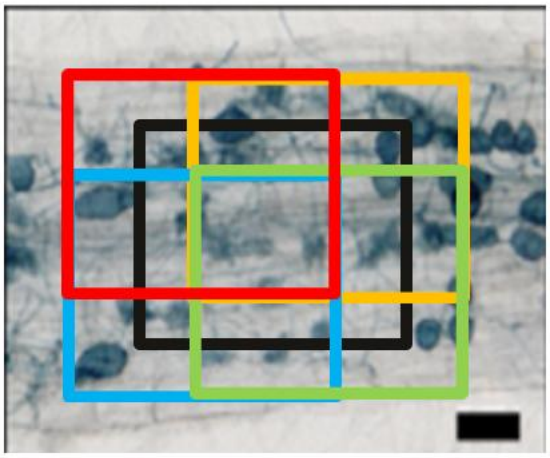


Figure 2: Contextual tiles with an overlapping amount of 75%

After inferring the contextual tiles along with the central one, we need to find ways to aggregate this information to re-assess the label of the central tile and change it if necessary. We use majority voting method for this where the class with the maximum number of votes is chosen as the label of the central tile. (Note: For a given tile, we still proceed with the choosing the class with the highest probability prior to applying max-voting). The following illustration explains this with an example:

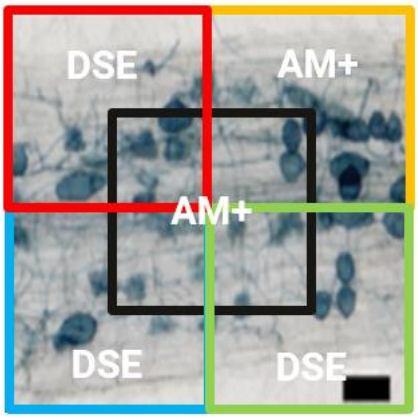


Figure 3: Example of inferring 4 contextual tiles in addition to the central tile. In this case as DSE has the highest number of votes (3), the central tile will be relabeled from AM+ to DSE.

13. REFERENCES

[1] E. Evangelisti et al., Deep learning-based quantification of arbuscular mycorrhizalfungi in plant roots, New Phytologist (2021), 232, 2207–2219

[2] Adv_Model_Analysis.xlsx – all hyperparameter sweeps conducted on dataset 2.

[3] Full_CNN1_Analysis.xlsx – all hyperparameter sweeps conducted on dataset 1.

[4] AMFinder_classification_guidelines_AM – guidelines for labelling an AM image.

[5] 2024_11_20_Kew_Performance_Metrics – an overview of metrics used to track model performance.

[6] ErM_classification_guidelines_20.02.2025 – guidelines for labelling an ErM image.